

Gilbert the Godslayer

Jack Brosnan

Centre Number: 56120

Analysis

Introduction:

My idea is to make a top-down shooter roguelike with the working title Gilbert the Godslayer, as the player is playing as a regular human man, Gilbert, who gets sent to slay Greek Gods. The player will spawn into an ancient Greek battlefield with the ability to move, throw hammers, and dash, launching them forward and making them temporarily invincible, allowing them to dodge enemy attacks and reposition as to not feel overwhelmed. These basic features will make for a strong foundation for the game which I can then build off of.

After some time, enemies will spawn around them, and try to attack. Some will fire projectiles, some will just run at them. Different enemies have different speeds, movement patterns, health, damage etc, as well as a variety of visual designs based around Greek mythology, adding variety to combat to engage the player.

Defeating an enemy will give exp, and getting enough exp will provide a choice between 3 stat-buff type power up. These may be health buff, damage buff, speed buff etc. They'll have different rarities, with rarer ones being better, giving players a reason to keep on collecting these power-ups - wanting to collect these rare, powerful effects.

There will also be power-up drops spawning randomly throughout the map, and when you defeat a stronger enemy. These ones randomly change the effect of your hammer. It may be that, whenever you shoot, there's a 10% chance that the hammer sets on fire, or a 20% chance it freezes the enemy. However, once again, there are rarities. Rarer ones will do things like spawn a T-rex which attacks enemies, or turn an enemy into a zombie which fights for you, or cause a massive explosion. I think the rarer, unique power ups, such as the T-rex, will be what sets this game apart from its competitors - people will want to find these wacky, interesting power-ups

After a certain amount of time, a boss spawns, which has a lot of health. If the player overcomes the challenge and defeats it, they win, and are able to watch a short cutscene bringing the game to a satisfying conclusion. If at any point they lose, they have the option to start another run immediately, hopefully helping them overcome any annoyance that comes from losing, and giving an addictive feel to the game.

The game takes inspiration from many games which I have been playing recently, and really enjoy, including Hades, the Binding of Isaac, and especially Enter the Gungeon. These games are a part of the roguelike genre, which is defined as a game where you lose everything if you die, giving it a run-like structure. These games can be immensely fun and intense, and also allow people short on time to play for 20 minutes before putting it down. The roguelike genre has become very popular in recent years, but I think it is missing a game which fully emphasises the randomness which makes the genre appealing, whilst combining it with chaos to become a very fun game.

The game engine I'm currently thinking of using is Unity, which uses C#, due to its wide range of tools for 2D game design, and intuitive UI, which will be good for me as I haven't done any game design before.

Research:

The first game that I took inspiration from was Enter the Gungeon, due to it having similar gameplay. I think it's a fair comparison due to both being top-down roguelikes with a focus on ranged combat.



1. Official Artwork, taken from Playstation Store <https://www.playstation.com/en-gb/games/enter-the-gungeon/>



2. Screenshot of a boss battle, taken from the Play Critically review <https://playcritically.com/2021/05/09/enter-the-gungeon-review/>



3. Screenshot of the game, from the Shacknews review <https://www.shacknews.com/article/94140/enter-the-gungeon-review-high-caliber>

Things I would like to take inspiration from for my project	Things I would like to avoid for my project
The cartoony pixel-art visuals are perfect for my game, both being easier for me to design, and allowing me to aim the game towards a younger audience as opposed to adults, due to it being full of appealing character designs, and avoiding anything graphic. The general movement system would work very well for my game. Both games feature walking, shooting, and a dash which grants invincibility. The walking of Enter the Gungeon allows your movements to be precise, perfect for dodging the many bullets being fired at you, and the aiming isn't tied to the direction you're moving, allowing you to avoid enemies whilst you shoot back. My game would feature more close range enemies, so similar principles would be useful, in my case for running away instead of dodging bullets. The dodge roll would be a great inspiration for my dash. It launches you forward, allowing you to become invincible for a second to avoid attacks, adding an element of skill due to the precise timings needed. The dodge is also very satisfying and very well choreographed visually to the player, allowing them to know when they're invincible, which is important for avoiding	Run variety in Enter the Gungeon mostly comes from collecting different guns which shoot differently. I, however, think it'd be better to stick with the one weapon, which simply shoots one projectile when you click. I want my run variety to come from power-ups, which should hopefully get quite complex, however adding in different weapons may completely overcomplicate it, and overwhelm the player with all the different options. It may also limit the design of my power-ups, as I'd have to make sure they work for all guns, and if an idea I have is incompatible with a certain weapon type I wouldn't be able to do it. Enter the Gungeon also implements an ammo system, where you can only use a weapon a certain amount of times before having to reload, and if you use it too much, the weapon runs out of bullets and can't be used further. I think it would be best to keep this out of my game, as the special effects on my weapons happen randomly. This would encourage you to shoot as many hammers as possible, to increase the chance that the effect occurs. An ammo system would discourage you from throwing out as many hammers as possible, and so you wouldn't see these effects as often. It would also add a resource management element to the game, which can cause stress to the player, which is something I

the frustrations of being hit when you didn't think you would be.

I also like that Enter the Gungeon doesn't take itself too seriously, and doesn't have a theme with the items. It is full of jokes, be it weapon names being puns, or the comedically massive quantity of references to guns all over the game. Weapons range from guns to lasers to a mailbox to the letter 'r'. This approach will be used in my power-ups, which should have this random nature, to hopefully make the player laugh. The game is marketed to a younger audience, so this would be a way to keep them engaged.

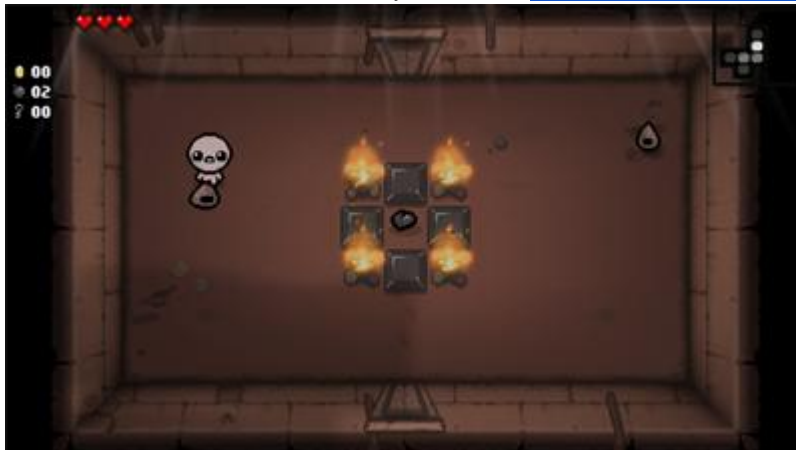
want to avoid when the game will already be fast paced and intense.

The levels are split up into rooms, which you are confined to until all enemies have been dealt with in that room. I'd rather have an open map, due to an increased number of enemies that would become overwhelming in a small space, turning the game frustrating.

The next game that I looked at for inspiration was The Binding of Isaac. I wanted to see what I could take from this game specifically due to it being a top-down roguelike that contains lots of good things that I like about it that I want to take inspiration from, whilst having a lot of flaws that I would like to avoid.



1. Official Artwork, taken from Playstation Store <https://www.playstation.com/en-gb/games/the-binding-of-isaac-rebirth/>



2. Screenshot of gameplay from LaunchBox Games Databox <https://gamesdb.launchbox-app.com/games/images/80151-the-binding-of-isaac-afterbirth>



3. The sprite sheet for all items in the Binding of Isaac, which I got from The Spriters Resource https://www.spriters-resource.com/pc_computer/bindingofisaacrebirth/sheet/188967/



4. A pause menu found on Steam <https://steamcommunity.com/app/250900/discussions/0/133258092255867313/>

Things I would like to take inspiration from for my project	Things I would like to avoid for my project
One of the biggest things I will take inspiration from is the wide range of items. There are 717 collectible items in The Binding of Isaac, all of which have unique effects. This ranges from simple changes like a damage buff, to completely changing how the game is played, such as the 'Mom's Knife' item changing your projectile based attack into a melee one. This is the type of variety I want in the power-ups of my game - some being simple damage buffs, and some completely changing how you play. Whilst I doubt I'll be able to reach the sheer number of items that Isaac has, it can still serve as an aspirational goal to even come close to the amount of variety it has. Another thing I like about The Binding of Isaac is the projectile Isaac fires. At the start of the game, he shoots a single tear, doing a bit of damage. However, collecting items changes it massively into being far more interesting, which is the ethos I want in my item design, which will also have the main projectile be a single shot, in this case a hammer. As seen in the pause menu, the UI is very simple, and thematic for the game, being designed to look like it was scribbled onto a few bits of parchment which Isaac carries around with him. It simply shows your stats, options, a resume button, exit button, and your items, all laid out clearly and simply. It can't be easier to understand, which is something I'll want if I'm appealing to teenagers, who may be adverse to too much reading.	The setting is completely inappropriate for my game. It is set under Isaac's house, but is made to be as disgusting as possible, featuring blood, flesh, and excrement everywhere. While this adds some shock value for an older audience, if I'm aiming the game to under 18s, I definitely need to avoid this as there's a good chance they wouldn't be allowed to play my game with this setting. Each room is very small and claustrophobic, which wouldn't work in my game due to a high enemy count which would quietly cause it to get full and become impossible to dodge. You can only shoot a projectile in the 4 cardinal directions. In my game, where there will be a lot of enemies all over the place, that'd make it very difficult to hit enemies, and force you to have to move strangely just to aim, making the game feel tedious. It is also a very difficult game, and I don't want my game to be too hard, as a teenage audience may get frustrated with the game and quit if they keep on losing. Whilst there is a wide range of items you can collect, there is no in game explanation for what they do. For my game, the player might find this confusing, and I want to make sure they have enough information as they could need to avoid creating a frustrating experience.

The final game I took inspiration from is Hades. It is yet again a top-down roguelike, similar to my game, but with massive gameplay differences, letting me know what I don't want, but also giving me ideas for things not commonly found in other, more standard top-down shooters.



1. Official art, taken from Supergiant Games' website <https://www.supergiantgames.com/blog/hades-faq/>



2. A screenshot of gameplay, taken from Supergiant Games' website <https://www.supergiantgames.com/games/hades/>



- The boon (powerup) selection screen, taken from Rock Paper Shotgun <https://www.rockpapershotgun.com/hades-boons>

Things I would like to take inspiration from for my project	Things I would like to avoid for my project
I'd love to use the layout of the boon selection screen for my power-up selection screen. It lays out 3 options very neatly and clearly, with a short simple explanation of everything you need to know. It also gives the rarity, which is important for my players to know if they've found something special, making sure they don't miss out. Off to the side, it also gives an explanation of any effects that would be inflicted, which makes sure the main part doesn't get crowded, but is giving you all the information you need. It also has a similar setting - both being Greek mythology. Whilst this one is set in the Underworld, whereas mine would be above ground, the enemies and characters are all from Greek mythos, which I would also like to use.	The way the power-ups work are more restrictive than I want. In Hades, you can have one effect on your weapon, one on your dash etc, whereas I'd like to encourage you to stack as many power-ups as possible, as I think that can cause a chaos which would be appealing to a teenage audience. Hades is also technically a rogue-lite rather than a rogue-like, meaning that, despite it still being run-based, resetting if you lose, there is permanent progression that carries on, in Hades case being currencies like Darkness, which you can spend to become more powerful, making the game easier. I want my game to be more like something you can hop on, do a run or two, and then get off without anything carrying over, as a lot of teenagers may not have the time to put into a long game with permanent progression because they are bogged down with studies. This is seen in their affinity for online games, which once again don't really have permanent progression.

Stakeholders:

My initial target audience for this game are casual gamers between the ages of 12-18. The game is designed to be fast-paced and intense, which would keep this audience engaged, despite a shorter attention span. Runs also shouldn't take too long, allowing it to be played at any point when someone has a few minutes of free time. This works well for a 12-18 audience who are just casual gamers, and may not have too much free time due to increased amounts of homework and studying that they must do.

The game would be cartoony, and avoid anything graphic, allowing it to be advertised to a younger audience, however may be too intense and violent for anyone under 12. The game should be challenging, as a player of this audience would want it to be difficult, otherwise they may get bored of it, however it wouldn't be so hard that it becomes frustrating, so that a player can consistently win once they get good, as a casual gamer may not want to spend ages trying repeatedly just for a single win. The Greek Mythology theme may also help attract this audience, as they have grown up reading Greek myths in school, and books like the Percy Jackson series by Rick Riordan.

Stakeholder Survey:

To see how the stakeholders react to my idea, I sent out a survey. It features these questions:

How much do you enjoy playing video games?

How much do you enjoy top-down shooter games?

What art style would work best for the game?

What atmosphere would be most appealing for this game?

How difficult would you want the game to be?

How long would you like a run to last?

If you could only have one, would you prefer a lot of enemy types, or a lot of power-up types?

Would the game be improved if it recorded your best score (most exp collected)?

Do you like the idea of a Greek Mythology Setting?

How close should the enemy designs stick to this setting?

How much do you like the sound of this game?

When I have a functional build of the game, do you want to test it and do another survey later?

- The first 2 are just to get an idea of who is taking the survey, and how much they enjoy gaming in general.
- It then gets to art-style. I hadn't had my heart set on a specific art style yet, so I wanted to see what people said, and confirm my suspicions that a 12-18 audience would enjoy a not-realistic art direction.
- The general atmosphere was another thing I needed input on. Whilst I suspect under 12s want a brighter atmosphere, and adults would prefer a grittier one, I wasn't sure what would appeal most to the 12-18 audience.
- Difficulty was something I thought it would be good to get input on, as it is very easy to make the game too easy or too hard, and I needed an idea of where to aim for.

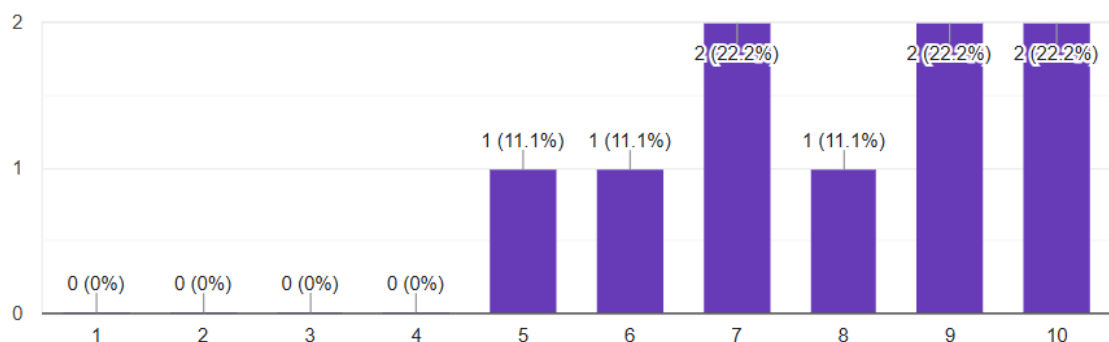
- Run length was another thing I wanted input on. I knew I wanted them to be short, to fit in with a teenager's tight schedule, however I didn't know how short I wanted them to be, as 'short' can range from a quick 3 minute run to around 15 minutes.
- Because I am a single developer who probably won't have too much time to work on this, I also wanted to know where's best to focus the majority of time on after I get the basics of the game down. Should I focus on enemies or power-ups?
- I also had the idea of adding a high-score mechanic based on the amount of exp you collect. I wanted to see if that was something the stakeholders would care about and if it would improve the game, as if they seem excited about the idea, it could improve replayability.
- My original idea for the game was a Greek mythology setting, as I thought that could be something that would appeal to a teenage audience. I wanted to see if my hypothesis was correct so I added that as a question, too.
- If people enjoyed the setting, I also wanted to know how accurate the enemies should be to the setting. They may want every design to be fully true to the origins, or they may be okay with me taking liberties with the designs.
- Lastly, I wanted to see if the whole game appeals to my original target audience, or if I had made a mistake and should change it. This was arguably the most important question in the survey for me.

After some time, the results came in:

How much do you enjoy playing video games?

[Copy chart](#)

9 responses

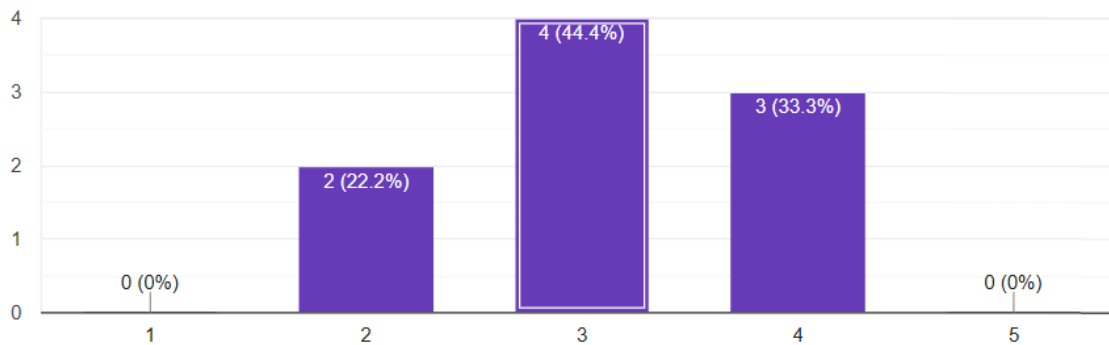


Question 1 shows that, as I expected, everyone who was in my target audience had a favourable opinion of video games, so the information I get from my survey should be valuable for creating a video game.

How much do you enjoy top-down shooter games?

 Copy chart

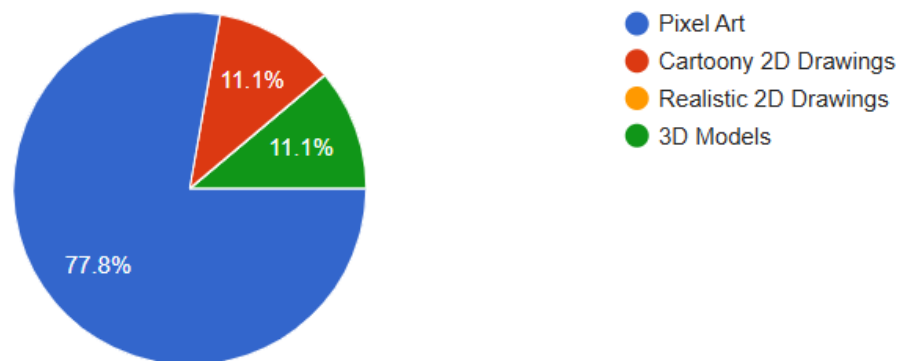
9 responses



The results of the second question showed that none of the stakeholders I surveyed actively hated top-down shooters, however none of them loved them either, ranging from slightly negative to slightly positive. This would be useful for once I complete the survey on a beta release of my game, to see how they'd compare my game to others in the genre, as well as being able to fix the gripes they have with games in this genre.

What art style would work best for the game?

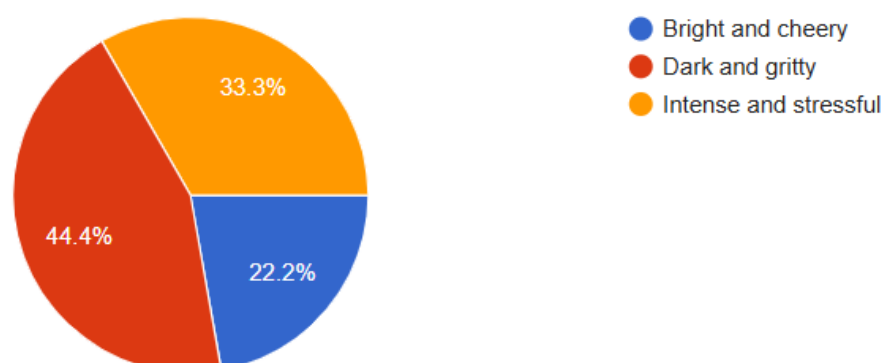
9 responses



For the art style, the vast majority of people wanted pixel art.

What atmosphere would be most appealing for this game?

9 responses

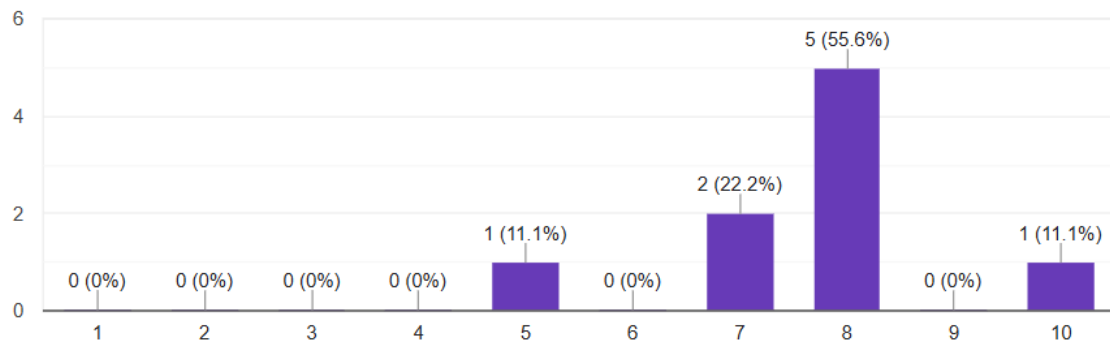


For the atmosphere of the game, most people wanted it to be dark and gritty, or intense and stressful. These make sense as the target audience will be in the transition of child to adult, so making games with a less childish atmosphere would come across as appealing. This suggests I could set it in a cave or dungeon, and have a lower light level to the game.

How difficult would you want the game to be?

[Copy chart](#)

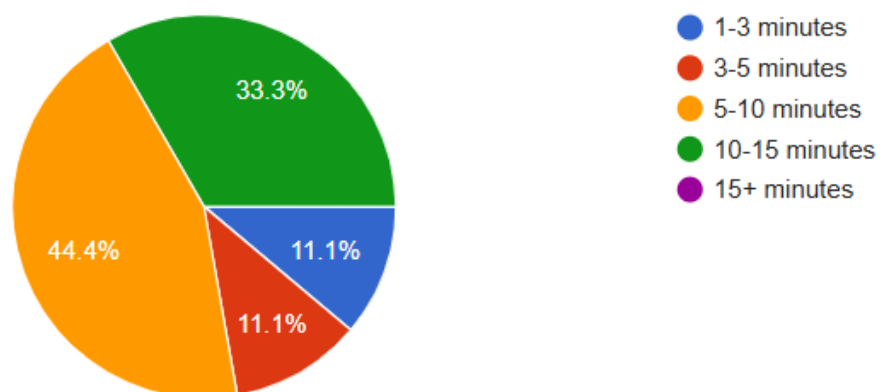
9 responses



In terms of difficulty, most people wanted it to be reasonably challenging, with the majority putting it as an 8. This suggests I should put focus on balancing the game to make it fair, whilst it being quite difficult to get a broken game-destroying run.

How long would you like a run to last?

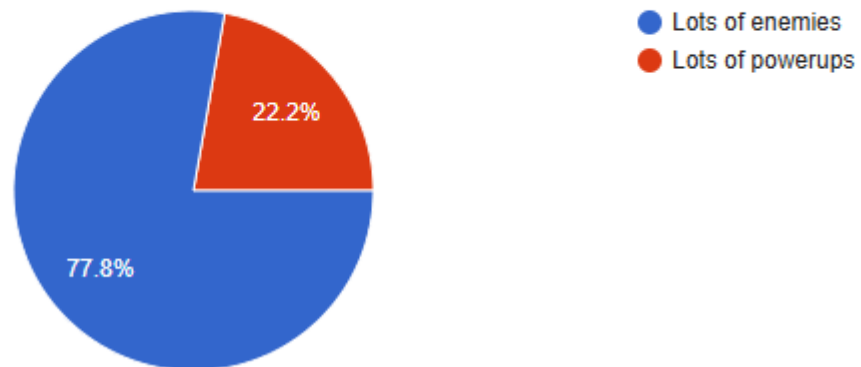
9 responses



The poll for run length showed that most people wanted a run to last between 5 and 10 minutes, or between 10 and 15 minutes. The average of this poll can be calculated by $(2+4+4(7.5)+3(12.5))/9$, which comes out as 8.2 minutes. This means I should aim for a run to be around this time. As I want it to be that the boss spawns after a constant amount of time, I could have it spawn at 6 minutes, and take around 2 minutes to defeat, averaging 8 minutes per run.

If you could only have one, would you prefer a lot of enemy types, or a lot of power-up types?

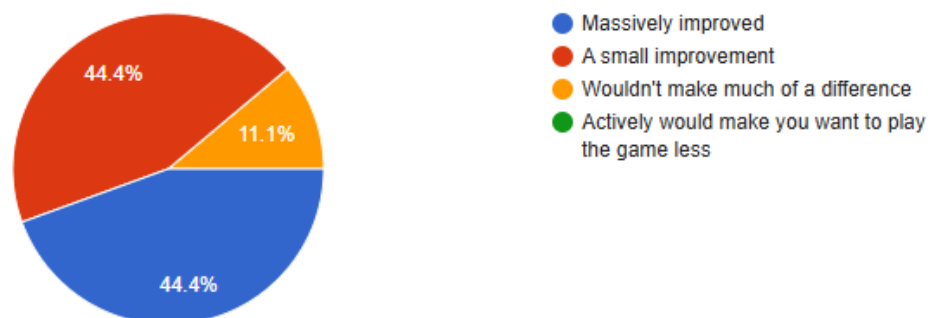
9 responses



The vast majority of people wanted lots of enemies, suggesting I should create a small group of useful power-ups and focus more on creating interesting combat experiences with a diverse cast of enemies.

Would the game be improved if it recorded your best score (most exp collected)? [Copy c](#)

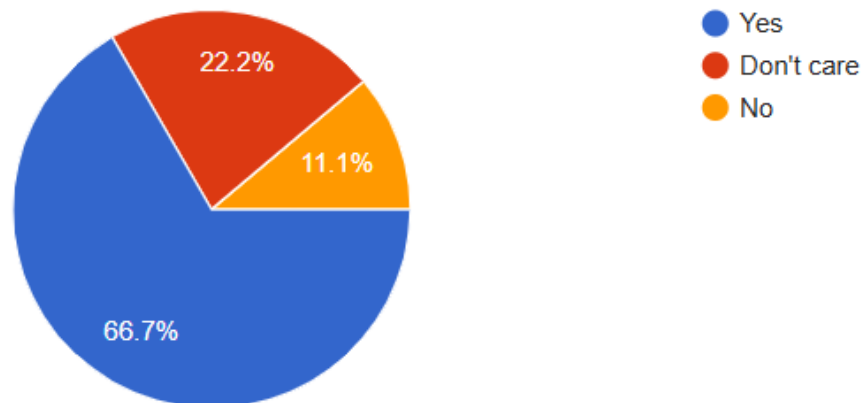
9 responses



People seemed generally positive towards the idea of a high score system, however over half didn't think it would be a massive improvement to the game. This suggests it is something I should add further down the line if I have time, but isn't an essential feature.

Do you like the idea of a Greek Mythology Setting?

9 responses

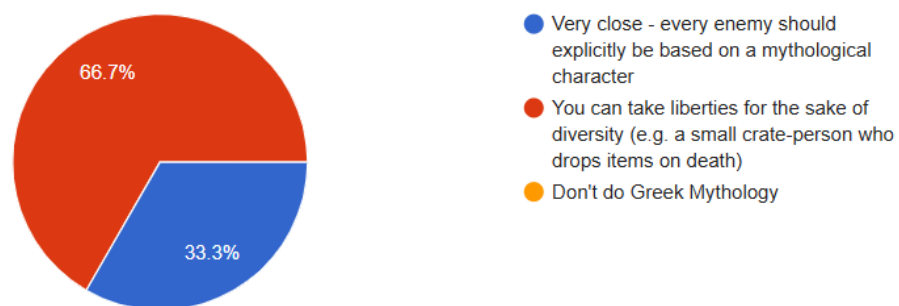


This confirmed my suspicions that my stakeholders would enjoy the Greek mythology setting, and it is the route I should go down.

How close should the enemy designs stick to this setting?

[Copy chart](#)

9 responses

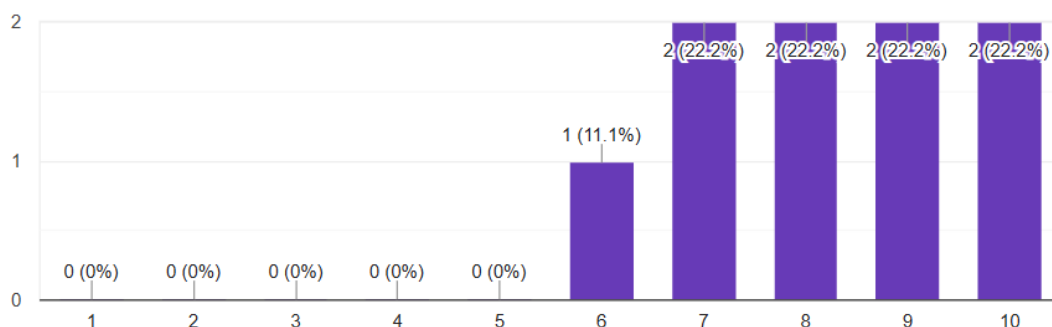


Whilst the Greek mythology setting was liked, the result of this poll shows it doesn't have to be completely accurate, and I can take liberties for the sake of game design.

How much do you like the sound of this game?

[Copy chart](#)

9 responses



The final question was to see how much they enjoyed the sound of the game, with the average rating being 8.2. This shows that I have done a good job with the concept, and aiming it towards a 12-18 audience I'm designing it for.

User Requirements and Success Criteria:

Importance key:

Essential = These features are critical for the success of the project

Important = Very high priority

Desired = Improves the game massively, but doesn't ruin the game if it's excluded

Nice to Have = Benefits the game, but is at a low priority and will only be added if I have time

Feature	Description	Justification	Importance
Start Menu	When opening the game, the start menu is what greets you. It will contain the title of the game, Start Button, and High Score (if these are implemented).	Every game I researched has a start menu for when you open the game. It makes sure you aren't immediately thrown into a game when you aren't ready. This is important, as it would frustrate the player if they left their computer for a minute to get water, for example, to find the game has been running, and they've been getting attacked for the past minute. It also gives a home to the player's High Score, if that gets implemented.	Desired
High Score System	When the player defeats enemies, they collect exp. This is used both for getting power-ups, but could also be used as a score to measure how well they're doing. The score would be displayed on the HUD whilst playing, and their highest score would be recorded by writing it to a file, to be displayed on the Start Menu.	A high-score system would increase replayability, as players try to get as high of a score as they can. It may also encourage them to take risks, and defeat harder enemies to improve their score further, creating more intense moments, which the stakeholders seem to want, with 33% voting for an intense and stressful atmosphere. However, when asked about this system specifically, over half of the stakeholders said it would only bring a small improvement, or no improvement at all, and it doesn't affect the main gameplay loop much, so it will only be added if I have time.	Nice to Have
Start Button	If the Start Menu is implemented, it will feature a Start Button which puts you into the game.	If there is a start menu, there will have to be a way to transition between menu and gameplay, and I think a clickable button is the way to do it, as it will be obvious enough to only be clicked on intentionally, making sure all games start only when the player wants it to. I also considered having the game automatically start after a certain amount of time, however that takes away some player freedom, as they can't start early, if they want to jump into a game immediately, or they can't wait a long time, possibly leaving the computer, as a	Essential

		game will start automatically, and they may not be there when it does.	
Movement System	In the game, your character needs to move. Moving will use the WASD keys to move the character in each cardinal direction.	A movement system is essential for gameplay, or else the player will be a sitting duck for enemies, which may be frustrating. Teenagers appreciate freedom, and no movement would remove some of that. I decided to use WASD to move, as it is quite common in the industry and the games I researched, and would be most comfortable to play when combined with a mouse.	Essential
Shooting System	When you click the left mouse button in a game, a Hammer projectile will be fired from your character in the direction of your mouse position. This will be the attack used to defeat enemies. Hammers will travel in a straight line for a small distance and then despawn.	The player will need a way to shoot enemies, or else the game isn't in the player's control and they will feel weak, causing frustrations. I decided to use the mouse to aim, as that will allow the player to be more accurate with their shots, avoiding annoying misses that aren't because of skill. I considered using the arrow keys to aim, like in the Binding of Isaac, however, during my research, I found it to be quite difficult to aim, and became quite tedious.	Essential
Dash	When you click the right mouse button, your character will be launched in the direction that they're moving. This will also give them a small amount of invincibility from enemy attacks. There will be a period after you dash where you cannot dash again.	This mechanic was found in 2 of the games I researched, and made them immensely more enjoyable. It gives a sense of control to the player, and allows them to avoid enemies' attacks with precise timing, making the game require some more skill, which would help increase the difficulty to a level around what my stakeholders wanted. I deemed it to be important and not essential as the game will still work without it, however I think it is a mechanic that should be high priority for the sake of making the game enjoyable.	Important
Enemies Spawn	Various enemies will appear at random intervals in random places on the map, however never on the player's screen.	This is mandatory as without there being enemies, there is no game. I think making it random will help increase replayability, and make each run feel different, as you never know where the enemies will come from, and it also increases difficulty, which the stakeholders want, as you can't predict where they'll come from. The enemies shouldn't spawn on the player's screen both to increase immersion into the game, as it may be quite jarring for an	Essential

		enemy to spawn randomly, but also to decrease frustrations with a powerful enemy spawning right on top of you.	
Enemies Attack	Enemies will move towards a position where they can do damage to you. This may be running at you and dealing damage when they touch you, or keeping their distance and firing a projectile that damages you if it touches you. To implement this, the player will need a 'Health' variable which will decrease if hit by a successful attack.	To make the enemies threatening, they need to do damage. This will be the main source of difficulty that the players must overcome. The melee ones increase intensity, which the stakeholders want, as they'd constantly chase you down, and the projectile ones increase diversity in enemy types, making the game more interesting, but also make it more difficult, as you have to focus on dodging their attacks as well as the horde of melee enemies.	Essential
Enemies can Be Defeated	If an enemy is hit by a Hammer, it will take damage. If it takes enough damage (the exact number depending on the type of enemy), it will be defeated, and disappear, and you will gain exp.	To make sure the game is possible, you must be able to defeat the enemies. Dropping exp gives the player a reason to go after enemies, creating these intense situations which the stakeholders wanted.	Essential
10 Enemy Types	There should be at least 10 types of enemies, which will all have different designs. They may also have different movement speeds, health, damage, and attack differently. If they are more powerful (having significantly more health or damage), they will spawn less often. There will be 4 ranged enemies, 5 melee enemies, and the 1 boss enemy.	To make sure the game doesn't get boring quickly, there must be a variety of types of enemies that work differently. The different visual designs will interest the teenage audience, and the gameplay differences between them will force them to change how they play, increasing the difficulty. Different enemy types were in all 3 games I researched, meaning it is clearly very important.	Essential
More Enemies	If there is time, I will add more enemy types	Whilst I feel like 10 enemies is a minimum for the game, my stakeholders clearly wanted a very large amount, with 78% voting for more enemies over more power-ups. This means that it is important for me to add as many as I can, which would increase replayability, as people want to see all the enemies that they can. All the games I researched had more than 10 enemy types, with the Binding of Isaac having over 300. Whilst I can't get to those numbers, I can try to get as many as I can, within reason.	Important
Gaining a	After defeating an enemy, you	This gives you an incentive to defeat	Important

Power-Up After Enough EXP is Collected	gain exp. If the amount of exp you have exceeds a certain number, you get a character-boosting powerup. The amount of exp you need to get another power-up increases each time you collect the power-up.	enemies and collect exp. The player will want to get more powerful, so they will have to fight enemies, and get into these dangerous situations, increasing intensity, which the stakeholders seemed to want.	
Power-Up Gems	At random points throughout the game, a Power-Up Gem will spawn. If you break it open, by throwing hammers at it, you gain a power-up that applies effects to your hammer. Power-up Gems cannot spawn on the player's screen. The player will be notified when and where the Power-Up Gem is.	This gives you an incentive to explore, rather than staying in one safe location. The player will want to get more powerful, and see the hammer power ups which are more interesting, forcing them to leave their comfort zone.	Essential
10 Power-Ups	There must be 10 types of power-ups you can get. They come in 2 types: hammer-changing and character-boosting, and there will be 5 of each. Hammer-Changing power-ups are probability based, and will do things when you throw a hammer. Character-boosting power-ups will strengthen your character. The power-ups will be: Damage Up Speed Up Health Up Dash deals Damage Passive Healing Flaming Hammer Electric Hammer Explosive Hammer Hypnotising Hammer T-Rex Spawning Hammer	The power-ups are what make you stronger. They incentivise exploration for power-up gems, and defeating enemies for exp. Without them, it would also be too difficult to defeat the boss, who should be especially powerful, and probably push the game to be harder than what the stakeholders want. The power-ups I chose are supposed to provide a variety of unique effects, with some being more interesting and rarer, giving the player incentive to replay until they come across the rarer ones.	Essential
More Power-Ups	If there is time, I will add more power-ups	Whilst 10 power-ups provides a good range of options, I do think there should be more, to increase run diversity, and give the player more reason to replay. However, it isn't a priority, as when I surveyed the stakeholders on whether they preferred more enemies or more power-ups, far more chose more enemies, with power-ups only having 22% of the vote.	Desired

Screen for Selecting Power-Ups	When you get a power-up, it should pause the game, and bring you to a different screen, where there is a choice of 3 power-ups. You click on the one you want, and it gets applied to your character.	I think it would be good to have a separate screen for the player to decide what power-up they want. It gives them a way to choose what they want, and create their own power-up compositions, giving them freedom which is something a teen will appreciate. I also considered just randomly giving them a power-up, which would keep them in the battle as opposed to taking a break on a different screen, however that takes away some player control, which I don't think is a good idea. This power-up screen will be heavily influenced from Hades, as that perfectly displays all important information.	Desired
HUD	Whilst playing the game, at the top of your screen, it shows your health, score, and how close you are to your next power-up. After you dash, it will also show a bar which decreases until you can dash again. There will also be the timer until the boss spawns.	A good HUD should give the player all information they should know to allow them to understand what's happening. For example, if they have low health, they'll see that and understand that, to win, they must be careful. I thought other information that would be useful would include how close they are to the next power-up, which may encourage them to take risks to get enough exp, a bar to show when they can dash, allowing them to more precisely dodge attacks, and the timer, so they know how long they have to get more powerful until the boss spawns. I also considered putting this information in the pause menu, which would give them more time to think about how they should play, however I think it could become tedious to switch between menus, and all the games I researched have this info on their HUD.	Desired
Boss Spawns after Timer	After around 6 minutes of gameplay, Zeus spawns. He is an especially powerful enemy who can throw lightning bolts at you, spawn enemies, and use a sword. He has a lot more health than most other enemies, and deals a lot of damage. However, if you defeat him, you win the game.	There needs to be a boss, both as a win condition, and to give the player a reason to get more powerful, as they need to beat this impossibly strong enemy. I thought it would be best to spawn after a timer, as it gives the player a time limit to rank up, otherwise they would be able to farm exp, and get too powerful with no time pressure. I am currently thinking 6 minutes is a good time for the timer, as my stakeholders wanted the game to last, on average, 8 minutes, so if there is a 6 minute timer, and it takes 2 minutes to beat the boss, it should reach that number in most	Essential

		games, however this is subject to change depending on how hard the boss is.	
Victory Screen	After you defeat Zeus, this is the screen it brings you to. It has a congratulatory message, and the option to restart.	I think a screen after you win would be a more satisfying reward to the player than just going back to the main menu, or immediately restarting the game, as it gives them time to look back at their victory. It also gives them a place to restart, rather than having to get back to the main menu. However, the game will function fine without the victory screen, so it is only important.	Important
Loss Screen	If at any point you run out of health, this is the screen it brings you to. It has a message that taunts the player, and the option to restart.	I think a screen after you lose would be good for replayability, as they can immediately jump into another run. If I add a light-hearted, taunting message, it may also make the player feel challenged to restart and try to win this time, especially for the competitive audience of a teenager. It also gives them a place to restart, rather than having to get back to the main menu. However, the game will function fine without the victory screen, so it is only important.	Important
Pause Menu	If you click the Tab button at any point, the game temporarily freezes, and it brings you to a different screen. On this screen, there's the option to unpause, and see the probability pie-chart if that's implemented.	Useful for if the player needs to take a break whilst playing. Games should last around 8 minutes, which is enough time for something to come up that you have to do which is more important than gaming. If that happens, you don't want to just lose your run, so it would be better to implement a menu that stops the game where you are, and you can carry on later.	Nice-to-Have
Pie-Chart Displaying Probabilities for Power-Up Activation	Either in the pause menu (if I am able to make it), or on the HUD, I may have a pie chart to show the probabilities that each hammer power up triggers.	With a multitude of probability based abilities all affecting the same attack, it could get quite confusing for the player. This would help the player understand the chances that their attacks trigger an ability without having to do complex maths. I'd prefer to have it located in a pause menu, as that would allow the player to read over the information and strategies without having to multitask. However, if it is well designed enough, it may be able to be placed on the HUD and still get all its information across. However, this information is neither	Nice-to-Have

		mandatory for the game to work, or important enough to be of a high priority, so I've only deemed it 'nice-to-have'.	
Pixel-Art Art Style	The game will be designed with a pixel-art art style	When given a choice between 4 different art styles, 78% of the surveyed stakeholders voted for pixel art. However, it is only visual, and would be quite time consuming to design whilst coding in the different features, so has only been deemed important.	Important
Animations	When moving, there will be simple animations for both you and the enemy, where the character models stretch and squash, to simulate movement. Hammers will spin when thrown.	Animations both help improve the look of the game, as they make it feel more alive and attention-grabbing, keeping the teenage audience engaged, but also help tell the player what's going on: If the hammer is spinning, it is flying through the air. The way I want to animate everything keeps it simple, meaning I don't have to create a new image for every frame of every animation.	Nice-to-Have
Victory Cutscene	After you win, there will be a simple animated cutscene showing you defeating Zeus, and returning to your world, victorious.	Currently, the game has no ending and no story. Adding a cutscene at the end of the game, I think, would make for a satisfying reward for beating the game, and gives a sense that there may be a deeper story, which may intrigue the player. Whilst children will be satisfied with games that don't really have a story, older audiences, like my stakeholders, will want at least some part of a story, so I should at least give them something. However, this is very unnecessary for the game to function, so it is possibly the lowest priority thing on my list.	Nice-to-Have
Dungeon-like Setting	The atmosphere will be dark, with rocks, pillars and bones everywhere. The screen will fade to black around the edges.	When surveying my stakeholders on the atmosphere of my game, the most votes were given to dark and gritty. A dungeon would fit this, and be able to fit into the Greek mythology theme which the stakeholders seemed to like the idea of. It also allows me to darken the edges of the screen, shortening the field of vision of the player, allowing enemies to sneak up on them creating the intense situation the stakeholders wanted. However, it is only visual, so isn't essential for the game to function.	Important
Sound	Sound effects will play when you hit enemies with hammers,	Having sound effects would be a great aesthetic addition that will also better	Nice-to-Have

	defeat enemies, collect a power-up, and when the boss spawns.	translate to the player what is happening, as there is audible feedback to every action. However, this will not change how the player plays the game at all, so is only really important for making the game better presented, as opposed to playing better.	
Volume Controls	In the home and pause screen, there will be a way to control all the volume if sound gets implemented. There will be a speaker symbol which has bars by the side where you increase the amount selected to change the volume.	If sound effects get implemented, different people will be used to different sounds, so giving them a way to choose their preferred volume will help them enjoy the game more.	Nice-to-Have
Colour Blind Mode	This will be an accessibility feature that increases the contrast and saturation of all colours so people with colour blindness can more easily see what's going on instead of having different objects blend into each other.	This feature will widen the range of players who can play my game, as without this colour blind people may find my game confusing or even impossible to play. However, it is only nice-to-have, as the vast majority of players won't need this.	Nice-to-Have

Limitations:

Limitation:	Justification:
Time Constraints	Due to the game benefitting from having as many enemies and power-ups as I can get, this game can scale near infinitely in size. The only limitation to how big it can get is the amount of time I have to implement all my ideas for power-ups and enemies, because I need to focus time on making the rest of the game too, with things like menu design, pixel art, and animations all also taking a lot of time. Once there is a good selection of power-ups and enemies, expanding other aspects of the game like the menu design will take priority because these aspects are important to help the game feel polished, benefitting the user experience more than simply adding more power-ups.
Lack of Experience	I have never made a game before, and have very limited experience in both C# and Unity. This means that I don't yet have the ability to code a lot of complex ideas that require vast knowledge of the language, and may result in the final code being less optimised, harming the performance of the final product. Although I can eventually learn how to do these, it is once again time constraints preventing me from getting past the limitation of my lack of experience.

Animations	If I get to implement animations, they will be quite simple, mostly being based around manipulating the sprite rather than combining multiple sprites to simulate movement, which will decrease the realism of the game and make enemies' attacks less readable, possibly catching the player off guard and causing frustration. This is because I, once again, don't have the time to create many sprites, nor the artistic ability to make it look good without having to go through many iterations.
Language	The game will only be in English, as this is the only language I know well enough to add to the game. However, this will isolate non-English speakers from being able to fully understand my game, and may cause them confusion. However, all the surveyed stakeholders spoke English fluently, so solving this isn't a high priority. The basic gameplay should also be simple enough to allow non-english speakers to still understand what to do, allowing them to still get some enjoyment out of my game.
Simplistic Mechanics	In the game, the player only really has 3 things they can do - move, dash and attack. This means the game will be quite simple, removing some complexity which I would want. From the games I researched, Hades had 3 different attacks as opposed to my 1, and Enter the Gungeon has 2 different item types you can make use of. Both of these add complexity and decision making which my game will lack. This means, to make the game as difficult as the stakeholders want it to be, I will instead need to make the enemies harder to defeat, as opposed to the mechanics being harder to learn, which may cause frustration.

Hardware and Software Requirements:

The player will need a mid-range computer with a solid CPU and any graphics card, as the game shouldn't be so power-intensive that it needs a very good CPU, but there will be too many enemies and moving parts happening at once to run on a weak CPU, and graphically it shouldn't be intensive at all due to it being rendered in 2D with only simple graphics. They will need a mouse and a keyboard, as those are the controls I have chosen for the game, and a monitor to see the game. The user will need Windows 10 or later, as that matches the operating system version I am testing the game with, however the game may be able to be played on older versions, depending on how resource intensive it is.

For development, I will be using Unity (the game development program I will be using to make my game), therefore I need a computer which can run Unity. From the Unity website, I can see this means I will need to have at least 8GB of RAM in my computer, and for it to run on either Windows 10 or 11, if running Unity 6.1. ("Manual: System requirements for Unity 6.1"). Of course, software-wise, I will also need to download Unity, which I am using as it is good for beginner game makers due to its intuitive AI and premade attributes. I will also need Visual Studio, an IDE used for coding in C#, the language Unity uses.

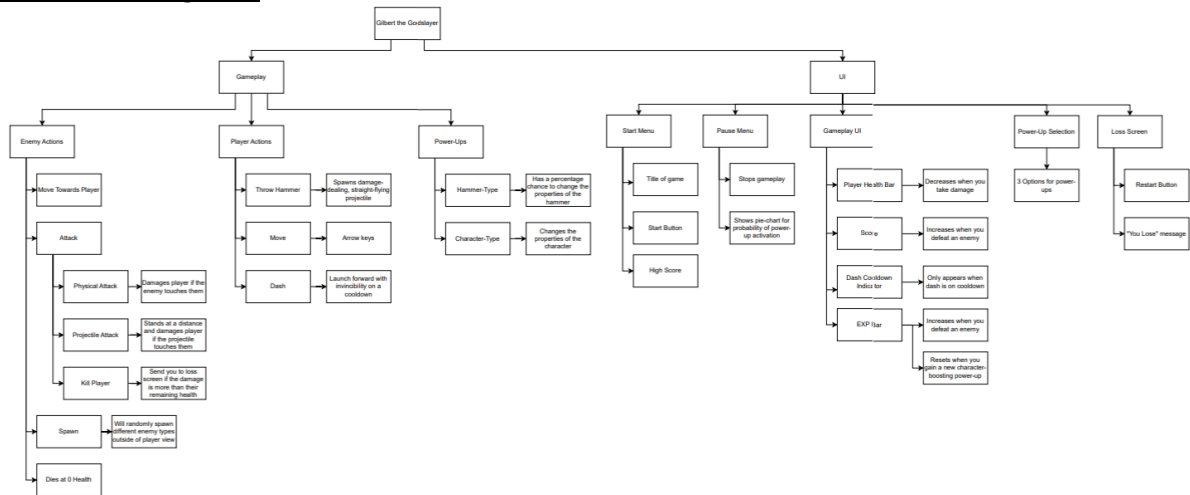
What Makes This a Computable Task:

Whilst making this project, I will be using many computational methods:

- Decomposition will be used to break up all the different features in the game into simpler, more manageable parts. For example, when making the enemies, I'd break the problems down into designing their movement, attack pattern, stats, taking damage, spawning criteria etc. This turns a large, overwhelming task into a bunch of smaller tasks which are easy to think about and create the solution for.
- Solving the problem computationally also makes it easier to make changes after testing. If I find a balance issue, I just have to alter a variable, and retest until it feels fair, however if this was a physical board game, for example, it would be incredibly time-consuming and expensive to make iterative changes like this.
- Once I've used decomposition, I can use problem recognition to determine what type of problem it is, and if I notice two problems have a similar solution, I can reuse code in multiple places throughout the program if I solve it computationally. For example, the code to decide what way the enemy moves will be similar to the code that decides what way the hammer flies. If I solve one of these issues, I can reuse the majority of the code, making a few minor alterations, to solve the other, saving a lot of time and effort which can be used to develop more features.
- Concurrent processing will be needed, as many things will be happening at once. For example, in just one frame, you might move, enemies will chase you, you might spawn a projectile, and a damage calculation will take place. All these will happen at once, and only a computer can deal with them all simultaneously.
- Due to all the power-up triggers being based on probability, the program requires a way to generate random numbers. Computers can simulate pseudo-random numbers, which will work perfectly, and be far more efficient and feel more random than anything a human can do.
- There will have to be a lot of precise calculations done in fast succession, as the direction enemies move and hammers are thrown will be calculated using trigonometry. A computer will do this incredibly quickly, and be correct, so nothing unexpected will ever happen.
- Abstraction will also be used to simplify the game and make it fun. In real life, throwing a hammer would not make it travel in a straight line, as it would be affected by gravity. However, for the sake of letting your throws be constantly accurate, I'm removing the unnecessary detail of gravity, and allowing the hammers to fly straight.

Design

Structure Diagram:



The first thing I designed was the structure diagram. It consisted of taking all the different features I planned for the game and organising them into a far more readable and well structured format. I did this to speed up future development as it will be far easier to see what I need to do in this format than in a table. I first split the features up into gameplay and UI features. This is so I could more easily see how much I will have to split my time between features that need to be coded (the gameplay ones) and the ones that need to be designed (the UI ones). I then broke it down further into the individual features, which I broke up even more into all the aspects of each feature. This meant I ended up with lots of individual tasks which are very easy to tackle. It also gave me clear stages to divide my work into, e.g. a stage where I tackle all the tasks relating to enemies, or one for power-ups. I could also lump all the UI elements into one stage, due to them being very design focused, and not requiring much code, making it a reasonably easy stage to complete. Overall, this diagram gives me a clearer direction on where to take my game and what features should be tackled at once, speeding up.

GUI Designs:

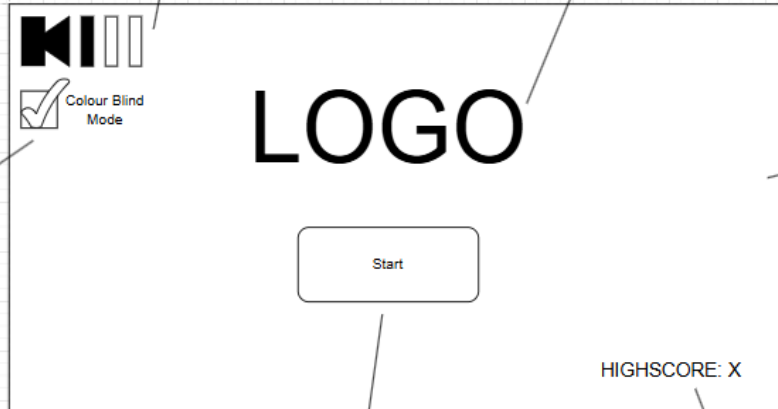
I mocked up some GUI designs for all the different screens that will be in my game. This is so that, when I eventually develop these screens, it will be useful for me to have some pre-made designs that I can then use for the final screens so that I don't have to design it whilst coding at the same time, possibly changing things and backtracking and wasting a lot of time. I have also explained all the features of these screens in annotations on the design.

The font for all text will be Silkscreen as to emulate a retro video game vibe that goes with the pixel art style I'd like my game to have, otherwise there will be a disparity between the GUI elements and gameplay that may break the players emersion.

MAIN SCREEN

I'll have volume controls in the top right of the screen as to not distract from the main things the player should be looking at - the logo and start button. The player will control the volume by filling in the bars, with all of them being filled in representing full volume and none of them representing no sound. I chose this as its a recognizable, simple way of going about volume control that doesn't require any text

The logo is very large and probably the first thing the player is going to see, as to tell the player that this is the game they are playing, and to help create recognition for the brand for any future projects that may relate to 'Gilbert the Godslayer'.



The background to this screen will be part of the dungeon that the main gameplay takes place in, so that the first screen the player will ever see accurately reflects the vibe of the rest of the game

The Start button is large and in the middle of the screen to immediately grab the players attention so they can't miss it and not know how to play the game

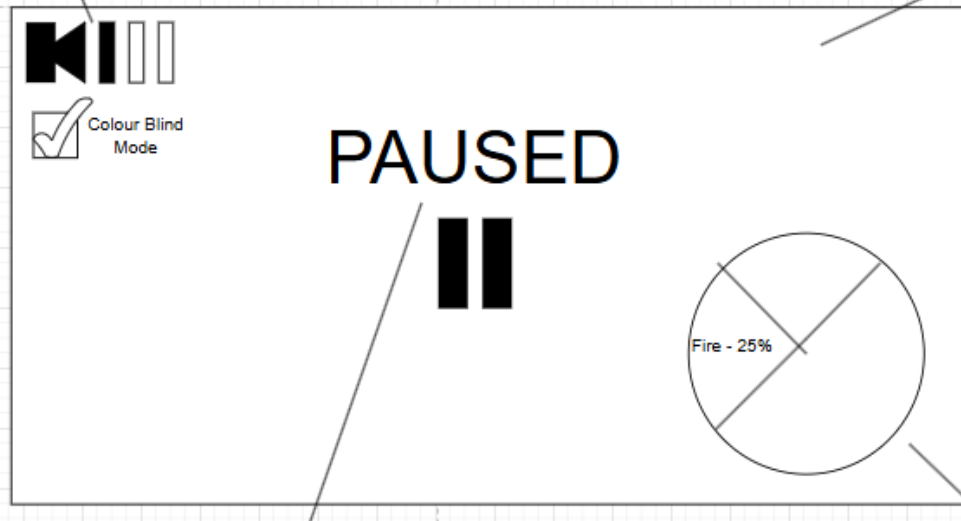
The high score counter is small and in the corner as to not be immediately attention grabbing, due to it not being a fundamental part of the gameplay, but still visible so the player won't miss it entirely.

I first designed the main menu. This will be the menu that the player will be greeted by when they open the game, and contains the 'Start' button. I wanted this screen to focus on making it very clear to the player how they start playing so they can get into a game without any confusion.

PAUSE SCREEN

The volume and colour blind mode controls return from the main menu in the exact same place in case the player wants to alter these options after they've started playing. They didn't move so that it keeps them extremely easy to find.

The background will be a simple dark grey to emulate a feeling of plainness and take all things exciting out of the pause menu, emphasizing this screen is a break from the game where nothing intense happens. Clicking on the background will unpause the game, to make it easy to do without requiring an extra button that may overcomplicate the screen.



In the centre of the pause screen, there will be text telling you the game is paused, and the internationally recognised pause symbol. This is to remove all confusion from the player if they are wondering why the game has stopped.

The pie-chart which display the probabilities that a power-up activates will be very large and in the corner. The large size will hopefully make it more readable as there can be a lot of information. Each segment will be a different, bright colour as to once again help readability, due to there being a high contrast between each segment. The text will only appear when hovering over a specific segment, so you're only getting the information you want. It will tell you the name of the power-up and percentage activates - the only important info you'll need.

Next I designed the pause screen, a screen that is accessed by pressing escape, and is supposed to give the user a break from the gameplay if they ever need to stop for a few moments. I wanted to give this screen a purpose beyond simply stopping the game, so this is where I put some useful accessibility options, as well as the pie-chart which provides useful information on power-ups, as these are features that may have to be changed/checked during gameplay.

GAMEPLAY SCREEN

The health appears in the top corner, making it easy to find. It simply displays your health as a number, removing any confusion to the player of how much they have left. It also shows their health as a fraction so they know their max health at all times, giving them an idea of how well they've been playing, and letting them better figure out if they should take risks or not.

The score appears in the top centre of the screen, acting as a constant reminder to the player of how well they're doing. Putting an emphasis on the score will hopefully push the player to try and better it, creating replayability.

The timer for the boss appearing appears in the top corner, acting like a clock which spawns the boss when the hand hits the top. A clock is a very recognisable way to represent a timer, and having the boss's name in the centre to give the player an idea of who the boss is (which may be helpful if more bosses are ever added). It also doesn't give the player too much information, hopefully surprising the player when it spawns for the first time, creating a memorable experience.

Health - X/X

SCORE - X



The exp bar resides in the top left of the

After that I designed the gameplay screen, which is the screen where the actual game is played. I wanted this screen to have as much empty space as possible, because it is important that the player can see what is going on without being distracted by non-gameplay elements, so that they can fully understand what is going on in the game.

DEATH SCREEN

YOU LOSE

Score: X
Highscore: X

Restart

Main Menu

In the centre, there is text telling the player they've lost. It is very large and should make it very clear the player has lost. It being large may also taunt the player, making them play again so next time they'll win.

The player's high score and score are in the direct centre of the screen. This is because they are the most useful information that the player can get from this screen, so should be easily found. They are put together because both pieces of information are intrinsically linked, with the score determining the high score.

There will be a button to restart the game so that the player can immediately jump into a new game. It will be decently large so that the player doesn't miss it.

If the player wants/needs to go back to the main menu, they can. This is in case they need to change volume/accessibility features or simply

VICTORY SCREEN

YOU WIN

Score: X
Highscore: X

Restart

Main Menu

In the centre, there is text telling the player they've won. It is very large and should make it very clear the player has won. It being large may also enforce feelings of pride and victory making them feel better about their win.

The player's high score and score are in the direct centre of the screen. This is because they are the most useful information that the player can get from this screen, so should be easily found. They are put together because both pieces of information are intrinsically linked, with the score determining the high score.

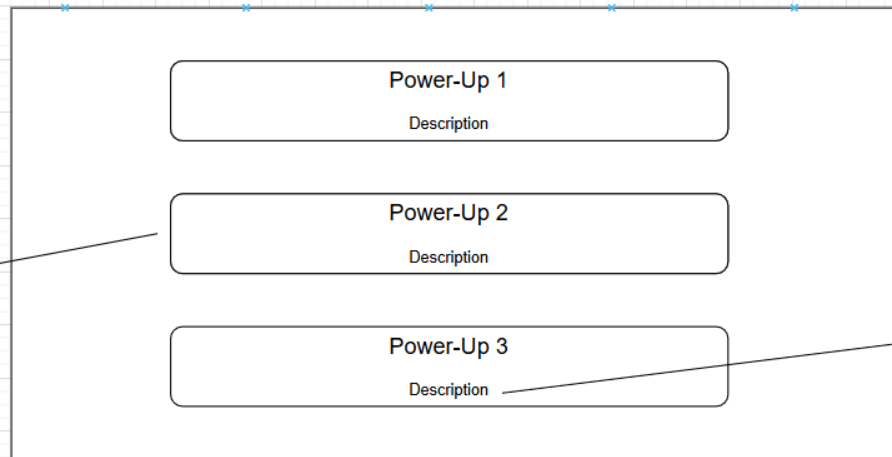
There will be a button to restart the game so that the player can immediately jump into a new game. It will be decently large so that the player doesn't miss it.

If the player wants/needs to go back to the main menu, they can. This is in case they need to change volume/accessibility features or simply want a break from the game.

I designed the Win and Loss screens, which would pop up after you defeat the final boss, or die, respectively. I want these screens to be simple, so that the player can easily read the relevant information and then immediately jump into another game.

Power-Up Selection Screen

There will be 3 buttons associated with the 3 power-up choices. This is the easiest, most simple way to demonstrate to the player that they need to make a choice between 3 options.



Each button will have a short description of the power-up. This is to allow the player to make a more informed decision on what might be best, giving the player more freedom to create a build that best suits them, hopefully making them enjoy the game more.

Finally, I designed the power-up selection screen. The aim for this one was to make it as simple as possible, allowing the player to immediately know what to do and get off this screen as quickly as possible so that they can get back into the gameplay fast.

Development Plan:

Next, I created an approximate development plan that broke my project down into stages, as well as giving a rough time-frame. This is to give me guidance for what I should be coding when, and it should make sure I don't forget to add anything in.

Stage 1 - The Basics

This stage contains the most fundamental features that will be necessary for having a functional game and needed for testing all other features.

Contains:
Movement System
Shooting System

Time-Frame - Approximately 4 Lessons

Stage 2 - Enemies

This stage is based around having functional enemies that can move and attack.

Contains:
Basic Enemy parent class

Making enemies move
Making enemies attack
Making enemies spawn
Making enemies defeatable
Gaining exp from enemies
Dash

Time-Frame - Approximately 6 lessons

Stage 3 - Power-Ups

This stage focuses on creating the power-up system

Contains:

Creating Power-Up Selection Screen
Creating Power-Up Gems
Creating 5 character power-ups
Creating 2 hammer power-ups

Time-Frame - Approximately 4 lessons

Stage 4 - Aesthetics 1

This stage focuses on some of the more important non-gameplay features

Contains:

Start Screen
Death Screen
Creating some important sprites
Health Bar
Exp bar

Time Frame - 3 Lesson

Stage 5 - More Enemies and Power-Ups 1

This stage focuses on adding in more enemies and power-ups, hopefully enough to fill out the amount stated in the user requirements

Contains:

The Boss
4 melee enemies
3 ranged enemies
3 hammer power-ups

Time frame - 6 Lessons

Stage 6 - Aesthetics 2

More, less useful things to make the game look better

Contains:

Animations

Less important sprites

Time-Frame - 2 Lesson

Stage 7 - Extras

This stage focuses on adding features deemed Nice-to-Have

Contains:

High Score System

Pause Menu

Pie-Chart

Victory Cutscene

Sound

Time-Frame - 5 Lessons

Stage 8 - More Enemies and Power-Ups 2

The final 'stage' will be me using any extra time to add in as many enemies and power-ups as I can come up with. This stage is not mandatory.

Explanation:

I want the first stage to be the basics as that is the groundwork that all further features will be based off of. It will be impossible to test the enemies or power-ups without being able to attack, for example. Being able to move and shoot are the two most fundamental features that need to be implemented immediately for me to make the rest of the game.

I think it's then going to be most important to develop the features based around the enemies because having enemies is the next most important feature that allows me to test the later features, such as power-ups. It also means I have an actual game, with the gameplay loop consisting of running around defeating enemies.

The next stage will be based around power-ups. This third stage completes the core gameplay loop, and all later stages will be based more around polishing the game.

After that, I want to focus on adding more aesthetic features which are useful for conveying information to the player, and allowing me to see useful information during testing like how much health the player character has. I also expect the features in this stage to be pretty simple to develop meaning I'm not spending too much time early on on aesthetic features.

Next, I want to add more enemies and power-ups. In the user requirements, I said I wanted 10 of each, so this stage would be focused on getting to this number. This stage is important for making the game more fun and less repetitive, and is the last stage with gameplay features I deemed important. This is the final stage that it is crucial I get to, as everything after this stage makes for nice additions, but isn't fundamentally imperative that I add.

Following that, I'd like to add more visual changes to polish the game more. This is one of the last stages because it really isn't affecting gameplay at all, but it's still nice to have to make the game look better.

The last real stage is adding all the gameplay features deemed Nice-to-Have. These are features that, in my stakeholder survey, people didn't think were super important to have, but still would improve my game, so if I have any time left over, I should add in.

Lastly, with any left over time, I'll create more enemies and power-ups. I think this is the best way to add more content to the game, as it increases run diversity, making the game more replayable.

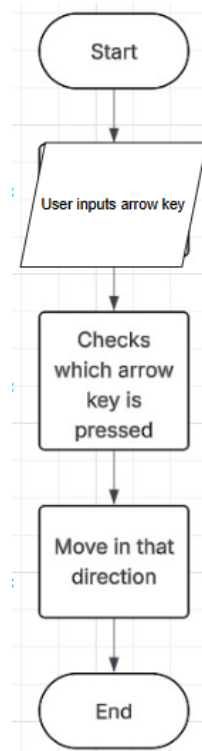
Every stage will be tested after completing it.

Stage 1 – Movement and Shooting

Data Dictionary:

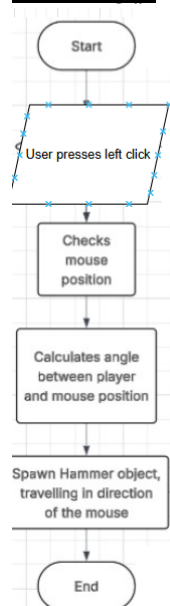
Class	Attribute	Data Type	Method	Description
Player	Health	Integer		Decreases when damaged, ends game when =0
	Speed	Float		The speed the player moves at
			Movement()	Method for moving
			Shooting()	Calculate angle between mouse and character and spawns hammer
Hammer	Direction	Float		The angle the hammer moves at
	Speed	Float		The speed the hammer moves at
	DespawnTimer	Integer		Decreases every second, destroys hammer when =0
	Damage	Integer		How much damage the hammer does
			Flying(float direction)	Method for hammer travelling

Movement():



Movement will be simple. The program will check if a key has been pressed, and if it has, it will change the objects x and y co-ordinate attributes, making it look like the player moved. For something as basic as movement, because it is currently a basic feature that will be built upon further later, I shouldn't overcomplicate it and make sure it feels as fluid as possible so that it is easy to add to without having to completely rework it.

Shooting():



This method will work by checking if the user has inputted a left click. It will then check the mouse position, and use that to calculate the angle between the player and mouse, then instantiate the Hammer, travelling in the direction of the angle that was clearly calculated. This is a very clear sequence of actions, which is why I used a flowchart.

Flying() - Pseudocode:

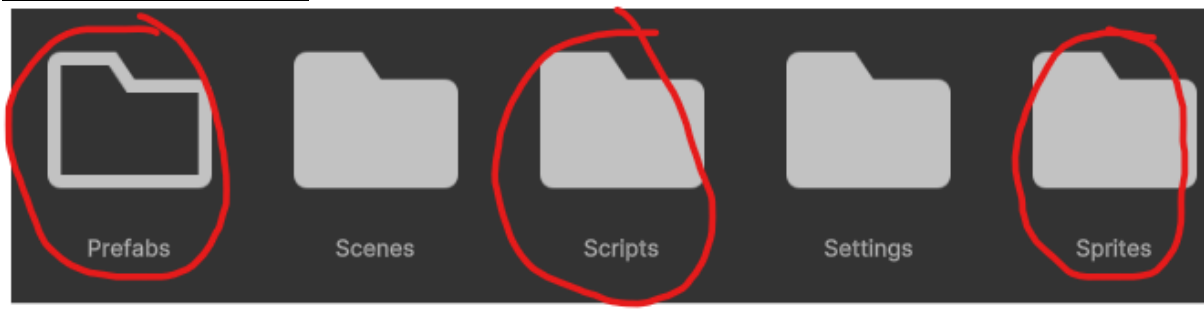
```
WHILE DespawnTimer!=0
    PositionX = PositionX + speed*cos(direction)
    PositionY = PositionY + speed*cos(direction)
    DespawnTimer = DespawnTimer - 1
    IF DespawnTimer = 0
        destroy().this
    ENDIF
ENDWHILE
```

Every hammer spawned will immediately travel in the direction passed into the flying() method non-stop. I decided to add a despawn timer so that, if the player never hits anything with the hammer, the hammer will eventually still be removed from the world, saving resources and helping the game run better. I used pseudocode here because this method will very easily translate from pseudocode into actual code, meaning I can just copy this exact code over, just substituting certain words to make it work, saving me a lot of time, and allowing me to add more features.

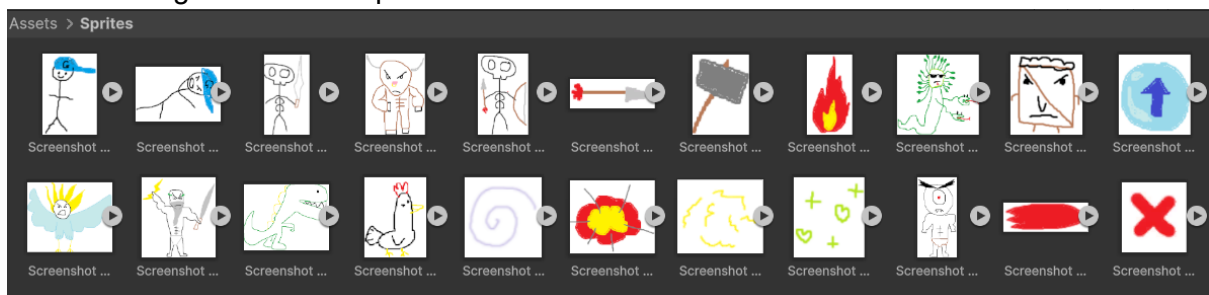
Test Table:

Test	Expected Result	Reason
Press the 'D' key	Player's X coordinate increases	This test is needed to make sure the player can move right, so that they can access the right side of the map
Presses 'A' key	Player's X coordinate decreases	This test is needed to make sure the player can move left, so that they can access the left side of the map
Press the 'W' key	Player's Y coordinate increases	This test is needed to make sure the player can move up, so that they can escape enemies if they hoard below them
Press the 'S' key	Player's Y coordinate decreases	This test is needed to make sure the player can move down, which is necessary to make sure the movement feels fully formed, and not buggy or limiting
Move the mouse	Crosshair sprite follows the cursor	This makes sure that the crosshair accurately represents the point which the hammer will fly to without misleading the player
Left click the mouse	Spawns a hammer from the player that travels in the direction of the mouse	This ensures the hammers shoot correctly, because if there is an issue with this system, it may become very unfun to fight enemies

Stage 1 Development:



Before I did any coding, I wanted to add some organisational folders to my project, which I did by adding a folder for all Scripts, Sprites, and Prefabs. This should make navigating my project far easier, and will reduce the amount of time I spend looking for specific items due to them all being stored in one place.



I then created a few beta sprites in MS Paint and imported them into my project. I want to have a pixel-art art style for my game, however creating a good-looking sprite for every enemy and effect I need will take a lot of time. In my development plan, I specifically allocated the sprite creation into stages 4 and 6, however even before I get to those phases, it will be very useful to have some temporary sprites so that I can still visualise what is what, decide on good hitboxes, and it will hopefully reduce the time I spend creating pixel-art, because I have a foundation I can improve into something that looks good.

After that, I started coding. The first thing I wanted to design was making the player move. I created an empty object, assigned it the player sprite so that I know that this object is for the player, turned it into a prefab so that I can easily get it back in case I accidentally delete the object, and finally created the PlayerMovement script. I first created a float called 'speed'. This will correlate to how fast the player can move. I used a float because it is very likely that speed altering power-ups aren't going to change the speed variable by integer values, which may break the program if I have speed set as an integer. I also made sure it was changeable in the Unity editor by adding [SerializeField] above it, making it far easier to test and find the perfect speed because I don't need to go into the code for the script to change the value of speed.

I then moved into the Update() method for the rest of the code, as that is the method that is called every frame, meaning it makes the most sense to use it for things that need to be responsive, like the movement. I knew that you used Input.GetAxis("Horizontal"), and Input.GetAxis("Vertical") to be able to tell what the player inputted, however I couldn't remember what you had to do beyond that, so I checked back on the notes I made on the Unity tutorial Youtube video I used to learn how to use Unity. It reminded me that I needed to make floats equal to the inputs, because they will equal 1 when the keys are pressed, which

will be useful for the calculations for movement. I could have alternatively just used IF statements to check if the button is pressed, but that would take up more unnecessary space and make the project run slower, as it will be constantly checking every IF statement every frame.

(freeCodeCamp)

I then got a vector called 'position' which is equal to the position vector of the player. This vector would then be altered using maths. If a directional button is pressed, h / v will stop equalling 0, and the position will change by the speed in the correct direction. I also had to multiply this by Time.deltaTime. This is because the Update() method is called every frame, meaning computers that run faster and have a higher FPS will run the Update() method more, and move faster. This would make the game feel terrible for anyone who has a very high FPS, because they'd move uncontrollably fast, or a really low FPS, because they'd move painfully slow, and it would just be unfair for everyone to have differences in the game based on how good of a computer you have. However, Unity has the solution for this, which is simply multiplying by Time.deltaTime. Finally, I set the position vector of the player to this new changed vector, making it look like the player has moved.

```
public class PlayerMovement : MonoBehaviour
{
    [SerializeField]
    float speed = 20f;

    void Update()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");

        Vector2 position = transform.position;

        position.x += h * speed * Time.deltaTime;
        position.y += v * speed * Time.deltaTime;

        transform.position = position;
    }
}
```

There was, however, an issue with the movement, as the player would constantly move in a north-east direction. On inspection of my code, however, I quickly found that this was just because I had added Time.deltaTime instead of multiplying by it.

Stage 1 Vid 1

```
pos.x += h * speed * Time.deltaTime;
pos.y += v * speed * Time.deltaTime;
```

I changed this, and it worked pretty much as intended.

Stage1Vid2

There was one issue with the movement, however. Due to the calculations for moving horizontally and vertically being completely separate, if you move in both directions at the

same time, due to the laws of Pythagoras, you are moving faster than usual by a factor of $\sqrt{2}$, which is an issue because it makes movement feel inconsistent. To solve this, I changed it so that instead of the calculations using the base 'speed' value, they would use individual 'speedV' and 'speedH' values. These will usually just equal the base speed, but if both a vertical and horizontal key is pressed, they will equal speed divided by $\sqrt{2}$. I could've removed the base speed entirely and just had everything calculated with these two variables, but if I want to change how fast the player moves, I think it will be easier to just change the base speed instead of both speed variables, especially because issues could be caused if I change one and forget to change the other. However, to perform this calculation in the code, I needed to have a way to take the square root of a number, which was something I didn't know how to do. I was able to find out how after some short research of the Unity documentation though, and it worked perfectly.

("Scripting API: Mathf.Sqrt")

```
private float speedV;
private float speedH;
@ Unity Message | 0 references
private void Update()
{
    speedV = speed;
    speedH = speed;
    float h = Input.GetAxis("Horizontal");
    float v = Input.GetAxis("Vertical");
    Vector2 pos = transform.position;

    if (h == 1 && v == 1)
    {
        speedV = Mathf.Sqrt(speed*speed*0.5f);
        speedH = Mathf.Sqrt(speed * speed * 0.5f);
    }

    pos.x += h * speedH * Time.deltaTime;
    pos.y += v * speedV * Time.deltaTime;
```

After this, I wanted to start on the shooting mechanics. For this to work, I would need the location of the mouse so that the hammers can calculate the direction they need to travel in. I also decided the best way to translate this to the player would be the Crosshair, otherwise the player wouldn't be able to know precisely where the hammers will travel. I didn't know how to get the position of the mouse, so I looked at the Unity documentation and found that you had to get the vector position of the mouse with `Input.mousePosition`. After this, I assumed it would be as simple as setting the position to this new vector, because that's how it worked with player movement.

("Unity - Scripting API: Input.mousePosition")

```

public class Crosshair : MonoBehaviour
{
    Unity Message | 0 references
    void Update()
    {
        Vector2 mouse = Input.mousePosition;

        transform.position = mouse;
    }
}

```

However, after testing it, there seemed to be an issue where the position of the mouse didn't correlate with the game screen.

Stage1Vid3

I assumed the issue was something to do with the code using the mouse's position on the UI instead of the camera, but I had no clue how to fix it, so I found a tutorial on making the player move with the mouse on YouTube. It turned out you needed to specify that it was using the camera position, as seen below. After that, it worked perfectly.

(Modding by Kaupenjoe)

```

public class Crosshair : MonoBehaviour
{
    private Camera mainCamera;

    Unity Message | 0 references
    private void Start()
    {
        mainCamera = Camera.main;
    }

    Unity Message | 0 references
    void Update()
    {
        Vector2 mouse = mainCamera.ScreenToWorldPoint(Input.mousePosition);

        transform.position = mouse;
    }
}

```

Stage1Vid4

I then started on creating the code for the hammer projectile. It needs a damage variable, which I want in the Hammer to simplify the future code for hitting enemies, a direction variable, so it knows what angle to fly at, and a speed variable, so it knows how fast to fly. I originally was going to have these all assigned through a constructor, which I thought would be best so that when the player calls the hammer, it could be called with these stats being different depending on the power-ups the player has, making it easier for me to do everything from the player class code. I then created the move() method, which takes in the direction and speed, the two variables that will affect movement. I then took some code from my PlayerMovement script, because this code will be very similar, so reusing it will save me time. I did have to alter it to include the trigonometric calculations for the rate that the x and y

coordinates change, to make it fly towards the mouse.

```
public class Hammer : MonoBehaviour
{
    int damage;
    float speed;
    float direction;
    private Camera mainCamera;
    1 reference
    public Hammer(int damage, float direction, float speed)
    {
        this.damage = damage;
        this.direction = direction;
        this.speed = speed;
    }

    0 Unity Message | 0 references
    public void Update()
    {
        move(direction, speed);
    }

    1 reference
    public void move(float direction, float speed) {
        Vector2 pos = transform.position;

        pos.x += speed * Mathf.Cos(direction) * Time.deltaTime;
        pos.y += speed * Mathf.Sin(direction) * Time.deltaTime;

        transform.position = pos;
    }
}
```

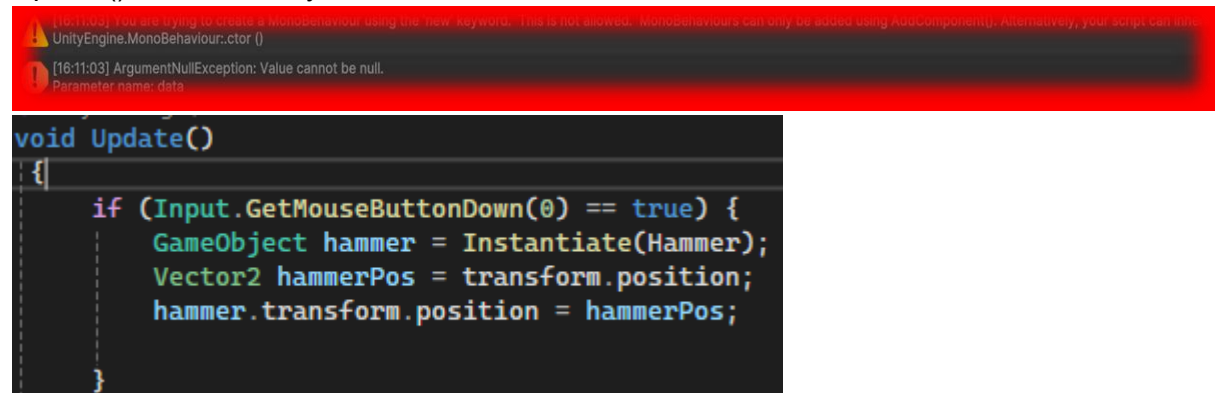
```
public class PlayerShoot : MonoBehaviour
{
    [SerializeField]
    private GameObject Hammer;
    [SerializeField]
    private int damage;
    [SerializeField]
    private float speed;
    private float direction;
    private float xDif;
    private float yDif;

    0 Unity Message | 0 references
    void Start()
    {
        Crosshair crosshair = GetComponent<Crosshair>();
        Vector2 crosshairPos = crosshair.transform.position;
        Vector2 playerPos = transform.position;
        xDif = crosshairPos.x - playerPos.x;
        yDif = crosshairPos.y - playerPos.y;
        direction = Mathf.Atan2(xDif, yDif);
    }

    0 Unity Message | 0 references
    void Update()
    {
        if (Input.GetMouseButtonDown(0) == true) {
            Hammer hammer = new Hammer(damage, direction, speed);
            Vector2 hammerPos = transform.position;
            Hammer.transform.position = hammerPos;
            Instantiate(hammer);
        }
    }
}
```

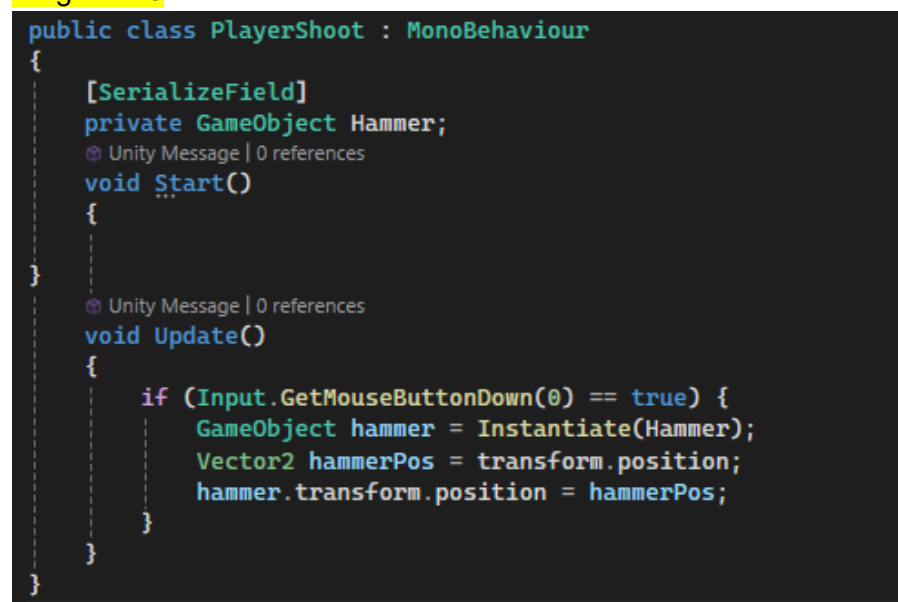
After I created the Hammer, I moved on to the PlayerShoot() method, which is important so that I can test the Hammer code. I thought it would be best to get the location of the

crosshair, find the difference between that and the player positions, and then calculate the angle between the player and the crosshair. In Update(), it would check each frame if the mouse was clicked, and if it did it should initialise a Hammer with the correct parameters. I chose to use the crosshair to calculate the angle, as that is a better and larger visual indicator for the player than my other option, the mouse. I also put the check for the mouse getting clicked in the Update() method, so that it would be as responsive as possible, as Update() is called every frame.



When I tried running it I received these errors. There seemed to be something wrong with using the 'new' key word, and also that something had the value of null. I thought the cause of the null error may have been that I instantiated the Hammer at the end, so I changed that, and to fix the error with using 'new', I just had to check back at my notes to see I was instantiating it like I would with regular C code, which doesn't work for Unity. These changes got the Hammers to spawn, however they didn't move.

Stage1Vid5



The first change I did was moving all calculations to the Hammer class. This is so I won't have to rely on passing the Direction variable in, and also makes it easier to work on the Hammer code, as all things Hammer related are in the Hammer class. I also calculated the angle with the mouse position, as opposed to the crosshair. Whilst it may cause some disparity between what the player is seeing and how the projectile fires, the difference between the crosshair location and the mouse will be a few pixels at most, and this option is simpler to code, and more reliable overall.

```

public void directionCalc() {
    Vector2 mouse = mainCamera.ScreenToWorldPoint(Input.mousePosition);
    direction = Vector2.SignedAngle(mouse, transform.position);
}

```

These changes got the Hammers to travel, but in seemingly random directions. It wasn't fully random, though, as I noticed there was a periodic pattern, so I assumed the issue was that the Atan gave the angle in degrees, whereas sin and cos were using the angles though it was in radians.

Stage1Vid6

```

direction = direction * 3.14159f / 180f;

```

I added some code to change direction from degrees to radians, which was better, but seemed to be firing the Hammer in the complete opposite X direction than what it should.

Stage1Vid7

```

-Mathf.Cos(direction)

```

I reversed the X direction by multiplying Cos(direction) by -1. This was better, but still inconsistent. However after some testing, this seemed to be because the code was using (0,0) to calculate the angle instead of the Player's position.

Stage1Vid8

```

public void directionCalc() {
    Vector3 mouse = mainCamera.ScreenToWorldPoint(Input.mousePosition);
    directionVector = (mouse - player.transform.position).normalized;
    direction=Mathf.Atan(directionVector.y/directionVector.x);
}

```

The issue was that the code was doing calculations with 2 Vectors that both stem from the origin, when I needed to do calculations with the vector connecting these 2 points. This was an easy enough fix, and after that it worked perfectly.

Stage1Vid9

```

private Vector3 directionVector;
[SerializeField]
private float timer;

timer=timer - 1*Time.deltaTime;
if (timer <= 0)
{
    Destroy(this);
}

```

In the code's current state, the Hammers would travel infinitely and never get destroyed. If I kept it like this, this may cause performance issues after a lot of Hammers have been fired, as there would be a lot of calculations performed every frame, causing lag which may annoy the player. It may also cause balance issues, as the player may be able to sit in the corner of the map and hit anything with infinite range projectiles. Because of this, I wanted to add a way for the Hammers to be destroyed after a certain amount of time. I decided to do this by having a timer in the Update() method that decreased by 1 every second, and is destroyed when the timer hits 0. Alternatively, I could have it in a separate script, but to keep the Hammer easy to work on, I decided to combine it with the regular Hammer script.

Stage1Vid10

This code didn't fully work. After the timer runs out, the Hammers freeze in place after the timer runs out. To try and diagnose the issue, I added 'print(timer);' after the timer changes value to check if it was an issue with that, but it wasn't.

Stage1Vid11

I really didn't know what the issue could be, so I tried looking at a discussion on the Unity website about someone with a similar issue. It turns out that Destroy(this) destroys the script of the gameObject, not the gameObject itself, meaning the Hammers would remain just without any code, so I changed it to Destroy(this.gameObject), which made it work fine. ("Destroy(this) not working properly - Questions & Answers")

Stage1Vid12

```
public class CameraMovement : MonoBehaviour
{
    private Transform player;
    private Vector3 pos;
    Unity Message | 0 references
    void Start()
    {
        player = GameObject.FindWithTag("Player").transform;
    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {
        pos = player.position;
        transform.position = pos;
    }
}
```

I then started working on making the camera follow the player - the final part of Stage 1 of development. I made a CameraMovement script that gets the player's location, and transforms the Camera's location to that spot. This was the most simple solution I could come up with, only using 5 lines of code. However, when I ran the program, you couldn't see the player.

Stage1Vid13

```
pos = player.position;
pos.z = -10;
transform.position = pos;
```

When looking at the Camera's default position before it changed, it was at (0,0,-10). Setting the position to be equal to the player would make it be at (0,0,0), causing a change in the Z coordinate. I simply made it so that the Z coordinate would be set back to -10 after it changes, which worked perfectly. To demonstrate this I also added the beta design for the background, which hopefully reflects the dungeon setting I decided on from the stakeholders feedback.

Stage1Vid14

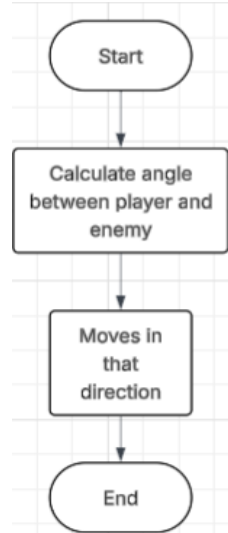
Test	Expected Result	Actual Result
Press the 'D' key	Player's X coordinate increases	Player's X coordinate increases Stage1Vid2
Presses 'A' key	Player's X coordinate decreases	Player's X coordinate decreases Stage1Vid2
Press the 'W' key	Player's Y coordinate increases	Player's Y coordinate increases Stage1Vid2
Press the 'S' key	Player's Y coordinate decreases	Player's Y coordinate decreases Stage1Vid2
Move the mouse	Crosshair sprite follows the cursor	Crosshair sprite follows the cursor Stage1Vid4
Left click the mouse	Spawns a hammer from the player that travels in the direction of the mouse	Spawns a hammer from the player that travels in the direction of the mouse Stage1Vid9

Stage 2 – Dashing, Melee and Ranged Enemies:

Data Dictionary:

Class	Attribute	Data Type	Method	Description
Enemy	Health	Integer		Decreases when damaged, destroys object when =0
	Speed	Float		The speed the enemy moves at
	Damage	Integer		Damage dealt to player if they get hit
			Movement()	Method for moving in the direction of the player
			Shooting()	(RANGED ONLY) shoots projectile at the player
			Melee()	(MELEE ONLY) damages player if it touches them
			Damaged(int damage)	If they collide with a hammer, they lose health, and get removed if health =0
EnemyProjectile	Direction	Float		The angle the projectile moves at
	Speed	Float		The speed the projectile moves at
	DespawnTimer	Integer		Decreases every second, destroys projectile when =0
	Damage	Integer		How much damage the projectile does
			Flying(float direction)	Method for projectile travelling

Movement():



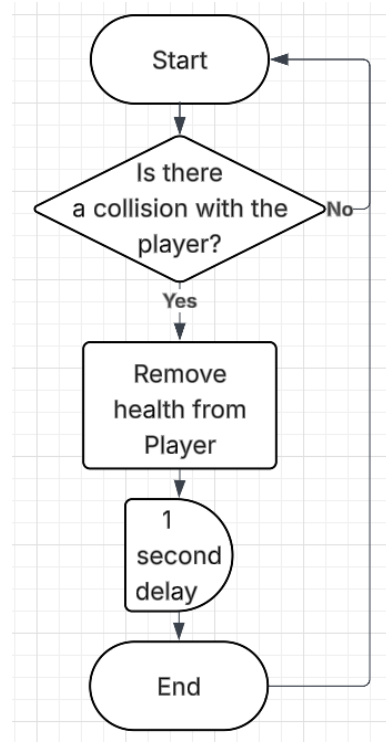
The movement will be very simple, with the enemy simply moving towards the player non-stop. To keep the enemy AI simple to understand for the player, all it does is move towards the player's direction, worked out with simple trigonometric calculations, making it quick and easy to code for me, allowing me to focus on adding more, different to increase variety in the game.

Shooting():



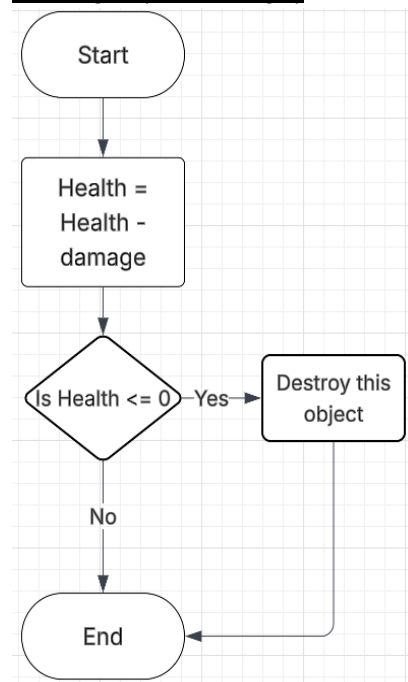
Shooting will be in the form of an IF statement in the Update() function that is always checking if the player is in range for the attack. I'm using an IF statement so that no other code has to run if the player isn't in range. If they are in range, it will spawn a projectile which will share a lot of code with the Hammer, saving me time. I used a flowchart here to demonstrate the sequencing of this method, along side the decision of if the enemy is in range, in a readable manner that will help me understand how to code it when I get onto that part.

Melee():



Melee attacks will have a repeating IF statement to constantly check if the player is touching a melee enemy. I want to do this so that the instant the player touches an enemy, they get damaged, increasing the difficulty and removing any buggy-looking interactions. I'm also adding a 1 second delay so that the player has some wiggle room and isn't damaged every single frame, as that would make the game feel impossible and kill the player almost instantly. The delay needs to be after the player takes damage, or else the game will feel unresponsive which may irritate the player.

Damaged(int damage)



Damage will be very simple, just removing the health when called, with the damage passed in as an integer variable. I'm using integers for health and damage for the sake of it being easier to understand for the player, as there will be less digits than in a float. It then checks if the enemy has 0 health or less - meaning it's dead - and removes it if it is. I put this IF statement in the damaged() method because they both are to do with health, and it saves storage space because I don't have to code an entirely different method.

Flying()

```
WHILE DespawnTimer!=0
    PositionX = PositionX + speed*cos(direction)
    PositionY = PositionY + speed*cos(direction)
    DespawnTimer = DespawnTimer - 1
    IF DespawnTimer = 0
        destroy().this
    ENDIF
ENDWHILE
```

Flying works the exact same way as it does in Hammer, meaning I can reuse the code, saving me time and testing. I designed the flying() code to be reusable for this exact reason, as the only difference between any object that uses this method is the direction value, which will be passed in.

Test Table:

Test	Expected Result	Reason
Wait	Various enemies will spawn around the player	Enemies should spawn around the player on a timer, so all you can do to test this is wait
Wait	Enemies move towards the player	Similarly to the last test, enemies will move towards the player without having to do anything, so you can just wait and see
Press the 'Space' key	Move forward without taking damage	This ensures the dashing feature is fully functional, because if it isn't and a glitch occurs whilst the player is in danger, they may lose the run on the spot due to a bug
Be within a ranged enemy's range	Enemy fires a projectile that travels in the player's direction	This makes sure ranged enemies can attack you, making them pose as an actual threat
Destroy an enemy	Increase exp attribute	This lets the player properly level up. It is important I make sure this is properly working now, because later stages will

		need this to be a functional feature for them to work.
Collide with a melee enemy	Health attribute decreases	This makes sure that melee enemies are able to do damage, because otherwise they aren't at all threatening to the player

Stage 2 Development:



I started stage 2 by preparing my pre-existing objects so that they're ready for anything I add in Stage 2. I needed a way to detect if the player or hammer is touching an enemy, so I decided to use the inbuilt colliders for Unity, as to save time developing and testing my own ways. I did consider using the coordinates to detect if they touch, but I thought using the inbuilt colliders would be far easier and more sensible.

```

public class ParentEnemy : MonoBehaviour
{
    private int health;
    private int damage;
    private float speed;
    private float attackSpeed;
    Unity Message | 0 references
    void Start()
    {
    }

    Unity Message | 0 references
    void Update()
    {
    }

    Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Hammer")) {
            GameObject hmr = GameObject.FindWithTag("Hammer");
            Hammer hammer = hmr.GetComponent<Hammer>();
            damage = hammer.getDamage();
            damaged(damage);
        }
    }

    1 reference
    public void damaged(int damage) {
        health -= damage;
        if (health <= 0) {
            Destroy(this.gameObject);
        }
    }
}

```

I then decided to start on making a general parent enemy class. This would be what all other code for the enemies gets based off, so it made sense to do it early. I added in the attributes and methods that would be shared between melee and ranged enemies, being the health, damage, speed, and attackSpeed attributes, as well as the method for taking damage. This is because all enemies need health and the damaged() method so that they can be defeated, speed to move, and damage and attack speed so they can act as threats to the player that can damage the player.

The damaged() method would get called whenever a hammer collides with the enemy, and would simply decrease the enemy's health by the damage from the Hammer, and destroys the game object if its health is less than or equal to 0. I used OnCollisionEnter2D so that the enemy would only get damaged once by the hammer, ensuring they aren't immediately killed by things they aren't supposed to, helping the game feel balanced. It also destroys the hammer after this collision for the same reason.


```

public int getDamage() {
    return damage;
}

0 references
public void setDamage(int newDamage) {
    damage = newDamage;
}

```

I added a getter for damage in the hammer's script, so that the enemy will know how much damage to take when it gets hit. I also added the setter so that, in future stages, when I'm adding power-ups, I can change the damage of the hammer based on what the player has.

I then tried testing if my code worked. The hammers passed through the enemy without damaging it.

Stage1Vid1

To try to diagnose the issue, I added print("1") and print("2") to the collision and damaged() code respectively. It printed neither, showing a collision never occurred.

Stage2Vid2

To try and solve this, I unchecked isTrigger in the Unity editor, and changed to FindWithTag to collision.gameObject, because, with FindWithTag, it could use any hammer with the 'Hammer' tag, not necessarily the one which collided with the object. This solved the issue.

```

Unity Message | 0 references
private void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Hammer")) {
        GameObject hmr = collision.gameObject;
        print("1");
        Hammer hammer = hmr.GetComponent<Hammer>();
        damage = hammer.getDamage();
        damaged(damage);
        Destroy(hmr);
    }
}

1 reference
public void damaged(int damage) {
    print("2");
    health -= damage;
    if (health <= 0) {
        Destroy(this.gameObject);
    }
}

```

Stage2Vid3

It was at this point that I decided against going forward with one parent class for both melee and ranged enemies. Both enemy types are extremely different outside of one shared method and a few attributes, and it would be far easier and faster to create one melee and one ranged script that used [SerializedField], which can be reused quickly for different enemies. It would also drastically reduce the amount of scripts I'd need to create, making my files way more organised and easier to navigate.

I then started on getting the enemy to move. I was able to reuse a lot of code from the hammer, saving time, due to both objects needing to calculate directions and move non-stop.

```
private Transform player;  
private Vector3 directionVector;  
private float direction;
```

```
private void movement() {  
    Vector2 pos = transform.position;  
  
    if (directionVector.x > 0)  
    {  
        pos.x += speed * Mathf.Cos(direction) * Time.deltaTime;  
        pos.y += speed * Mathf.Sin(direction) * Time.deltaTime;  
    }  
    else  
    {  
        pos.x += speed * -Mathf.Cos(direction) * Time.deltaTime;  
        pos.y += speed * -Mathf.Sin(direction) * Time.deltaTime;  
    }  
}
```

1 reference

```
private void directionCalc()  
{  
    directionVector = (player.transform.position - transform.position).normalized;  
    direction = Mathf.Atan(directionVector.y / directionVector.x);  
}
```

This was the code that I came up with, however, when I ran it, the enemy didn't move.

Stage2Vid4

I added print(1) when it calculates direction, print(2) when update runs, and print(3) when changing the position vector. If the code is working somewhat as intended, this would tell me the section of code the error occurred in because it doesn't print what it's supposed to, allowing me to more easily find the issue.

Stage2Vid5

However, it printed everything, showing it should work fine, telling me the error is something else. I rechecked my code, helping me realise I never set the transform to equal pos. I made sure to change that.

Stage2Vid6

This made it move, which is a step in the right direction, but it would only ever move in one direction. This is because directionCalc() only ever runs once, meaning it will only travel in this initial direction. I put directionCalc() in the Update() method, because this means it gets the correct direction every frame, making it always travel towards the player.

Stage2Vid7

This worked, up until the enemy and player were on the same space. This is because the distance between the player and enemy is 0, so in the calculation for the direction, it will divide by 0, causing an error. I changed the code so that it would only run when they aren't on the same place.

```

if (transform.position != player.transform.position)
{
    directionCalc();
    movement();
}

```

This worked perfectly.

Stage2Vid8

I then started on the dash mechanic. I did this now because it will make the enemy attacks far easier to test, by allowing me to have invincibility whenever I want, avoiding damage if needed.

```

if (Input.GetKeyDown("space"))
{
    dash();
}

if (dashing > 0f)
{
    dashing = dashing - 1f*Time.deltaTime;
    h = tempH;
    v = tempV;
    speedV = speed*10;
    speedH = speed*10;
}
else {
    tempH = h;
    tempV = v;
}

```

```

private void dash() {
    dashing = 0.5f;
}

0 references
public bool getDashInvincibility() {
    return dashing > 0f;
}

```

I came up with this code for the dash. If the player presses space, a timer that counts down until the dash ends will begin. While this timer is active, 'h' and 'v' will be set to a temporary value equal to their value when the space bar is pressed, causing it to travel permanently in that direction whilst the dash is active, and the speed is increased to 10 times the base speed. I did it like this because this is the easiest way I could come up with to, in essence, freeze the direction of travel and increase the speed. I considered having the 'dashing' variable be a Boolean rather than a float that acts as a timer, however, it being a float lets me change its length so that the dash lasts for the perfect amount of time. I also added a getter that returns true if the 'dashing' timer is dashing. This getter will be used by enemies to decide if the player is invincible. Alternatively, I could've manipulated the player's collisions to be turned off when they dash, but that could have unforeseen consequences in the future if I want it to be able to collide with things when dashing, so I stuck with a simple getter. The code worked basically perfectly, but just **travelled too fast**.

Stage2Vid9

I changed the values of the 'dashing' timer, and the speed increase, so that it felt more fair and fun to use.

Stage2Vid10

```
if (Input.GetKeyDown("space") && dashCooldown==0f)
{
    dash();
    print(1);
}

if (dashing > 0f)
{
    dashing = dashing - 1f*Time.deltaTime;
    h = tempH;
    v = tempV;
}
else {
    tempH = h;
    tempV = v;
}

if (dashCooldown > 0f) {
    dashCooldown = dashCooldown - 1f * Time.deltaTime;
}
```

The dashing mechanic would be broken and make the game too easy if you can dash whenever you want, contrasting what my stakeholders want, as you can't ever be hit, so I wanted to add a cooldown to the dash. I came up with this code – a timer that starts when dash() is called, and stops you from dashing whilst active, checked with an IF statement. However, this code made you only be able to dash a single time (I'm spamming space in the video).

Stage2Vid11

I remembered '-Time.deltaTime' will decrease the dashCooldown by a float value, meaning it probably won't equal exactly 0, which is what the IF statement checks for. I simply changed it to check if dashCooldown <= 0, which worked perfectly.

Stage2Vid12

However, to avoid confusion from the player of when they can and can't get hit, so I wanted to change the player sprite to one that looks like its dashing to represent this better.

However, I didn't know how to change the sprite of the player, so I looked in the Unity documentations to figure it out.

(Unity, n.d.)

```

private SpriteRenderer spriteRenderer;

[SerializeField]
private Sprite dashingSprite;

[SerializeField]
private Sprite regularSprite;

Unity Message | 0 references
private void Start()
{
    spriteRenderer = GetComponent<SpriteRenderer>();
}

if (dashing > 0f)
{
    dashing = dashing - 1f*Time.deltaTime;
    h = tempH;
    v = tempV;
    spriteRenderer.sprite = dashingSprite;
}
else {
    tempH = h;
    tempV = v;
    spriteRenderer.sprite = regularSprite;
}

```

This code gets the player's SpriteRenderer component and simply changed the player's sprite to the regular one when it isn't dashing, and the dashing one when it is. It worked fine, but the sprite size needs reformatting. This can be done in the Aesthetics Stage later.

Stage2Vid13

I then realised I needed to add comments throughout my code. This is to help improve readability for myself and anyone else who looks at my code, making it easier overall to understand. On reflection, I should've been doing this throughout all my work thus far, and it will be something I shall try to do from now on. It also means you can read the comments to see how the code works, and I just need to justify it, giving me more time to develop my project. To avoid having to add screenshots of all the code, all comments are shown in the following video.

Stage2Vid14

I then decided to move all the player's stats into one centralised script. This script will contain all the stats, and getters and setters for each of them. This will make it far easier to alter these variables later, as it can all be done from one central location, instead of me having to find and get each individual component for a variable stored in that one location.

```

    Unity Script | 4 references
    public class PlayerStats : MonoBehaviour
    {
        [SerializeField]
        private float speed; //player speed

        [SerializeField]
        private int damage; //hammer damage

        private float hammerSpeed = 10f; //base speed of hammer
        private float exp=0; //starting exp (0)

        Unity Message | 0 references
        void Start()
        {
        }

        // Update is called once per frame
        Unity Message | 0 references
        void Update()
        {
        }

        1 reference
        public float getSpeed() { //returns speed
            return speed;
        }

        0 references
        public void setSpeed(float aspeed) { //sets speed
            speed = aspeed;
        }

        1 reference
        public int getDamage() //returns damage
        {
            return damage;
        }

        0 references
        public void setDamage(int adamage) //sets damage
        {
            damage = adamage;
        }

        1 reference
        public float getHammerSpeed() //returns hammer speed
        {
            return hammerSpeed;
        }

        0 references
        public void setHammerSpeed(float ahammerSpeed) //sets hammer speed
        {
            hammerSpeed = ahammerSpeed;
        }

        0 references
        public float getEXP() //returns exp
        {
            return exp;
        }

        0 references
        public void setEXP(float EXP) //sets exp
        {
            exp = EXP;
        }
    }

```

```

    playerStats = GetComponent<PlayerStats>();
    speed = playerStats.getHammerSpeed(); //gets the hammer speed from the stats script
    damage = playerStats.getDamage(); //gets the damage from the stats script

```

```

    playerStats = GetComponent<PlayerStats>();
    speed = playerStats.getSpeed(); // gets player speed from the stats script

```

I then started on the melee attack method, which I did first as it should be easier than the ranged one, whilst providing a foundation for it.

```

        if (collision.gameObject == player) { //if it collides with the player
            GameObject plyr = collision.gameObject;
            attack(plyr); //passes in the player game object to the attack method
        }
    }

private void attack(GameObject plyr) {
    playerStats = plyr.GetComponent<PlayerStats>(); //gets the stats and movement components
    playerMovement = plyr.GetComponent<PlayerMovement>();
    if (playerMovement.getDashInvincibility() == false) //makes sure the player isn't dashing (invincible)
    {
        playerStats.damaged(damage); //damages player
    }
}

public void damaged(int damage) {
    health = health - damage; //reduces player health by damage
    if (health <= 0) { //if the player should die from the attack
        Destroy(this.gameObject); //destroys player
    }
}

```

The code for attacking is split into 3 parts – the collision part, the attack part, and the damaging part - which makes it easier for me to work out where any issues that may arise have stemmed from. The comments explain how the code should have worked. This code didn't work, but also added some issues with how the enemy moved, causing it to go up or down after it collides with the player.

Stage2Vid15

To diagnose the issue, I added print statements that print 1 when it collides, 2 when it attacks, and the health variable when the player takes damage. Nothing printed, telling me there was most likely an issue with the collision detection.

```

if (collision.gameObject.CompareTag("Player")) { //if it collides with the player
    print(1);
    GameObject plyr = collision.gameObject;
    attack(plyr); //passes in the player game object to the attack method
}

```

I changed the if statement that detects collision with the player to something more similar to the code I used for the hammer's collision detection, as I know that's something that works, however this still didn't work. This tells me I probably messed something up in the editor when altering the Rigidbody or Colliders.



Originally, in the player's Rigidbody, it was excluding every layer, which I thought might mean it never collided with anything. I changed it to include everything and exclude nothing, meaning the collision between the enemy and player should now be detected.

This worked but got some errors towards the end after the player has been defeated.

Stage2Vid16

! [09:34:34] MissingReferenceException: The variable player of CameraMovement doesn't exist anymore. You probably need to reassign the player variable of the 'CameraMovement' script in the Inspector.

! [09:34:34] MissingReferenceException: The variable player of ParentMeleeEnemy doesn't exist anymore. You probably need to reassign the player variable of the 'ParentMeleeEnemy' script in the Inspector.

These errors were easily solved by adding `if(player!=null)` before any calculations that use the player's variables.

```
if (player != null)
{
    pos = player.position;
    pos.z = -10; //makes sure the z position of the camera never moves
    transform.position = pos;
}
```

```
if (transform.position != player.transform.position && player!= null)
{
```

I added this everywhere I could think of, but I still missed one.

! [09:38:08] MissingReferenceException: The object of type 'UnityEngine.Transform' has been destroyed but you are still trying to access it. Your script should either check if it is null or you should not destroy the object.

```
if (player != null)
{
    if (transform.position != player.transform.position) //makes sure the enemy isn't at the same location as the player
    {
        directionCalc();
        movement();
    }
}
```

With how the code works currently, the enemy would only attack once per collision, meaning if the player sat on the enemy, they wouldn't take any damage from it. This made the enemies feel non-threatening, so I wanted to change it.

Collision Detection Continuous

```
private float attackTimer = 0;
```

```
if (playerMovement.getDashInvincibility() == false && attackTimer<=0) //
{
    playerStats.damaged(damage); //damages player
    attackTimer += 1;
}
```

I added a timer for when it can attack, which both makes the enemies easier to balance and customise. I also changed the collision detection to continuous, as I thought that may mean it would continuously collide, fixing my issue. Neither of these helped, however, so I looked at the Unity Documentation for collision detection to see how I could make this happen. From this, I learned about `OnCollisionStay2D()`, which is the method I need to use to continuously detect collisions. I implemented this and it worked.

(Unity Technologies, n.d.)

```
private void OnCollisionStay2D(Collision2D collision) //every frame that there are collisions
{
    if (collision.gameObject.CompareTag("Player"))
    { //if it collides with the player
        GameObject plyr = collision.gameObject;
        attack(plyr); //passes in the player game object to the attack method
    }
}
```

Stage2Vid17

```
private void damaged(int hDamage)
{
    health -= hDamage; //decreases health by the damage of the hammer
    if (health <= 0)
    {
        GameObject plyr = GameObject.Find("Player"); //gets the player
        PlayerStats playerStats = plyr.GetComponent<PlayerStats>(); //gets the players stats
        playerStats.setEXP(playerStats.getEXP() + expYield); //increases the player's total exp by expYield
        print(playerStats.getEXP());
        Destroy(this.gameObject); //destroys this game object
    }
}
```

I then started working on changing the player's exp when they defeat an enemy. I added the print() statement so I could see what exp I was at when I tested the code. I also made sure to put all the exp code before the Destroy() statement, otherwise the object would be destroyed before this code worked. This worked perfectly.

Stage2Vid18

```
public void levelCalc()
{
    if (exp >= nextLevelBarrier)
    {
        level += 1;
        exp = exp - nextLevelBarrier;
        nextLevelBarrier = (nextLevelBarrier + 5) * level;
        GameObject lvlUp = Instantiate(effect);
        Vector2 lvlUpPos = transform.position;
        lvlUpPos.y = lvlUpPos.y + 1;
        lvlUp.transform.position = lvlUpPos;
    }
}
```

I then worked on making the player level up when they have enough exp. This code checks if the player's exp is more than the next exp barrier, and if it is, increases your level and decreases your total exp. It will then calculate the next exp barrier. I considered having it be the same amount of exp needed to level up every level, but I think this will give the game a better sense of scaling and also lets me make harder enemies feel more rewarding to defeat, as they can now give more exp than basic enemies.

I also wanted to create a special effect indicator for levelling up, so that I know when you've levelled up for testing, and the player knows they've levelled up for gameplay. I wanted it to be a little sprite that popped up when you level up and gradually loses opacity until it disappears. I wasn't sure how to change opacity however, so I had to find the answer in a Unity discussion forum.

(zombiegorilla, n.d.)

```

using UnityEngine;

@ Unity Script (1 asset reference) | 0 references
public class EffectScript : MonoBehaviour
{
    float timer = 1; //timer until it despawns
    float opacity = 1; //opacity
    SpriteRenderer spriteRenderer;
    @ Unity Message | 0 references
    void Start()
    {
        SpriteRenderer spriteRenderer = GetComponent<SpriteRenderer>();
    }

    // Update is called once per frame
    @ Unity Message | 0 references
    void Update()
    {
        spriteRenderer.color = new Color(1f, 1f, 1f, opacity); //sets the opacity to the new opacity
        opacity = 1-Time.deltaTime; //decreases opacity
        timer = timer-Time.deltaTime; //decreases timer
        if (timer <= 0) {
            Destroy(gameObject); //destroys game object if the timer runs out
        }
    }
}

```

I created this script that can be reused for each effect. It used a timer which is tied to the opacity, and when it runs out, the effect object is destroyed. However, when I ran the code, I got this error:

 [16:54:35] NullReferenceException: Object reference not set to an instance of an object
EffectScript.Update () (at Assets/Scripts/EffectScript.cs:16)

```

@ Unity Message | 0 references
void Start()
{
    spriteRenderer = GetComponent<SpriteRenderer>();
}

```

This error meant that I was using a component not in the code, so I changed the start() method to this, and it got rid of the error, but the opacity still wasn't changing at all.

Stage2Vid19

```

opacity = opacity-Time.deltaTime; //decreases opacity
timer = timer-Time.deltaTime; //decreases timer

```

I noticed that this might've been because opacity was 1-time, not the previous opacity - time, meaning the opacity never changed at all.

Stage2Vid20

I added enemies back in to test levelling up. This worked perfectly (as seen by the level up effect appearing).

Stage2Vid21

I then started working on the ranged enemy code. It is largely similar to the melee enemy code, so I started by copying all that over and slightly altering it, adding attributes for range and distanceFromPlayer.

```

[SerializeField]
private int health; //how much damage the enemy can take
[SerializeField]
private int damage; //how much damage the enemy deals to the player
[SerializeField]
private float speed; //how fast the enemy will move
[SerializeField]
private float attackRate; //how fast the enemy will attack
[SerializeField]
private int expYield; //how much exp the enemy gives
[SerializeField]
private float range; //how close it will try to get to the player

private float attackTimer = 0;

private Transform player;
private Vector3 directionVector;
private float direction;
PlayerStats playerStats;
PlayerMovement playerMovement;
private float distanceFromPlayer;

```

I then worked on getting it to move correctly, meaning it would move towards the player until it was in a certain radius, where it'd stop. This code worked perfectly.

```

private void Update()
{
    if (player != null)
    {
        distanceFromPlayer = (player.position - transform.position).magnitude; //calculates distance from the player
        if (distanceFromPlayer > range) //checks if enemy is within range of the player
        {
            directionCalc();
            movement();
        }
    }
}

```

Stage2Vid22

I then started on getting it to attack. It needed code for the projectile, too, so I designed that first. It would need to get a lot of attributes from the enemy that fired it, so I decided to create the SetEnemy() function, that would work as a constructor, and set all the attributes needed to that of the enemy that wields the projectile. Everything else I could take from the Hammer code, simply changing what the projectile is fired towards. I did this because it saved me a lot of time and testing.

```

@ Unity Script (1 asset reference) | 2 references
public class EnemyProjectile : MonoBehaviour
{
    int damage; //the amount of damage the projectile will do
    float speed; //how fast the projectile should travel
    float direction; //the direction the projectile will travel in
    float range; //how far the projectile will travel

    private GameObject player;

    private GameObject enemy;

    private ParentRangedEnemy enemyStats;

    private Vector3 directionVector; //the direction vector of the direction it should travel in
    private bool ready=false; //checks if setEnemy() has happened
    float timer;

    1 reference
    public void setEnemy(GameObject thisEnemy)
    {
        enemy = thisEnemy;
        enemyStats = GetComponent<ParentRangedEnemy>();
        player = GameObject.Find("Player");
        speed = enemyStats.getProjectileSpeed(); //gets the projectile speed from the stats script
        damage = enemyStats.getDamage(); //gets the damage from the stats script
        ready = true;
        timer = 1.2f*(range / speed); //uses time=distance/speed to calc how long it should travel for, plus a little extra
        directionCalc();
    }

    private void Update()
    {
        if (ready)
        {
            Vector2 pos = transform.position;

            if (directionVector.x > 0) //checks if it is moving in a positive direction
            {
                pos.x += speed * Mathf.Cos(direction) * Time.deltaTime;
                pos.y += speed * Mathf.Sin(direction) * Time.deltaTime; //calculates the new location
            }
            else
            {
                pos.x += speed * -Mathf.Cos(direction) * Time.deltaTime;
                pos.y += speed * -Mathf.Sin(direction) * Time.deltaTime;
            }

            transform.position = pos; //sets the new position for the projectile

            timer=timer-Time.deltaTime;
            if (timer <= 0) {
                Destroy(gameObject);
            }
        }
    }

    1 reference
    private void directionCalc()
    { //method for calculating the direction
        directionVector = (player.transform.position - enemy.transform.position).normalized; //calculates the direction vector
        direction = Mathf.Atan(directionVector.y / directionVector.x); //calculates the angle for the direction the projectile should travel in
    }
}

```

Once I had the projectile, I made the code in the enemy's script for attacking. In the attack() method, it would create the projectile, and use setEnemy() to give the projectile all necessary attributes. I added attack() in update(), and had it alter the attackTimer once attack() was called. This is because, otherwise, there would be too many projectiles, and the game would be too difficult and crash.

```

private void attack()
{
    GameObject Projectile = Instantiate(projectile); //creates the projectile
    Vector2 projectilePos = transform.position;
    Projectile.transform.position = projectilePos;
    projectileStats = Projectile.GetComponent<EnemyProjectile>();
    projectileStats.setEnemy(this.gameObject); //sets this as the projectile's enemy
}

```

```

        if (attackTimer <= 0) {
            attack();
        }
    }
    if (attackTimer >= 0)
    {
        attackTimer -= Time.deltaTime * (attackRate/8); //decreases timer proportionally to attack rate
    }
}

```

This code however, didn't work and returned this error.

Stage2Vid23

 [18:27:50] NullReferenceException: Object reference not set to an instance of an object
EnemyProjectile.setEnemy (UnityEngine.GameObject thisEnemy) (at Assets/Scripts/EnemyProjectile.cs:25)

That meant I had to get the ParentRangedEnemy component, so I added that in.

```

enemy = thisEnemy;
enemyStats = enemy.GetComponent<ParentRangedEnemy>();
player = GameObject.Find("Player");

```

That did stop the error, and an arrow spawned, but it didn't move anywhere

Stage2Vid24

To test what the error could be, I added print(1) in setEnemy(). It was just printing every frame.

```

projectileStats.setEnemy(this.gameObject); //sets this as the enemy
Projectile = null; //resets Projectile and projectileStats
projectileStats = null;
attackTimer += attackRate; //restarts timer until it can attack

```

I realised it was probably creating a new Arrow every frame, probably because I wasn't resetting attackTimer(), so I started resetting it. I also set the Projectile and projectileStats to null so that they won't be affected after they're initialised.

Stage2Vid25

This video shows that the arrows are getting summoned fine, just disappearing way too quickly. I realised I was missing getRange(), which is why they disappear instantly.

```

public float getRange() {
    return range;
}

```

It worked, with the caveat that it was firing when you're not in their range, instead of being in their range.

Stage2Vid26

```

private void Update()
{
    if (player != null)
    {
        distanceFromPlayer = (player.position - transform.position).magnitude; //calculates distance from the player
        if (distanceFromPlayer > range) //checks if enemy is within range of the player
        {
            directionCalc();
            movement();
        }
        else if(attackTimer<=0) { //if the player's in range and the timer=0
            attack();
        }
    }
}

```

That was simple enough to change, just needing to move where attack() is called. I can use else if() here, which saves file size.

I then let the player be damaged by the arrows, by adding an OnCollisionEnter2D() method, which will call the player's damaged() method, in the same way it does for melee enemies, saving me time coding. This worked perfectly.

```
private void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Player")) {
        GameObject plyr = collision.gameObject; //takes the player as a game object
        playerStats = plyr.GetComponent<PlayerStats>(); //gets the stats and movement components
        playerMovement = plyr.GetComponent<PlayerMovement>();
        if (playerMovement.getDashInvincibility() == false) //makes sure the player isn't dashing (invincible), and enemy can attack
        {
            playerStats.damaged(damage); //damages player
        }
    }
}
```

Stage2Vid28

To finish off this stage, I needed to work on creating spawners. To get a general Spawner script, I'd need a global timer, because that can be used to start some spawners later on in a run. To do this, I'd need to make use of the UI elements, which was something I wasn't sure on how to do. I was able to create them from my own knowledge, however I needed to look up how to update the text on the timer.

(Raspberry Pi Foundation, n.d.)

```
using UnityEngine;
using TMPro;

// Unity Script | 0 references
public class Timer : MonoBehaviour
{
    [SerializeField]
    private float timer;

    private int intTime;

    [SerializeField]
    TextMeshProUGUI m_Object;

    // Unity Message | 0 references
    void Start()
    {
        intTime = (int) timer;
    }

    // Update is called once per frame
    // Unity Message | 0 references
    void Update()
    {
        timer=timer-Time.deltaTime;
        intTime = (int)timer;
        m_Object.text = intTime.ToString();
    }
}
```

Using the information I found, I was able to come up with this code. It would use both an integer and a float timer. This is because I can only make use of time.DeltaTime using floats, which I need to do to make sure the timer is equal across all frame rates, meanwhile an integer value for the timer is much simpler and easy to read for the player, due to it making use of less digits. This code worked perfectly.

Stage2Vid29


```

using System.Collections;
using UnityEngine;

// Unity Script (1 asset reference) | 0 references
public class Spawner : MonoBehaviour
{
    [SerializeField]
    private GameObject[] spawnTable; //the enemies it can spawn

    [SerializeField]
    private GameObject[] spawnMaxCounts; //the max quantity it will spawn

    [SerializeField]
    private GameObject[] spawnMinCounts; //the min quantity it will spawn

    [SerializeField]
    private float spawnRate; //how fast it will spawn enemies

    [SerializeField]
    private float timeUntilSpawn; //when it will start spawning

    // Unity Message | 0 references
    private void Start()
    {
        StartCoroutine(Spawning());
    }

    1 reference
    IEnumerator Spawning() {
        while (true) {
            GameObject spawn = Instantiate(spawnTable[Random.Range(0, spawnTable.Length)]); //creates a random enemy from the spawn table
            Vector2 newPos = transform.position;
            newPos.x = transform.position.x + Random.Range(-1f, 1f); //sets it to a random position around the spawner;
            newPos.y = transform.position.y + Random.Range(-1f, 1f);
            spawn.transform.position = newPos;
            yield return spawnRate;
        }
    }
}

```

I then created this code for the spawner. I used these various attributes so that I could customise each spawner to make them unique, diversifying the gameplay. I also made the enemy spawn randomly around the spawner so that enemies wouldn't clump up as much, improving readability of the game.

This worked, but it spawned far too many enemies.

Stage2Vid30

```

IEnumerator Spawning() {
    while (true) {
        GameObject spawn = Instantiate(spawnTable[Random.Range(0, spawnTable.Length)]); //creates a random enemy from the spawn table
        Vector2 newPos = transform.position;
        newPos.x = transform.position.x + Random.Range(-1f, 1f); //sets it to a random position around the spawner
        newPos.y = transform.position.y + Random.Range(-1f, 1f);
        spawn.transform.position = newPos;
        yield return new WaitForSeconds(1f / spawnRate); //makes it wait for the spawn rate before
    }
}

```

I looked back on the notes I made when learning Unity, and added in WaitForSeconds at the end of the sequence. This makes it so that there is a delay between the IEnumerator being called. I also used the reciprocal so that higher spawn rate makes it spawn faster, making it easier to visualize when developing. The code worked.

Stage2Vid31

To make combat more intense and interesting, I want the enemies to spawn randomly around the player, to create the feeling of them closing in on the player. To do that, I deduced the enemies must spawn either 15 units away from the player on the x axis, or 10 units away on the y axis. To get this to work I concocted the following code.

```

spawn.transform.position = newPos;
newPos = player.transform.position; //new position is the player's position
randomSpawnLocationNumber = Random.Range(0, 4); //randomly selects what side it will spawn on
if (randomSpawnLocationNumber == 0)
{
    newPos.x += 15; //spawns to the left of the player
    newPos.y += Random.Range(-10f, 10f);
}
else if (randomSpawnLocationNumber == 1) {
    newPos.x -= 15; //spawns to the right of the player
    newPos.y += Random.Range(-10f, 10f);
}
else if (randomSpawnLocationNumber == 2)
{
    newPos.y -= 10; //spawns below the player
    newPos.x += Random.Range(-15f, 15f);
}
else
{
    newPos.y += 10; //spawns above the player
    newPos.x += Random.Range(-15f, 15f);
}
transform.position = newPos;

```

To make sure the spawning felt random, I used a random number generator between 0 and 3 to determine where they would spawn, as that is the quickest and easiest way for me to simulate randomness. However, it wasn't spawning the enemies correctly. In the following video, I moved away from spawn, but the enemies only spawned around (0,0) instead of around the player.

Stage2Vid32

```

IEnumerator Spawning() {
    while (player!=null)
    {
        yield return new WaitForSeconds(2/spawnRate); //makes it wait for the spawn rate before
        Vector2 newSpawnerPos = player.transform.position; //new position is the player's position
        randomSpawnLocationNumber = Random.Range(0, 4); //randomly selects what side it will spawn on
        if (randomSpawnLocationNumber == 0)
        {
            newSpawnerPos.x += 15; //spawns to the left of the player
            newSpawnerPos.y += Random.Range(-10f, 10f);
        }
        else if (randomSpawnLocationNumber == 1) {
            newSpawnerPos.x -= 15; //spawns to the right of the player
            newSpawnerPos.y += Random.Range(-10f, 10f);
        }
        else if (randomSpawnLocationNumber == 2)
        {
            newSpawnerPos.y -= 10; //spawns below the player
            newSpawnerPos.x += Random.Range(-15f, 15f);
        }
        else
        {
            newSpawnerPos.y += 10; //spawns above the player
            newSpawnerPos.x += Random.Range(-15f, 15f);
        }
        transform.position = newSpawnerPos; //sets spawner to new player position

        GameObject spawn = Instantiate(spawnTable[Random.Range(0, spawnTable.Length)]); //creates a random enemy from the spawn table
        Vector2 newEnemyPos = newSpawnerPos;
        newEnemyPos.x = newSpawnerPos.x + Random.Range(-1f, 1f); //sets it to a random position around the spawner
        newEnemyPos.y = newSpawnerPos.y + Random.Range(-1f, 1f);
        spawn.transform.position = newEnemyPos;
    }
}

```

I thought it might work better if I decide the spawner location first, then spawn the enemies, so I swapped these around and it worked.

Stage2Vid33

I then started to work on getting it to spawn different quantities of enemies, once again to diversify gameplay and provide an occasional challenge more often.

```

for (int i = 0; i < spawnMinCounts[randomEnemyNum]; i++)
{
    Range(spawnMinCounts[randomEnemyNum], spawnMaxCounts[randomEnemyNum] + 1); i++)
    {
        GameObject spawn = Instantiate(spawnTable[randomEnemyNum]); //creates a random enemy from the spawn table
        Vector2 newEnemyPos = newSpawnerPos;
        newEnemyPos.x = newSpawnerPos.x + Random.Range(-1f, 1f); //sets it to a random position around the spawner
        newEnemyPos.y = newSpawnerPos.y + Random.Range(-1f, 1f);
        spawn.transform.position = newEnemyPos;
    }
}

```

I used a for loop, as this is the easiest way to control how many times to loop the code. This worked, seen by archers always spawning solo, but swordsman sometimes spawning in pairs.

Stage2Vid34

Lastly wanted to make it so that I can add a delay before it starts spawning, which will become useful for both the Boss' spawner, as well as allowing me to start spawning more difficult enemies later on in the game, giving the game a sense of progression to it.

```

public int getTime() {
    return intTime;
}

```

```

IEnumerator Spawning() {
    while (player != null)
    {
        yield return new WaitForSeconds(5/spawnRate); //makes it wait for the spawn rate before
        if (timerTimer.getTime() < 300-timeUntilSpawn)
        {
            Vector2 newSpawnerPos = player.transform.position; //new position is the player's position
        }
    }
}

```

To get this to happen, I needed a getter from the timer, as that means I can use that as a centralised clock for all things, not just the spawner. From there, a simple if statement at the start of the IEnumerator will suffice. For the following video, I used a 5 second delay.

Stage2Vid35

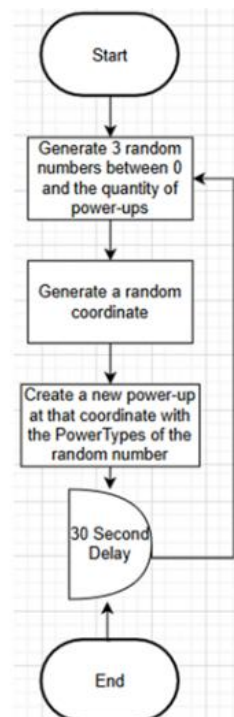
Test	Expected Result	Actual Result
Wait	Various enemies will spawn around the player	Various enemies spawn around the player Stage2Vid33
Wait	Enemies move towards the player	Enemies move towards the player Stage2Vid8
Press the 'Space' key	Move forward without taking damage	Move forward without taking damage Stage2Vid13
Be within a ranged enemy's range	Enemy fires a projectile that travels in the player's direction	Enemy fires a projectile that travels in the player's direction Stage2Vid26
Destroy an enemy	Increase exp attribute	Increase exp attribute Stage2Vid21
Collide with a melee enemy	Health attribute decreases	Health attribute decreases Stage2Vid17

Stage 3 – Power-Ups

Data Dictionary:

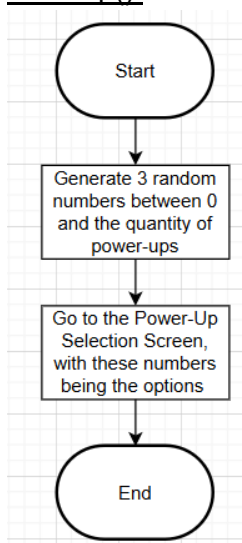
Class	Attribute	Data Type	Method	Description
Power-Up Controller	types[]	Integer		List of all power-up types
			Spawning()	Spawns power-ups periodically
			LevelUp()	Gives a power-up when you level up
Power-Ups	powerType	Integer		Number correlating to the type of power-up you've collected
			Collision()	Adds power-up type to the Player's power-up list
Player	powerUps[]	Integer		Power-ups the player has collected
	nextFreePowerUp	Integer		The next free index of the powerUps[] array
			AddPowerType()	Adds integer to powerUps[]
			Shooting()	Changing shooting to factor in the effects of different power-ups

Spawning():



Spawning will need to randomly generate a coordinate for the power-up to spawn on, as well as a random number associated with the power-ups it could be. I need 3 power-ups as I want there to be a choice that the player can make, meaning they can only take the power-ups they want. It will then generate the power-up at that random coordinate and start a delay before repeating. I wanted to add the delay to make sure there aren't too many power-ups constantly spawning, breaking the game.

LevelUp():



Level-up will be similar to Spawning(), because both are methods for collecting power-ups. but doesn't need to spawn the power-up, more simply sending the player straight to the power-up selection screen. I need to make sure to generate the power-ups before switching scenes, because otherwise I'll be passing a null value into the buttons, which would cause the game to crash.

Collision() - Pseudocode:

```
FUNCTION Collision(int powerType, GameObject Player)
    Player.addPowerType(powerType)
ENDFUNCTION
```

AddPowerType() -Pseudocode:

```
FUNCTION addPowerType(int powerType)
    powerUps[nextFreePowerUp] = powerType
    nextFreePowerUp += 1
ENDFUNCTION
```

Collision() and AddPowerType() work together to give you the power-up when you collide with the physical power-up. Collision() is the method within the power-up object, that passes all relevant data to AddPowerType() in the player object, allowing it to add the power-up to the list of power-ups. I also increase the nextFreePowerUp value every time addPowerType is called so that power-ups don't get overridden, making the player confused at why their power-ups have gone missing.

Test	Expected Result	Reason
Player collides with PowerUpGem object	Sends the player to the Power-Up Selection screen	This is how the player is intended to collect power-ups, so if this doesn't work, the player isn't able to collect any power-ups
Player clicks a button on the Power-Up Selection Screen	The value relating to the power-up is added to an array in PlayerStats	This needs to work to allow the player to have a choice in what power-ups they get
0 enters the PlayerStats power-up array	The damage attribute in PlayerStats increases	This ensures the damage power-up is functional
1 enters the PlayerStats power-up array	The maxHealth attribute increases	This ensures the health power-up is functional
2 enters the PlayerStats power-up array	Increase the speed attribute	This ensures the speed power-up is functional
3 enters the PlayerStats power-up array	Increase the health attribute every second	This ensures the passive healing power-up is functional
4 enters the PlayerStats power-up array	Whilst decrease enemy health attribute if you collide with them while dashing	This ensures the dash damage power-up is functional
Fire hammer collides with Enemy	The enemy's health attribute decreases gradually over time	This ensures the fire power-up is functional

Electricity hammer collides with Enemy	An Electricity gameObject is instantiated	This ensures the electricity power-up is functional
Electricity collides with Enemy	The enemy's health attribute is decreases, and an Electricity gameObject is instantiated	This ensures the electricity power-up is functional

Stage 3 Development:

To begin this stage, I wanted to develop the power-up gems, which the player should run into to gain a power-up.

```
public class PowerUpGemScript : MonoBehaviour
{
    [SerializeField]
    int powerType; //the number associated with the power-up

    private void OnCollisionEnter2D(Collision2D collision)//gets called whenever there are collisions
    {
        if (collision.gameObject.CompareTag("Player"))//checks if it collides with the player
        {
            GameObject player = collision.gameObject;
            PlayerStats playerStats = player.GetComponent<PlayerStats>();
            playerStats.addPowerUp(powerType); //adds this power-ups powerType to the array
        }
    }
}
```

The script I made for it will simply detect if there is a collision, and add the number associated with the power-up to something in PlayerStats, which I now need to make.

```
private int[] powerUps; //array of power-ups
private int nextFreePowerUp = 0; //the index of where the next power-up will go

public void addPowerUp(int powerType) //code for adding power-up to power-up list
{
    powerUps[nextFreePowerUp] = powerType;
    nextFreePowerUp += 1; //makes sure the next power-up goes into next index
}
```

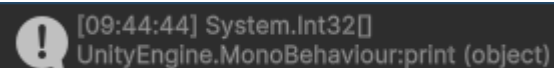
In PlayerStats, I created an array that will store the power-ups for the player. I chose an array because, as opposed to a stack or queue, I need to have code be able to access any item in the array. I also created the addPowerUp code, which simply adds the passed in value to the array, and increases the pointer value for the next power-up location. This allows me to constantly be able to rerun this code without writing over my old code. I added 'print(powerUps);' so I can test if this code.



However, I got this error, which I thought may be because I never set the size of the array.

```
powerUps = new int[100];
```

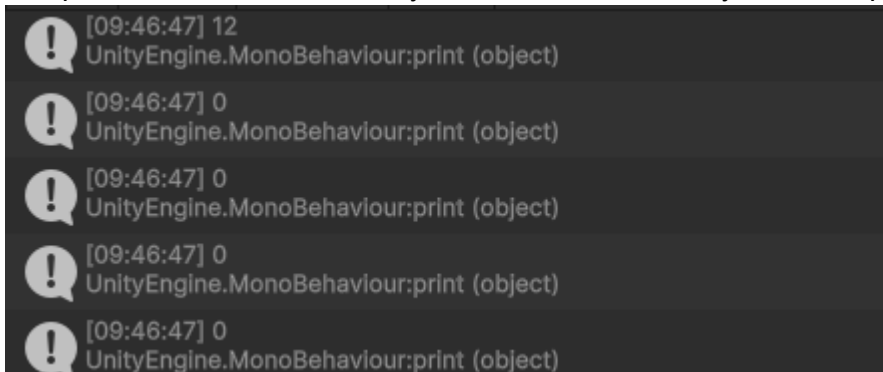
To attempt to fix this, I added this code, initialising the array as an integer array with a size of 100. 100 should be a fine size, because there should be no way to get more than 100 power-ups in a single run.



However, when I ran it again, it was just printing this, which isn't helpful information.


```
for (int i = 0; i < powerUps.Length; i++) {
    print(powerUps[i]);
}
```

To attempt to print the contents of the array, I added the above code, which loops through and prints each item in the array. This is the easiest way I know to print a whole array.



The screenshot shows the Unity console with five log entries. Each entry starts with a timestamp [09:46:47] and the text '0', followed by 'UnityEngine.MonoBehaviour:print (object)'. This indicates that the array contains 96 elements, all of which are '0'.

This printed the above (with 95 other '0's for the 95 other spaces in the array), showing it worked fine.

```
Destroy(this.gameObject);
```

I added code to destroy the power-up gem after it passes the value into the array. This is so that the player can't just collect power-ups from the same gem. I also made sure to destroy it at the last possible moment so that it doesn't get destroyed before any important code runs.

I then started working on the Power-Up Controller – the object that contains all code relating to power-ups that other scripts will want to access. I want this in a central object to make it easier for me to remember where all the attributes are stored, as they will all be in this one controller.

```
[SerializeField]
GameObject powerUp;

[SerializeField]
string[] hammerPowerUps; //array full of the list of hammer power ups

[SerializeField]
string[] characterPowerUps; //array full of the list of character power ups

int randP1;
int randP2;
int randP3; //the three numbers for the random power-ups

float randCoordX; //random coordinates
float randCoordY;

float timer = 1f; //timer until next spawn
```

I gave it these attributes. I used a float for the timer so that it has extremely specific values for the time.

```

void Update()
{
    timer=timer-Time.deltaTime;//decreases timer
    if (timer <= 0) { //when timer runs out, run spawnPowerUp() and reset timer
        spawnPowerUp();
        timer = 20f;
    }
}

```

I then put this in the Update() method. This code gradually decreases the timer, until it is less than 0 at which point I call the spawnPowerUp() method to create a power-up gem. The timer then resets. I want a power-up to spawn every 20 seconds as to ensure the player doesn't get too powerful and make the game too easy.

```

void spawnPowerUp() {
    randP1 = Random.Range(0, hammerPowerUps.Length); //sets randP1 to a random index in HPU
    if (hammerPowerUps.Length > 1) //checks if there is more than one HPU (there always should be)
    {
        while (randP2 != randP1)//makes sure randP2 and randP1 are different
        {
            randP2 = Random.Range(0, hammerPowerUps.Length); //sets randP2 to a random power-up in the array
        }
    }
    else {
        randP2= Random.Range(0, hammerPowerUps.Length);
    }
    if (hammerPowerUps.Length > 2) //repeats previous process but for randP3
    {
        while (randP3 != randP1 && randP3 != randP2)
        {
            randP3 = Random.Range(0, hammerPowerUps.Length);
        }
    }
    else
    {
        randP3 = Random.Range(0, hammerPowerUps.Length);
    }

    randCoordX = Random.Range(-118.3f, 99.3f); //gets any coordinate on the map
    randCoordY = Random.Range(-66.4f, 79.2f);

    GameObject spawnedGem = Instantiate(powerUp); //instantiates a power-up gem
    Vector2 newPos;
    newPos.x = randCoordX;
    newPos.y = randCoordY;
    spawnedGem.transform.position = newPos; //sets the PUG to the random position
}

```

This is the code I made for spawnPowerUp(). It needs to assign 3 random variables to the power-up gem it creates, which correlate to the power-ups the player can choose from. I cut out any repeated rolls for power-ups, ensuring the player encountered a wider range of power-ups, varying gameplay. I did this by using while loops, as they are the easiest way to repeat the code that makes sure nothing else can happen until the value of the power-up is unique. However, because no code can run until there are 3 unique power-ups, I added code which ensures there are 3 unique types of power-ups, so that there is no way for the program to get stuck in an infinite loop. Lastly, I made sure the place it spawned was random, spawning anywhere on the map, which encourages the player to explore the map instead of staying in one spot, which may eventually get boring.

```

public void setTypes(int p1, int p2, int p3) { //setter for the power type array
    powerType[0]=p1;
    powerType[1]=p2;
    powerType[2]=p3;
}

0 references
public int[] getTypes() { //getter for the power type array
    return powerType;
}

```

In the PowerUpGem code, I added getters and setters for the powerType array, in preparation for them to be used later.

```

PowerUpGemScript pugs = spawnedGem.GetComponent<PowerUpGemScript>();
pugs.setTypes(randP1,randP2,randP3); //sets the gems power-up types

```

I added code into spawnPowerUp() that makes use of the setter to set the PowerUpGem's powerType attributes.

To test this, I timer to equal 0.3, to increase the amount of gems spawning to make it easier to encounter them and test if they work.

Stage3Vid1

This video showed the spawning worked, but I got this error

```

[11:10:44] NullReferenceException: Object reference not set to an instance of an object
PowerUpControllefr.spawnPowerUp () (at Assets/Scripts/PowerUpController.cs:66)

```

I realised I never gave the PowerUpGem script to the PowerUpGem game object, which I quickly changed. This changed the error to the following, though.

```

[11:17:16] NullReferenceException: Object reference not set to an instance of an object
PowerUpGemScript.setTypes (System.Int32 p1, System.Int32 p2, System.Int32 p3) (at Assets/Scripts/PowerUpGemScript.cs:23)

```

```

for (int i = 0; i < 3; i++) {
    print(powerType[i]);
}

```

To test what the issue was, I added the above code to the collision code in the PowerUpGem script.

```

[11:25:38] 0
UnityEngine.MonoBehaviour:print (object)
[11:25:38] 0
UnityEngine.MonoBehaviour:print (object)
[11:25:38] 0
UnityEngine.MonoBehaviour:print (object)

```

It printed this after a collision, showing it was never receiving the p1, p2 and p3 attributes.

```

int[] powerType=new int[3]; //the number assosiated with the power-up
@ Unity Message | 0 references

```

I thought the issue may be that start() was called after setType(), meaning the attributes were being passed into an undeclared array, so I deleted Start() and changed powerType's declaration to this.

Stage3Vid2

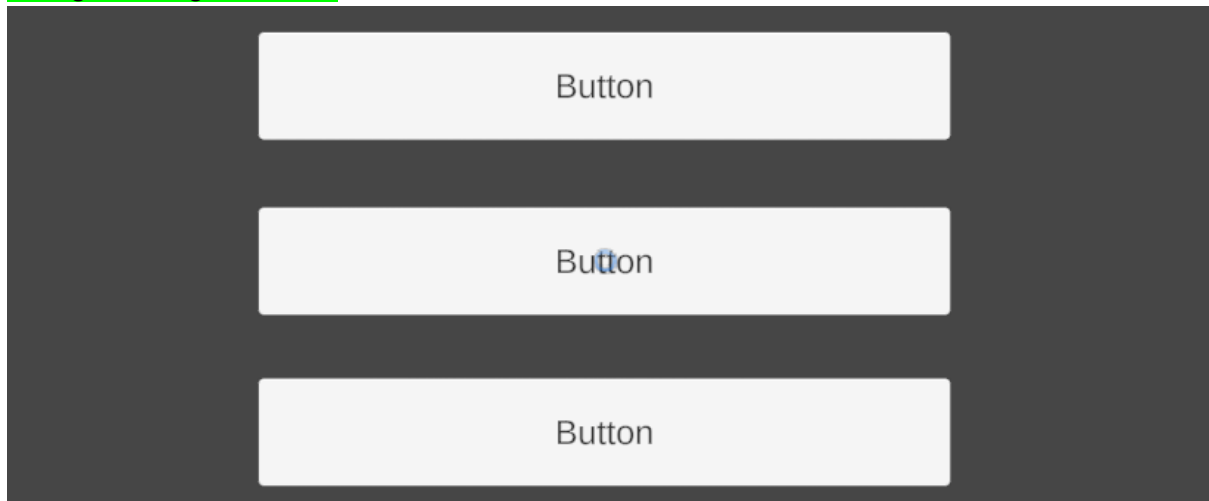
This worked better, removing the error, but the code printed the same number 3 times.

```

while (randP2 == randP1)//makes sure randP2 and randP1 are different
{

```

I saw that the code that should ensure there would never be repeating powerType's in a gem used != rather than ==, meaning it would repeat until all values are the same, which I changed, fixing the issue.



I then started working on power-up selection screen. The tutorial I followed to learn Unity contained instructions on how to make an alternative scene that allows you to use buttons to pass an attribute into the gameplay scene, which is exactly what I needed for the power-up selection screen.

(freeCodeCamp, 2025)

Using what I learnt from this video, I made 3 buttons names '1', '2', and '3'.

I made a ButtonController with the following code. It utilised the Singleton Pattern, which ensures only one ButtonController will ever exist between scenes by destroying any other ButtonController on scene change. This meant there would never be an instance where code gets confused on what ButtonController to get attributes from, which would cause issues.

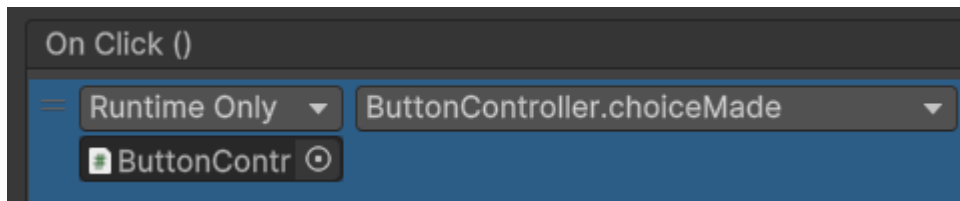
```
using UnityEngine;
using UnityEngine.SceneManagement;

// Unity Script (1 asset reference) | 1 reference
public class ButtonController : MonoBehaviour
{
    public static ButtonController instance; //creates a static ButtonController

    private int choice; //the choice the player makes on what power-up they want
    // Unity Message | 0 references
    void Awake()
    {
        if (instance == null) //if there is no button controller instance, it makes one
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else //if there was a button controller instance, it destroys this one
        {
            Destroy(gameObject);
        }
    }

    // 0 references
    public void choiceMade()
    {
        int clicked = int.Parse(UnityEngine.EventSystems.EventSystem.current.currentSelectedGameObject.name); //gets the name of the button clicked
        choice = clicked; //sets the player's choice to the button clicked
        SceneManager.LoadScene("Gameplay"); //sends you back to the gameplay scene
    }
}
```

I made the buttons run choiceMade() when they get clicked. This is because it is intuitive to the player to make their choice on the power-up they want by clicking on a button with its name on it.



The following code in ButtonController is called when you run into a PowerUpGem, and it is time to select a power-up. It uses Unity's inbuilt SceneManager to change the scene to the Power-Up Selection Screen, as that is a quick and simple way to swap scenes that I know how to utilise.

```
public void powerUpTime(int[] selection) { //sets the power-up selection
    powerUpSelection[0] = selection[0];
    powerUpSelection[1] = selection[1];
    powerUpSelection[2] = selection[2];

    sceneSwapped = -1; //signified we've swapped to power up selection screen
    SceneManager.LoadScene("Power-Up Selection"); //sends you back to the gameplay scene
}
```

I then altered choiceMade() to better fit the new code, taking the number from the button clicked and adding it to playerStats.

```
public void choiceMade()
{
    GameObject clicked = UnityEngine.EventSystem.current.currentSelectedGameObject; //gets the button clicked

    ButtonScript button = clicked.GetComponent<ButtonScript>();
    int choice = button.getPowerType(); //gets the PowerType of the button clicked

    SceneManager.LoadScene("Gameplay"); //sends you back to the gameplay scene

    GameObject player = GameObject.FindWithTag("Player");
    PlayerStats playerStats = player.GetComponent<PlayerStats>();
    playerStats.addPowerUp(choice); //adds this power-ups powerType to the array
}
```

I also made the script for the buttons. They would get the selection from ButtonController to decide what power-up would be associated with each button. The way it would know what power-up in the selection that it would have is decided by the buttonNum attribute that I would assign the button using the serialize field feature. This means I can use the same code for all buttons, and just alter this one variable to differentiate the buttons.

```
public class ButtonScript : MonoBehaviour
{
    [SerializeField]
    int buttonNum; //the number associated with this button

    int powerType;
    GameObject ButtonController;
    ButtonController buttonControllerScript;
    Unity Message | 0 references
    void Awake()
    {
        ButtonController = GameObject.FindWithTag("ButtonController"); //gets the button controller
        buttonControllerScript = ButtonController.GetComponent<ButtonController>();
        int[] selection = buttonControllerScript.getSelection(); //gets the selection array from the BC
        powerType = selection[buttonNum]; //gets the power type in the correct index
    }

    1 reference
    public int getPowerType() { //getter for powerType
        return powerType;
    }
}
```

```
public int[] getSelection() { //getter for powerUpSelection array
    return powerUpSelection;
}
```

Also I added code to the gem script that calls powerUpTime() when you run into it, allowing me to test out the power-up selection screen.

```
if (collision.gameObject.CompareTag("Player"))//checks if it collides with the player
{
    buttonControllerScript.powerUpTime(powerType);
    Destroy(this.gameObject);
}
```

When I collided with the gem, it changed the scene without any errors, but after the new screen loaded I ran into this error.

Stage3Vid3

```
[18:29:47] NullReferenceException: Object reference not set to an instance of an object
ButtonController.choiceMade () (at Assets/Scripts/ButtonController.cs:41)
```

This is probably because it was running the code to find the player before the new scene fully loaded, hence there was no player, so I moved stuff into the Update() function. I then made sceneSwap = 1 just before the new scene is loaded so that the program knows it has just changed scene, and can run the appropriate code.

```
private void Update()
{
    if (sceneSwap == 1)
    {
        GameObject player = GameObject.FindWithTag("Player");
        PlayerStats playerStats = player.GetComponent<PlayerStats>();
        playerStats.addPowerUp(choice); //adds this power-ups powerType to the array

        sceneSwap = 0;
    }
}
```

I tried it again with the new code, but still encountered the exact same error.

```
private void Update()
{
    if (sceneSwap == 1)
    {
        GameObject player = GameObject.Find("Player");
        if (player != null)
        {
            PlayerStats playerStats = player.GetComponent<PlayerStats>();
            playerStats.addPowerUp(choice); //adds this power-ups powerType to the array

            sceneSwap = 0;
        }
    }
}
```

I changed the code so that it wouldn't run until it found a player, and that worked, no longer giving me the error any more.

I then tried going from power-up selection screen to gameplay back to the power-up selection screen, and back to gameplay, however it didn't let me return to gameplay for a second time.

Stage3Vid4

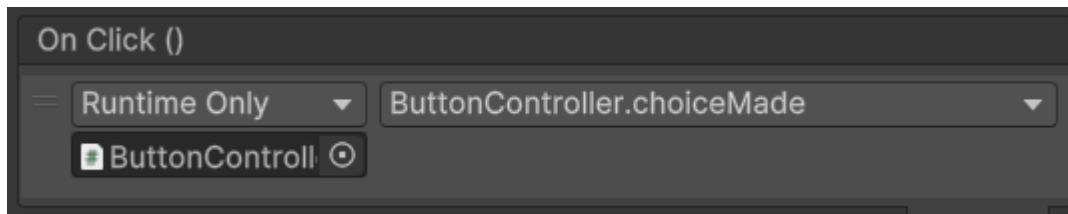
I thought unloading the power-up selection screen after clicking a button might help so I added this after Gameplay is loaded.

```
SceneManager.UnloadSceneAsync("Power-Up Selection");
```

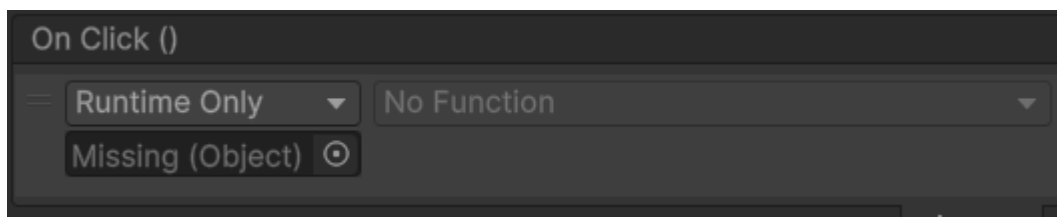
This didn't work and instead gave me this warning:

[19:03:08] Unloading the last loaded scene Assets/Scenes/Power-Up Selection.unity(build index: 1), is not supported. Please use SceneManager.LoadScene()/EditorSceneManager.OpenScene() to switch to another scene. UnityEngine.SceneManagement.SceneManager:UnloadSceneAsync (string)

I replaced 'UnloadSceneAsync' with 'LoadScene' again, and noticed that on the first click the button looked like this:



But after reloaded the scene it looked like this:



This means that the buttons have 0 function after reloading the power-up selection screen. After some more testing, I realised something – there was no simple way to keep the gameplay scene in the state it was after reloading it, meaning I had to find an alternative solution other than using scenes.

I decided it was best to have everything in one scene, and simply stop all movement and move the camera to a different location, making it appear like the scene has changed without actually having to change anything. This is because changing scenes restarts everything on the previous scene, meaning no progress will be saved and the game fully resets every time the player selects a power-up. This would be incredibly infuriating to any players who want to make any progress in the game.

I first altered ButtonController to not swap the scene, simply removing all code that relates to changing the scene.

```
public void powerUpTime(int[] selection) { //sets the power-up selection
    powerUpSelection[0] = selection[0];
    powerUpSelection[1] = selection[1];
    powerUpSelection[2] = selection[2];
}
```

```
private void Update()
{
    if (sceneSwap == 1)
    {
        if (player != null)
        {
            playerStats.addPowerUp(choice); //adds this power-ups powerType to the array
            print(2);
            sceneSwap = 0;
        }
    }
}
```


I then made it so the button controller actually creates the buttons during powerUpTime(). This is so they can appear when needed without being constantly loaded. To do this, they simply instantiate a prefab for the buttons, and then destroy them when a button is clicked. I also put the buttons in an array, so I can easily destroy all buttons by destroying the array.

```
[SerializeField]
GameObject b1; //button 1
[SerializeField]
GameObject b2; //button 2
[SerializeField]
GameObject b3; //button 3
```

```
GameObject bOne = Instantiate(b1); //creates the buttons
GameObject bTwo = Instantiate(b2);
GameObject bThree = Instantiate(b3);

buttons[0] = bOne; //saves the buttons in an array for later use
buttons[1] = bTwo;
buttons[2] = bThree;
```

```
Destroy(buttons[0]); //removes the buttons from existence
Destroy(buttons[1]);
Destroy(buttons[2]);
buttons[0] = null;
buttons[1] = null;
buttons[2] = null;
```

The following video shows that this code was creating the buttons, but they weren't visible.

Stage3Vid5

I believe this is because it is creating the object, just not under the Unity Canvas, which it needs to be to be visible in Unity.

```
this.transform.SetParent(canvas.transform); //makes the canvas its parent
```

(Unity, n.d.)

I didn't know how to make it a child of the Canvas, though, so I had to look up the Unity Documentation to figure it out.

Stage3Vid6

I decided against using an array to store the buttons in, and instead just tagged each button and used FindWithTag to find and destroy them. This saves memory, as an array won't need to be stored, and also removes the possibility of unforeseen issues that may arise when using an array.

```
GameObject button1 = GameObject.FindWithTag("Button1");
Destroy(button1);
GameObject button2 = GameObject.FindWithTag("Button2");
Destroy(button2);
GameObject button3 = GameObject.FindWithTag("Button3");
Destroy(button3);
```

This worked.

Stage3Vid7

To appear like it travels to a different screen, I'll move the camera whilst this the buttons are up. To do this, I changed cameraMovement to have the setMove() method, which will be used to change a attribute that decides if the camera is allowed to move to the player's location or not. I used an attribute to decide this as it means I can check if the camera can move from within the class, instead of having to create more methods and getters to get this

information from other classes, which is far more complex and will require me to make sure far more moving parts work perfectly together.

```
void Update()
{
    if (player != null && moving == true)
    {
        pos = player.position;
        pos.z = -10; //makes sure the z position of the camera never moves
        transform.position = pos;
    }
}

0 references
public void setMove(bool move)
{
    moving = move;
}
```

I added this code into the ButtonController's powerUpTime() method, which tells the camera to move to the coordinates (1000,0). At these coordinates I placed an image that will appear as the background for the power-up selection screen. This should create the appearance that the player has gone to a different screen without it ever actually having to change. This means I don't have to create a different scene just for power-up selection.

```
Camera cam = Camera.main;
CameraMovement cameraMovement = cam.GetComponent<CameraMovement>();
cameraMovement.setMove(false); //stops camera from moving
Vector3 pos = cam.transform.position;
pos.x = 1000;
pos.y = 0;
cam.transform.position = pos; //sets camera to the blue background location
```

I also added this code in choiceMade() which simply changes 'moving' in camera to true. This is all I have to do to change the camera's position back to the player's because it means that the update function can now run, which automatically changes the position of the camera to that of the player.

```
Camera cam = Camera.main;
CameraMovement cameraMovement = cam.GetComponent<CameraMovement>();
cameraMovement.setMove(true); //lets the camera move again
```

This worked perfectly.

Stage3Vid8

To stop things from moving when choosing a power-up, I created a pause controller. This is important because it means that nothing happens when the player can't see it, giving them more clarity and better control of what is going on, and avoiding unfair moments. This will be what decides when the game is paused, meaning nothing can move or activate, and when it can un-pause.

```

public class PauseController : MonoBehaviour
{
    public int unpaused = 1; //global variable for everything not moving

    0 references
    public void pause() //stops all movement and timers
    {
        unpaused = 0;
    }

    0 references
    public void unpaused() { //resumes all movement and timers
        unpaused = 1;
    }
}

```

It made 'unpaused' a public variable. This is because everything will constantly be accessing this attribute to know if it can move, and if it needs to be changed it will be changed for everything at once. The other option is for me to make it private and have getters and setters for it, however that means that hundreds of methods will have to run every frame to check if anything can move, and it would increase the size of the file massively. I altered all scripts wherever there is movement or a timer in a way like this:

```

pos.x += speed * Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused;
pos.y += speed * Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused;

```

This is because, if the game should be paused, 'unpaused' will equal 0, meaning nothing will change. This is easier and requires less computation than making 'unpaused' a Boolean value, and using IF statements everywhere.

I added these at the end of choiceMade() and powerUpTime() respectively, pausing the game when you should be at the other screen, and unpausing it when you make your decision.

```

PauseController pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
pauseController.unpaused();

PauseController pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
pauseController.pause();

```

This code worked, as you can see by the Timer freezing when paused.

Stage3Vid9

Then I went on to make the buttons display the text of the power-up they'd get, which will give the player an opportunity for decision making, making the game more deep and skill-based, which will allow me to increase the difficulty, which is something the stakeholders wanted.

I tried to do this through the following code – changing the text to that received by getting the name of the power-up from PowerUpController with the same powerType as the button.

```

[SerializeField]
TextMeshProUGUI buttonText;

PowerUpController powerUpController = GameObject.FindWithTag("PowerUpController").GetComponent<PowerUpController>();
buttonText.text = powerUpController.getHammer(powerType); //sets the button text to the text relating to the power-up's number

```

However, when I ran it, I got these errors.

```

[16:36:15] NullReferenceException: Object reference not set to an instance of an object
ButtonScript.Awake () (at Assets/Scripts/ButtonScript.cs:32)

[16:36:15] NullReferenceException: Object reference not set to an instance of an object
ButtonScript.Awake () (at Assets/Scripts/ButtonScript.cs:32)

[16:36:15] NullReferenceException: Object reference not set to an instance of an object
ButtonScript.Awake () (at Assets/Scripts/ButtonScript.cs:32)

```

I added the following to try and diagnose the place at which the issue occurs.

```
if (powerUpController != null) {  
    print(1);  
}  
if (buttonText != null) {  
    print(2);  
}
```

It printed 1 but not 2, which means that **buttonText was null.**

I realised this was at the start of Awake(), which was most likely causing the issue by running before there is anything to get the text component from.

```
void Awake()  
{  
    buttonText = GetComponent<TextMeshProUGUI>();  
}
```

Deleting this helped, but displayed an issue that slot 3 always displayed 'Fire', which is associated with type 0. This could mean that there is an error with the random number generator always rolling 0, however I think the more likely option is that **slot 3 simply has a null value, which defaults to 0, hence displaying 'Fire'.**



To test this, I put various print statements when choosing the power-up for the 3rd slot to see what was and wasn't functioning.

```
if (hammerPowerUps.Length > 2) //repeats previous process but for randP3  
{  
    while (randP3 == randP1 && randP3 == randP2)  
    {  
        randP3 = Random.Range(0, hammerPowerUps.Length);  
        print(1);  
    }  
}  
else  
{  
    randP3 = Random.Range(0, hammerPowerUps.Length);  
    print(2);  
}
```

It printed neither, meaning this code never ran.

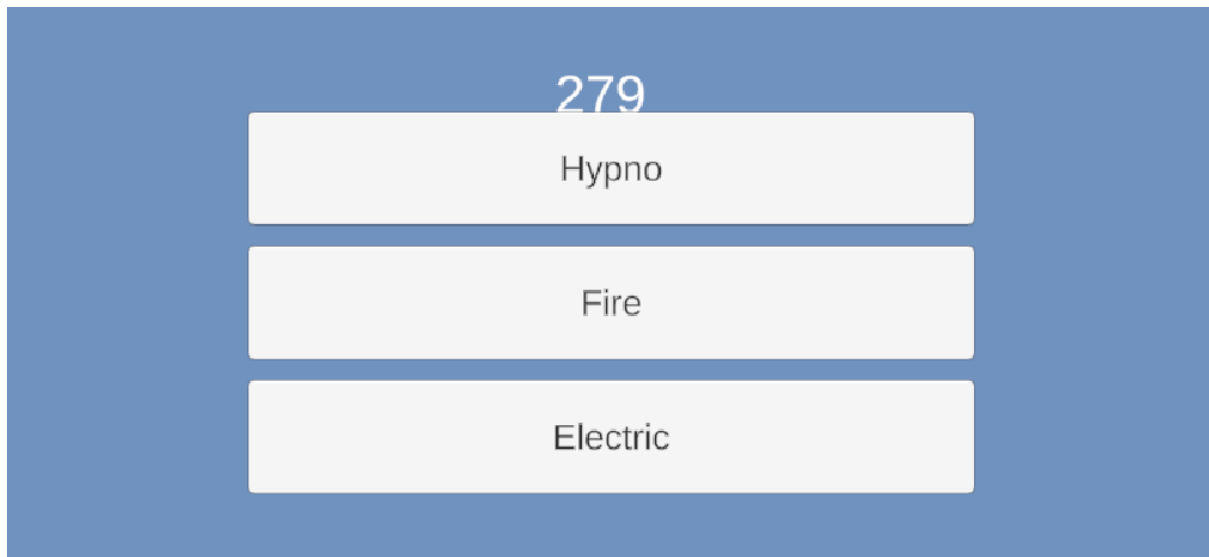
I realised this is because it won't ever run the while statement if the attributes for the random variables start out as different values, which it would be after randP1 is selected, so I changed it to this, meaning the code would run at least once.

```

if (hammerPowerUps.Length > 2) //repeats previous process but for randP3
{
    randP3 = Random.Range(0, hammerPowerUps.Length);
    while (randP3 == randP1 && randP3 == randP2)
    {
        randP3 = Random.Range(0, hammerPowerUps.Length);
    }
}
else
{
    randP3 = Random.Range(0, hammerPowerUps.Length);
}

```

This worked.



However, from further testing, I could see there was still an issue where it would sometimes repeat, which it shouldn't be doing.



This is because I used an AND instead of an OR in the following code, meaning it needed to be the same for all 3 slots to repeat, with it allowing 2 to be the same. Changing this to an OR fixes this, as if the 3rd slot is the same as either of the other slots, the code will repeat until it is different.

```
while (randP3 == randP1 || randP3 == randP2)
{
    randP3 = Random.Range(0, hammerPowerUps.Length);
}
```

This worked, as I haven't seen another repeat since the change.

Currently, the power-up selection screen is only built to handle hammer-type power-ups. To begin to fix this, I made this code for playerStats. It contains an array for the character power-ups, and a pointer to the next free one. This is the same system that is used for hammer-type power-ups, allowing me to not have to change the code that gives the player power-ups.

```
private int[] charPowerUps; //array of power-ups
private int nextFreeCPowerUp = 0; //the index of where the next power-up will go

public void charPowerUp(int powerType) //code for adding character power-up to power-up list, and changing their stats
{
    charPowerUps[nextFreeHPowerUp] = powerType;
    nextFreeCPowerUp += 1; //makes sure the next power-up goes into next index
}
```

Reusing the code that generates 3 power-ups to choose between from the power-up controller, I made this code which will activate when you level up. I can reuse the code here, as all it does is generate the numbers and pass it into a different method which gets the specific power-ups.

```
PowerUpController powerUpController = GameObject.FindWithTag("PowerUpController").GetComponent<PowerUpController>();
string[] characterPowerUps = powerUpController.getCharList();

randP1 = Random.Range(0, characterPowerUps.Length); //sets randP1 to a random index in HPU
if (charPowerUps.Length > 1) //checks if there is more than one HPU (there always should be)
{
    randP2 = Random.Range(0, characterPowerUps.Length);
    while (randP2 == randP1) //makes sure randP2 and randP1 are different
    {
        randP2 = Random.Range(0, characterPowerUps.Length); //sets randP2 to a random power-up in the array
    }
}
else
{
    randP2 = Random.Range(0, characterPowerUps.Length);
}
if (charPowerUps.Length > 2) //repeats previous process but for randP3
{
    randP3 = Random.Range(0, characterPowerUps.Length);
    while (randP3 == randP1 || randP3 == randP2)
    {
        randP3 = Random.Range(0, characterPowerUps.Length);
    }
}
else
{
    randP3 = Random.Range(0, characterPowerUps.Length);
}
```

To let the buttonScript know if it is dealing with hammer or character types, I added another value in the list that the buttonScript gets that is either 0 for hammer type or 1 for character type. I did this because it means I don't have to pass in anything more than just the array, keeping the code relatively simple and easy to understand.

This successfully told the button whether it was dealing with hammer or character type powerups, allowing it to display the correct text.

Stage3Vid10

I still needed to let the button controller to know how to deal with the power-ups, though, so I added a getter for the hammerOrChar value.

```
public int getHOC() {
    return selection[3];
}
```

ButtonController would get that type, and use it to decide if it should add the power-up to the array for character or hammer power-up types.

```
if (type == 0)
{
    playerStats.addPowerUp(choice); //adds this power-ups powerType to the array
}
else
{
    playerStats.charPowerUp(choice); //adds the character power up
}
```

At the end of charPowerUp() I added this. This should mean that if the player chooses the power-up with the powerType of 0, meaning they chose damage up, their damage increases. I did this because it is also easily expandable for future power-ups, as I can just add another IF statement. The test to see if this works is that, after collecting it, I should go from two-shotting the Skeleton Warriors, to one-shotting them, due to the bonus damage.

```
if (powerType == 0) //number associated with damage up
{
    damage += 3;
}
```

The following video demonstrates this worked.

Stage3Vid11

I then did a similar thing for speed and health. I used ELSE IF over IF because the player will only ever get one power-up at once, and if the powerType is low, it will save a lot of calculations as the game grows in scale and power-ups get added.

```
else if (powerType == 1) //number associated with speed up
{
    speed += 1;
}
else if (powerType == 2) { //number associated with health up
    health += 25;
    maxHealth += 25;
}
```

This also worked.

Stage3Vid12

This made me realise it would be best to have a short delay before the button appears so that the player knows they are choosing a power-up, and don't immediately accidentally click a button. This is useful because the player will be constantly clicking the screen, and if the button appears immediately, they may accidentally click it without reading what power-up they should get and may be confused at what changes have been made.

I made this code for before the buttons get instantiated. It instantiates a new effect, using the effectScript I previously created. I also moved the instantiation of the buttons to their own method. This is because it allows me to invoke it after a short wait, which I do because it is an easy way to create a delay that doesn't involve needing to code a timer.

```
GameObject effect = Instantiate(popUp); //creates the effect
Invoke("instantiateButton", 0.5f);
```



```
private void instantiateButton() {
    GameObject bOne = Instantiate(b1); //creates the buttons
    GameObject bTwo = Instantiate(b2);
    GameObject bThree = Instantiate(b3);
}
```

This worked.

Stage3Vid13

I then started on creating the passive healing power up. I created 2 values in playerStats, one of which would get increased when a power-up is added, and the other is a timer for when you should heal. I decided to use a timer because the healing will be in the update() function, which will be impractical to invoke as invoking a method every frame may get quite laggy.

```
private int passiveHealing = 0; //value relating to how much the player heals
private int healTimer; //timer for healing
```

```
else if (powerType == 4)
{
    passiveHealing += 1;
}
```

I then put this in update(). This is a simple timer for healing which is similar to timers I have used in the past. I made sure to check if health > maxHealth, meaning the player's current health will never usurp the maximum.

```
void Update()
{
    healTimer -= Time.deltaTime;
    if (healTimer <= 0) {
        health += 2 * passiveHealing; //heals every 4 seconds
        healTimer = 3;
    }
    if (health > maxHealth)
    {
        health = maxHealth; //ensures health never ursurps maxHealth
    }
}
```

To test it I put a print statement at the end which prints the current health. This is the best way to check if the healing functions properly.

```
void Update()
{
    healTimer -= Time.deltaTime;
    if (healTimer <= 0) {
        health += 2 * passiveHealing; //heals every 4 seconds
        healTimer = 3;
    }
    if (health > maxHealth)
    {
        health = maxHealth; //ensures health never ursurps maxHealth
    }
    print(health);
}
```

This worked.

Stage3Vid14

I felt it'd be best to rework dash damage into leaving a trail of fire behind you when you dash, because it is a more visually interesting and descriptive way of making the dash do damage.

To make this happen, I first created a variable and getter for the fireTrail attribute power up thing. I decided to store fireTrail as a variable in playerStats as opposed to a separate script because it means I don't need to get extra components in other scripts whenever I need to access fireTrail.

```
else if (powerType == 3)
{
    fireTrail += 1;
}
```

```
public int getFireTrail() {
    return fireTrail;
}
```

Using effectScript as a base, I created the fireTrail visuals. The main rework I did for now is making the timer equal to the amount of fireTrail you have. This means that collecting multiple fireTrails will feel impactful to the player.

```
public class FireTrailScript : MonoBehaviour
{
    private float timer; //timer until it despawns
    float maxTimer;
    float opacity = 1; //opacity
    SpriteRenderer spriteRenderer;
    PauseController pauseController;
    PlayerStats playerStats;
    Unity Message | 0 references
    void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>(); //gets the sprite renderer
        pauseController = GameObject.Find("PauseController").GetComponent<PauseController>(); //gets the pause controller
        playerStats = GameObject.FindWithTag("Player").GetComponent<PlayerStats>(); //gets the player's stats
        timer = playerStats.getFireTrail(); //sets the max time equal to the value of fire trail
        maxTimer = timer;
    }

    Unity Message | 0 references
    void Update()
    {
        spriteRenderer.color = new Color(1f, 1f, 1f, opacity); //sets the opacity to the new opacity
        opacity = opacity - ((Time.deltaTime / maxTimer) * pauseController.unpaused); //decreases opacity
        timer = timer - (Time.deltaTime * pauseController.unpaused); //decreases timer
        if (timer <= 0)
        {
            Destroy(gameObject); //destroys game object if the timer runs out
        }
    }
}
```

I then made it deal damage to the enemy. I did this by adding a collider that checks if it is colliding with an enemy, and then damages said enemy by 1. I made sure to get scripts for both melee and ranged enemies and damage both because there is no other way of knowing what type of enemy there is without tagging them separately, which would take me a long time and make me have to redo a lot of old code. This way I can simply check which one isn't null and damage that.

```
private void OnCollisionStay2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Enemy")) //if it touches an enemy
    {
        ParentMeleeEnemy melee = GetComponent<ParentMeleeEnemy>(); //gets the melee enemy script
        ParentRangedEnemy ranged = GetComponent<ParentRangedEnemy>(); //gets the ranged enemy script

        if (melee != null) {
            melee.damaged(1);
        }
        if (ranged != null) {
            ranged.damaged(1);
        }
    }
}
```

However, I was getting an error when trying to call the damaged. This is because the damaged() methods were set to private, so I changed that and the error went away.

The current type of collision would call every frame, doing way too much damage, so I want to include a damage cooldown. This would be the same as all other timers I have implemented. As per usual, I created a float variable for the timer.

```
float attackTimer = 0.25f; //cooldown between damage ticks
```

I added this in update to decrease and reset the timer if it is below 0.

```
if (attackTimer > 0) { //decreases attack timer
    attackTimer -= Time.deltaTime;
}
```

To implement this into attacking, I changed the collision code to the following. I also made it deal damage based on how much fireTrail you have as to make collecting more fireTrail feel more worth it for the player.

```
private void OnCollisionStay2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Enemy")) //if it touches an enemy
    {
        ParentMeleeEnemy melee = GetComponent<ParentMeleeEnemy>(); //gets the melee enemy script
        ParentRangedEnemy ranged = GetComponent<ParentRangedEnemy>(); //gets the ranged enemy script

        if (melee != null && attackTimer <= 0) {
            melee.damaged(playerStats.getFireTrail());
            attackTimer = 0.25f;
        }
        if (ranged != null && attackTimer <= 0) {
            ranged.damaged(playerStats.getFireTrail());
            attackTimer = 0.25f;
        }
    }
}
```

Finally, I need to get the fire to spawn while the player is dashing.

In playerMovement I added the following code. When the player dashes, if they have the fireTrail power-up, they'll spawn fire behind them in a short interval. I did this by invoking a method that instantiates the fire trail. I used invoking because it allows me to fully accurately control the time until it spawns, and if I used timers I would either have to use 3 different timers or create very specific bounds due to the use of floats.

```
[SerializeField]
GameObject fireTrail; //the fireTrail gameObject
```

```
private void dash() { //all the code that needs to be called once to set up the dash
    dashing = 0.2f;
    dashCooldown = 0.65f;
    if (playerStats.getFireTrail() > 0) {
        Invoke("spawnFire", 0.05f); //spawn fire after 0.05, 0.1 and 0.15 seconds
        Invoke("spawnFire", 0.1f);
        Invoke("spawnFire", 0.16f);
    }
}

private void spawnFire() {
    GameObject fire = Instantiate(fireTrail, transform); //spawns fireTrail at the current location
}
```

However, this didn't work, and instead the fireTrail stuck to the player.

Stage3Vid15

```
private void spawnFire() {
    GameObject fire = Instantiate(fireTrail); //spawns fireTrail at the current location
    fire.transform.position = transform.position;
}
```

I changed the way it sets the transform to this, which worked better. I believe the original made it so that the fireTrail's position was always equal to the player's position, not just when it instantiates. The way I did it this time is the more standard was that I've used up until now.

Stage3Vid16

This improved, however I felt the fire was too spread out, and it wasn't doing any damage to enemies. I first fixed the lack of fire by simply adding more.

```
Invoke("spawnFire", 0.03f); //spawns a fireTrail after short intervals
Invoke("spawnFire", 0.07f);
Invoke("spawnFire", 0.1f);
Invoke("spawnFire", 0.13f);
Invoke("spawnFire", 0.17f);
```

Stage3Vid17

I realised that this was because the enemies were on the wrong layer – PowerUpGem as opposed to Enemy, meaning no collisions were occurring.

However, changing this didn't fix it. I used print statements to see what was going wrong, and it showed collisions were occurring, but damage wasn't being dealt.

```
if (collision.gameObject.CompareTag("Enemy")) //if it touches an enemy
{
    ParentMeleeEnemy melee = GetComponent<ParentMeleeEnemy>(); //gets the melee enemy script
    ParentRangedEnemy ranged = GetComponent<ParentRangedEnemy>(); //gets the ranged enemy script
    print(3);

    if (melee != null && attackTimer <=0) {
        melee.damaged(playerStats.getFireTrail());
        print(2);
        attackTimer = 0.25f;
    }
    else if (ranged != null && attackTimer <=0) {
        ranged.damaged(playerStats.getFireTrail());
        print(1);
        attackTimer = 0.25f;
    }
}
```

I then noticed I was trying to get the fireTrail's components, not the enemies' ones, so I changed that by making ranged or melee equal collision.gameObject.

```
ParentMeleeEnemy melee = collision.gameObject.GetComponent<ParentMeleeEnemy>(); //gets the melee enemy script
ParentRangedEnemy ranged = collision.gameObject.GetComponent<ParentRangedEnemy>(); //gets the ranged enemy script
```

This fixed the issue.

Stage3Vid18

I then needed to work on Hammer type power-ups.

I changed the list in PowerUpController to be a 2D array, with the bottom row indicating the probability they trigger. This means all information anything will need about what hammer-type power-ups the player has will be stored in one array, making it far easier to work with.

```
[SerializeField]
string[][] hammerPowerUps; //array full of the list of hammer power ups, and the probability they trigger

public int getProbability(int i) { //returns the probability the power-up triggers
    return int.Parse(hammerPowerUps[i][1]); //returns the value in (i,1)
}
```

However, this meant that I couldn't serialize it, allowing me to edit it in the Unity editor, due to how Unity is designed, so I have to separate the arrays.

```
[SerializeField]
string[] hammerPowerUps; //array full of the list of hammer power ups, and the probability they trigger

[SerializeField]
int[] hammerProb; //the probability of the hammer power-up triggering
```

I made the `getProbability()` method which gets the integer in the `hammerProb` array which correlates with the chance of triggering the hammer power-up. It passes in an integer because I can make sure that the `powerType` is always passed in, meaning it returns the correct value.

```
public int getProbability(int i) { //returns the probability the power-up triggers
    return hammerProb[i]; //returns the value in (i,1)
}
```

I also changed the probabilities from an integer to a float to allow for rarer abilities and more precise trigger chances.

In the `PowerUpController`, I also put a getter for the probability of triggering, as well as the whole `hammerPowerUps` array.

```
private float[] hammerProb;

public int[] getHammerPowers() { //getter for hammer power-up list
    return hammerPowerUps;
}

0 references
public float[] getProb() { //getter for hammer power-up probabilities
    return hammerProb;
}
```

I created a new float attribute called 'sum' which acts as the sum of all probabilities for every hammer power-up the player has.

```
public void addPowerUp(int powerType) //code for adding power-up to power-up list
{
    hammerPowerUps[nextFreeHPowerUp] = powerType;
    hammerProb[nextFreeHPowerUp] = powerUpController.getProbability(powerType);
    sum += hammerProb[nextFreeHPowerUp]; //increases the sum of all probabilities
    nextFreeHPowerUp += 1; //makes sure the next power-up goes into next index
}
```

I then made this code so that their Hammers will randomly gain the power-up attributes based on the power-ups and their probabilities. It will first check if `sum > 100`. If it is, it will act differently, because a sum of over 100 means a power-up will trigger over 100% of the time, which is impossible. In either instance, it will generate a random number, either between 0 and `sum` for if `sum > 100`, or between 0 and 100 in any other case. It will then cycle through the items in the player's probabilities array, subtracting each probability from the random number. At the moment random becomes less than 0, that is the probability of the power-up that will be attached to the hammer. I use a switch case statement to check which power-up should be attached because it makes the code look cleaner, easier to format, and may help it run faster than using multiple IF statements.

```

PlayerStats playerStats = GameObject.FindWithTag("Player").GetComponent<PlayerStats>(); // gets the players stats
float[] probabilities = playerStats.getProb();
int[] powerUps = playerStats.getHammerPowers();
float sum = playerStats.getSum(); //gets all relevant values

int powerType=-1;
float random;

if (sum > 100)
{
    random = Random.Range(0, sum); //generates a random value between 0 and sum
}
else
{
    random = Random.Range(0, 100); //generates a random value between 0 and 100
}

for (int i = 0; i < probabilities.Length; i++)
{
    random -= probabilities[i]; //decreases the random value by the probability power-up i triggers
    if (random <= 0) //if that probability makes random < 0
    {
        powerType = powerUps[i]; //chooses that power-up to be used on the hammer
        break; //stops the loop
    }
}

if (powerType != -1) {
    switch (powerType) {
        case 0:
            Hammer.AddComponent<PUFire>(); break;
    }
}

```

I then made the script for PUFire (power-up fire). To keep my work organised, all the power-up scripts will be formatted like PU'name'.

First I made the hammer turn red, providing visual feedback to the player.

```

public class PUFire : MonoBehaviour
{
    SpriteRenderer spriteRenderer;

    [Unity Message | 0 references]
    private void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.color = Color.red;
    }
}

```

When testing this, though, this showed me an issue: despite not collecting any power-ups, the hammers were red, signifying they count as having the PUFire script.

Stage3Vid19

This is because, if I have no power-ups, hammerProb is empty, meaning sum=0, meaning random=0, meaning all hammers will have the powerType 0.

To solve this, I wrapped all the code in an IF statement that will only trigger if you have a power-up.

```

if (sum > 0)
{

```

This still didn't work. However, when I checked out the Hammer prefab to see if something was wrong there, I found this.



This must mean that using `AddComponent<>` will add the component permanently to the prefab, not once to the object. To fix that, I can simply change it so that it adds the component to the hammer instance, not the Hammer game object.

```
hammer.AddComponent<PUFire>();  
break;
```

And, with that change, it seemed to work.

Stage3Vid20

I then added the code for when it collides with an enemy. I thought it would be best to add a new component called `OnFire` to the enemy so that I don't have to fill both enemy scripts with situational code that will still need to be calculated every frame even when not relevant.


```

public class PUFire : MonoBehaviour
{
    SpriteRenderer spriteRenderer;

    [Unity Message | 0 references]
    private void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.color = Color.red; //sets the hammer colour to red to signify they are on fire
    }

    [Unity Message | 0 references]
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy")) {
            collision.gameObject.AddComponent<OnFire>(); //adds the OnFire component to the enemy
        }
    }
}

```

I then designed the OnFire script for enemies. It turns the enemy red, providing visual feedback to the player, and begins a coroutine to repeatedly damage the enemy all whilst a counter for the amount of times the enemy has been damaged increases. When the enemy has been damaged 5 times, the script will turn the enemy white again, and destroy itself. I used an enumerator because it's the easiest way to permanently repeat code, and means I don't need to add another timer.

```

public class OnFire : MonoBehaviour
{
    SpriteRenderer spriteRenderer;
    ParentMeleeEnemy melee;
    ParentRangedEnemy ranged;

    int damageCount=0; //count for the amount of fire damage the enemy has took

    [Unity Message | 0 references]
    private void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        melee = GetComponent<ParentMeleeEnemy>();
        ranged = GetComponent<ParentRangedEnemy>();
        spriteRenderer.color = Color.red; //sets the enemy colour to red to signify they are on fire
        StartCoroutine(fireDamage());
    }

    [1 reference]
    IEnumerator fireDamage()
    {
        yield return new WaitForSeconds(0.5f); //damages the enemy ever half a second
        if (melee != null)
        {
            melee.damaged(2);
        }
        else if (ranged != null)
        {
            ranged.damaged(2);
        }
        damageCount += 1;
        if (damageCount == 5) { //if the enemy has taken 5 ticks of fire damage
            spriteRenderer.color = Color.white; //resets the enemy's colour
            Destroy(this); //destroys this script
        }
    }
}

```

This seemed to work.

Stage3Vid21

However, I noticed that the Enumerator would most likely run even when Pause is active, which is unintuitive and incentivises the player to wait on paused screens, which I changed by doing this:

```
yield return new WaitForSeconds(0.5f); //damages the enemy every half a second
if (pauseController.unpaused == 1)
{
    if (melee != null)
```

I also realised that I needed a while(true) to make it repeat.

```
IEnumerator fireDamage()
{
    while (true)
    {
```

I then started on the electricity power-up to finish this stage. I created the PUElectricity script, which would just make the hammer yellow and instantiate an Electricity gameObject on collision. I kept it simple because most of the code for this power-up should be in the Electricity script.

```
public class PUElectric : MonoBehaviour
{
    [SerializeField]
    GameObject electricity;

    PauseController pauseController;

    SpriteRenderer spriteRenderer;
    Unity Message | 0 references
    void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.color = Color.yellow;
        pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
    }

    Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        while (true)
        {
            if (collision.gameObject.CompareTag("Enemy") && pauseController.unpaused == 1)
            {
                GameObject elecBall = Instantiate(electricity);
                elecBall.transform.position = transform.position;
                break;
            }
        }
    }
}
```

I put this code in the enemy scripts. This would stop the enemy from moving when electrocute() is triggered, and set an electrocuted variable to 1. I did this so that I can check if the enemy has been previously electrocuted meaning I can make sure it doesn't get electrocuted again and get permanently frozen, as that could make the game too easy.

```

public void electrocute() {
    if (electrocuted != 1) { //checks if the enemy has been electrocuted before
        frozen = 0; //freezes the enemy
        Invoke("unfreeze", 0.8f); //unfreezes the enemy in 0.8 seconds
    }
    electrocuted = 1;
}

0 references
public int getElectrocuted()
{
    return electrocuted;
}

0 references
public void unfreeze()
{
    frozen = 1;
}

```

I then made the Electricity script. It would first set the opacity to slightly lower, so that the player can see what is happening behind the effect, preventing them from being ambushed, which may have caused frustration. It also destroys itself after 0.2 seconds to not take up too much of the screen. If it collides with an enemy, it calls its electrocute() method and damages it. It will also create a new Electricity gameObject if the enemy hasn't been recently electrocuted, creating a cool visual effect and covering more area.

```

[SerializeField]
GameObject electricity;

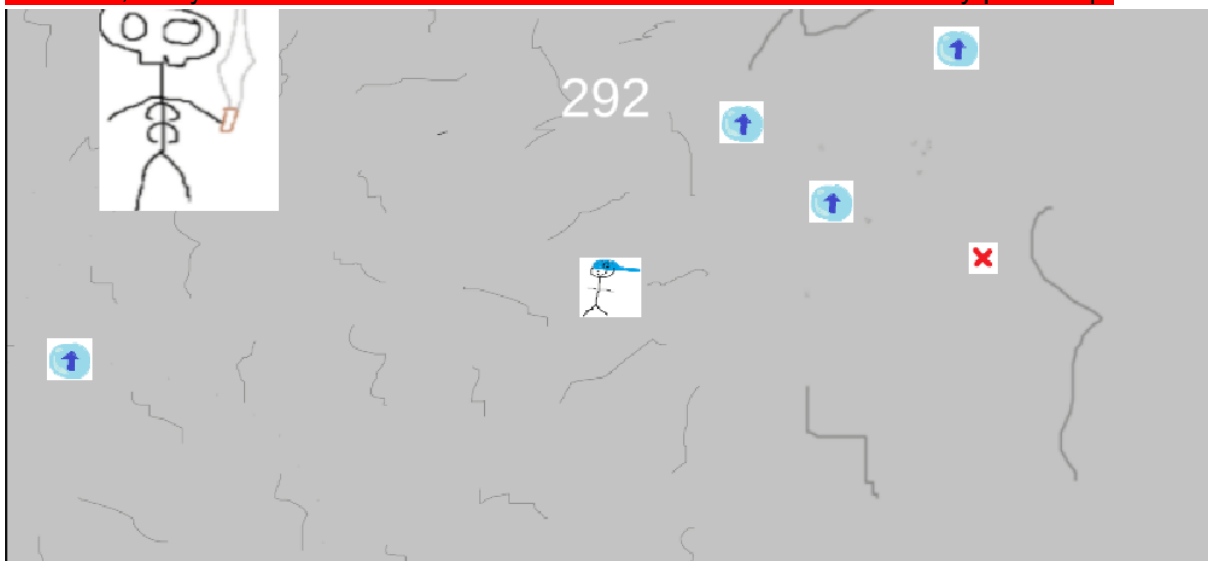
ParentMeleeEnemy melee;
ParentRangedEnemy ranged;
PauseController pauseController;
SpriteRenderer spriteRenderer;
@ Unity Message | 0 references
private void Start()
{
    melee = GetComponent<ParentMeleeEnemy>();
    ranged = GetComponent<ParentRangedEnemy>();
    pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
    spriteRenderer = GetComponent<SpriteRenderer>();
    spriteRenderer.color = new Color(0, 0, 0, 0.4f); //decreases opacity
    Invoke("destroyThis", 0.2f); //destroys the game object .2 seconds after creation
}

@ Unity Message | 0 references
private void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Enemy"))
    {
        if (melee != null)
        {
            melee.damaged(3);
            if (melee.getElectrocuted() == 0) {
                GameObject elecBall = Instantiate(electricity); //creates more electricity if it hits an unelectrocuted enemy
                elecBall.transform.position = transform.position;
            }
            melee.electrocute(); //electrocutes the enemy
        }
        else if (ranged != null)
        {
            ranged.damaged(3);
            if (ranged.getElectrocuted() == 0)
            {
                GameObject elecBall = Instantiate(electricity);
                elecBall.transform.position = transform.position;
            }
            ranged.electrocute();
        }
    }
}

0 references
private void destroyThis()
{
    Destroy(gameObject);
}

```

However, Unity would freeze whenever I fired a hammer with the electricity power-up.



I thought this may be because it was getting stuck on the while loop in the collision that I originally added for when the game pauses. I hashed it out to see if that changed anything.

```
private void OnCollisionEnter2D(Collision2D collision)
{
    //while (true)
    //{
        if (collision.gameObject.CompareTag("Enemy") && pauseController.unpaused == 1)
        {
            GameObject elecBall = Instantiate(electricity);
            elecBall.transform.position = transform.position;
            //break;
        }
    //}
}
```

This worked and the program wouldn't freeze, but still would crash whenever the electric hammer hit an enemy, giving me this error:

 [18:24:45] ArgumentException: The Object you want to instantiate is null.
UnityEngine.Object.Instantiate[T] (T original) (at <0900e0d4bb644dafbfd59eb7fd222a68>:0)

This is because, despite me specifying in the editor that it should spawn the electricity game object, that doesn't carry over to when I add the PUElectric script through code.



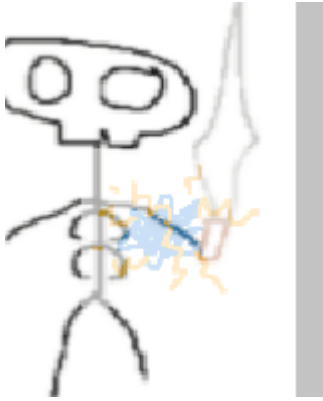
I put the electricity game object in the PowerUpController instead, chosen because it makes sense to keep all power-up related aspects of code in the PowerUpController, and gave it a getter, which the PUElectricity script would use.

This worked.

However, the electricity appeared looking like a black box.



I theorised this was because I set up the colours wrong when I changed its opacity, so I tried fixing that.



It looked better, with the colours being correct, but I also turned up the opacity to make it more visible.

Another issue is that it wasn't ever freezing the enemy. To try and diagnose the issue, I added print statements throughout my code, and it showed that no collision was being detected between the enemy and the electricity.

I realised that the electricity was getting its own enemy scripts, of which there are none, not the collision's enemy scripts, so I changed that by making melee and ranged = collision.gameObject.

```
if (collision.gameObject.CompareTag("Enemy"))
{
    melee = collision.gameObject.GetComponent<ParentMeleeEnemy>();
    ranged = collision.gameObject.GetComponent<ParentRangedEnemy>();
    print(" ");
}
```

This worked, stunning the enemy.

Stage3Vid22

Lastly, I wanted to test how well it chains between enemies. It didn't seem to work.

Stage3Vid23

However, this may have been because I was creating electricity at the location of the initial electricity ball, not the location of the enemy, making it appear like nothing happened. To fix this, I changed it to this:

```
GameObject elecBall = Instantiate(electricity);
elecBall.transform.position = collision.transform.position;
```

This worked, meaning I completed the final power-up for this stage.

Stage3Vid24

Test	Expected Result	Actual Result
Player collides with PowerUpGem object	Sends the player to the Power-Up Selection screen	Sends the player to the Power-Up Selection screen Stage3Vid8
Player clicks a button on the Power-Up Selection Screen	The value relating to the power-up is added to an array in PlayerStats	The value relating to the power-up is added to an array in PlayerStats Stage3Vid11
0 enters the PlayerStats power-up array	The damage attribute in PlayerStats increases	The damage attribute in PlayerStats increases Stage3Vid11

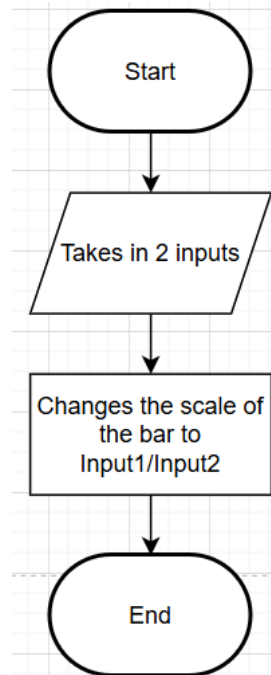
1 enters the PlayerStats power-up array	The maxHealth attribute increases	The maxHealth attribute increases Stage3Vid12
2 enters the PlayerStats power-up array	Increase the speed attribute	Increase the speed attribute Stage3Vid12
3 enters the PlayerStats power-up array	Increase the health attribute every second	Increase the health attribute every second Stage3Vid14
4 enters the PlayerStats power-up array	Whilst decrease enemy health attribute if you collide with them while dashing	Whilst decrease enemy health attribute if you collide with them while dashing Stage3Vid18

Stage 4 – Visual Improvements, and UI Upgrades

Data Dictionary

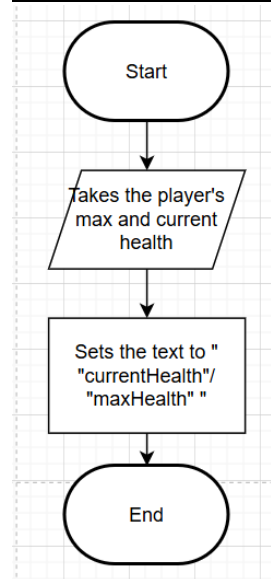
Class	Attribute	Data Type	Method	Description
Health-Bar	maxHealth	Integer		The maximum health of the player
	currentHealth	Integer		Spawns power-ups periodically
			changeSize()	Calculates the new size for the health bar, and sets its size to that
			setText()	Sets the text display to correctly display the player's current and max health
Exp-Bar	currentEXP	Float		The player's current exp
	nextLevel	Float		The next exp boundary the player needs to level up
			changeSize()	Calculates the new size for the exp bar, and sets its size to that
SceneChanger			deathScreen()	Sets the game to the death screen
			startGameplay()	Sets the game to the gameplay scene

changeSize() – Flowchart:



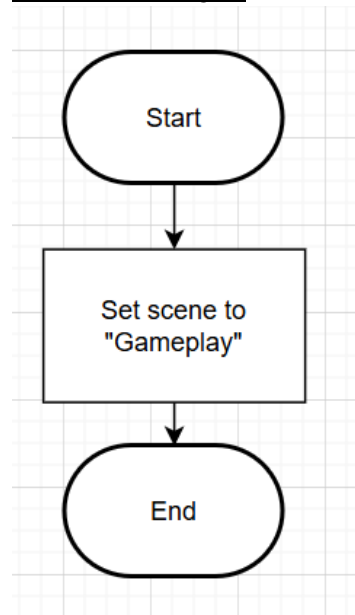
I designed `changeSize()` to be reusable, only taking in 2 inputs, and altering the scale of the object based on those. This is because `changeSize()` will be used in 2 different locations, for both the health and exp bar. Keeping it as non-specific as possible means I can reuse the code later on, saving me time. I chose a flowchart to represent this method due to it being a very linear process that is most simply represented this way.

setText() – Flowchart:



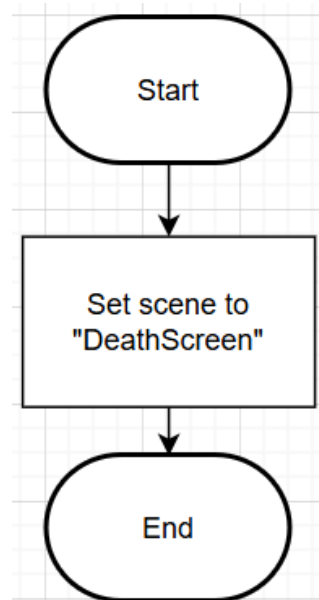
`setText()` will take in the 2 inputs relating to the player's health, and change the text display of the health bar to represent these values. I chose a flowchart to show that this is a very linear algorithm which flows very simply, only taking in 2 values and completing one process with them.

startGameplay():



`startGameplay()` is a very simple method with only one process involved – it just sets the scene to the one called “Gameplay”. A flowchart is a good way to show this simplicity. Due to the algorithm being very simple, I can also reuse some code from this for other scene-changing methods, which will save me time developing and testing in the future.

deathScreen():



`deathScreen()` is similar to `startGameplay()`, just having one function – changing the scene. This is so that, whenever I need to trigger the death screen, I just need to call this method. If there was other code, it would make it harder to use, and I may have to create different methods depending on the situation when, ideally, I would just have one to make the file size smaller, and the game less confusing to work on.

Test Table:

Test	Expected Result	Reason
Player's health damage decreases	The health bar scale decreases proportionally to the lost health	This needs to work to make sure the health bar is accurate and not misleading
Player's health damage increases	The health bar scale increases proportionally to the gained health	This needs to work to make sure the health bar is accurate and not misleading
Player's maxHealth increases	The health bar scale changes proportionally to the maxHealth gained so it is still accurate	This needs to work to make sure the health bar is accurate and not misleading
Player gains exp	The exp bar scale increases proportionally to the gained exp	This needs to work to make sure the exp bar is accurate and not misleading
Player levels up	The health bar scale changes proportionally to the new exp value	This needs to work to make sure the exp bar is accurate and not misleading
Player's health goes below 0	The scene is changed to the Death Screen	This ensures the death screen correctly appears when it should
Player clicks the 'RETRY' button on the Death Screen	The scene is changed to the Gameplay scene	This ensures the player can retry once they lose without any errors occurring
Player clicks the 'START' button	The scene is changed to the Gameplay scene	This ensures the player can begin the game

Stage 4 Development:

I started this stage by designing and coding the health bar. I calculated the coordinates for each end of the bar and called it minPos and maxPos. I also made attributes for the player's maxHealth and currentHealth which would be used to calculate the size of the bar. I added setters for these so they can be updated when the player changes health. I also made the changeSize() method which calculates the percentage of max health the player currently has, and shrinks the health bar's size, making it seem like it is disappearing.

```
public class HealthBar : MonoBehaviour
{
    float minPos = -104.5266f;
    float maxPos = -6.399989f;

    int maxHealth;
    int currentHealth;

    0 references
    public void setMax(int max)
    {
        maxHealth = max;
        changeSize();
    }

    0 references
    public void setHealth(int health)
    {
        currentHealth = health;
        changeSize();
    }

    2 references
    private void changeSize()
    {
        transform.localScale = new Vector3(currentHealth / maxHealth, 1, 1); //sets the size
        Vector2 newPos = transform.position;
        newPos.x = minPos + (maxPos - minPos) * (currentHealth / maxHealth); //sets the new location so its left edge is in the right location
        transform.position = newPos;
    }
}
```

I added setHealth() and setMax() in playerStats at all points that the player's health would be changed.

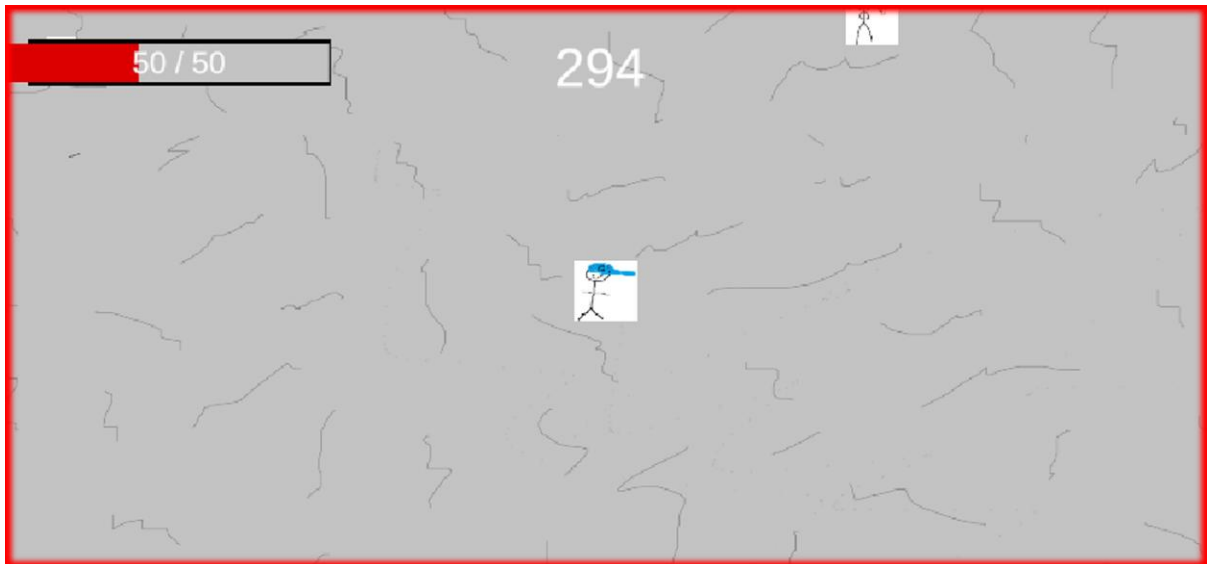
When I ran it, I got this error:

 [17:39:59] DivideByZeroException: Attempted to divide by zero.
HealthBar.changeSize () (at Assets/Scripts/HealthBar.cs:26)

This is most likely because, when setHealth() is originally called when the program first loads, it calls changeSize() whilst getMax() is equal to 0, meaning the program divides by 0. To fix this, I just make sure neither method calls changeSize() whilst the other is 0, like this:

```
if (maxHealth != 0)
{
    changeSize();
}
```

It ran now, but the health bar looked like this.



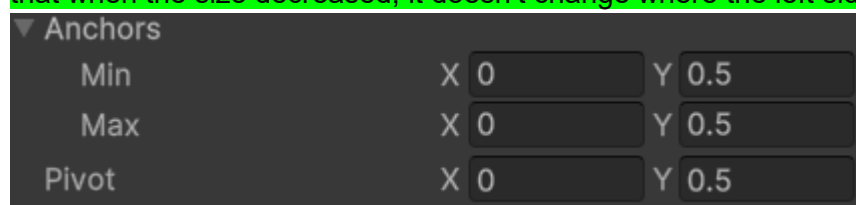
This is because it uses minPos as a base, meaning when you're at max health the position is equal to minPos, whereas it should be maxPos. To change this I altered the code to this:

```
newPos.x = maxPos - ((minPos - maxPos) * (currentHealth / maxHealth));
```

This also didn't fully work, though.



At this point, I realised that the maths behind this would be too complex for me to figure out, so I looked for an alternative solution. I checked the transform component to see if I could move what counts as the centre of the object, and whilst testing things out I found the solution – altering the anchors and pivot to be on the left side of the object. This would mean that when the size decreased, it doesn't change where the left side of the object is located.



This worked, until I took damage, at which point the red disappeared.

Stage4Vid1

To test why I added this to setHealth:

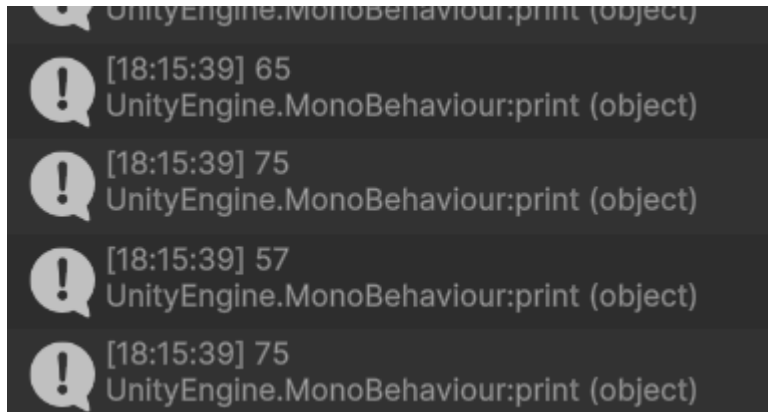
```
print(currentHealth);
```

It printed the correct health.

I moved the code to changeSize(), and added print(maxHealth), too.

```
print(currentHealth);  
print(maxHealth);
```

This also printed the correct values, meaning there wasn't anything wrong with the sequence that the code executes in.



I realised that, due to currentHealth and maxHealth both being integers, it can't compute the fractions correctly, so I changed them to floats to see if that helped.

This worked, however, would go into the negatives when you died, causing the health bar to act weirdly.

Stage4Vid2

To stop it going into the negatives, I simply made it so it can't be less than 0.

```
if (currentHealth < 0) {  
    transform.localScale = new Vector3(0f, 1f, 1f); //sets the size  
}
```

I then worked on getting the text to work correctly, basically taking the code from the regular health bar, and altering it slightly. Instead of changing size, I made it so the text would change to display 'health' / 'maxHealth'. This would tell the player exactly what health they are at, giving them information which lets them make decisions on how risky to play.

```

public class HealthBarText : MonoBehaviour
{
    [SerializeField]
    TextMeshProUGUI m_Object;

    int maxHealth;
    int currentHealth;

    0 references
    public void setMax(int max)
    {
        maxHealth = max;
        if (currentHealth != 0)
        {
            setText();
        }
    }

    0 references
    public void setHealth(int health)
    {
        currentHealth = health;
        if (maxHealth != 0)
        {
            setText();
        }
    }
}

```

```

private void setText() {
    string string1 = currentHealth.ToString();
    string string2 = maxHealth.ToString();
    m_Object.text = string1 + "/" + string2;
}

```

I then put setMax and setHealth in the HealthBar code. They would get called whenever the same method but for HealthBar is called. This means both are changed at the same time, so there is no delay between the values that either is displaying.

```

public void setMax(int max)
{
    maxHealth = max;
    healthBarText.setMax(max);
    if (currentHealth != 0)
    {
        changeSize();
    }
}

```

```

public void setHealth(int health)
{
    currentHealth = health;
    healthBarText.setHealth(health);
    if (maxHealth != 0)
    {
        changeSize();
    }
}

```

This worked.

Stage4Vid3

I then created code for the EXP Bar by reusing and altering the health bar code. I can do this because they are functionally the same, just using different variables.

```
float nextLevelBoundary;
float currentEXP;

0 references
public void setBoundary(int boundary)
{
    nextLevelBoundary = boundary;
    if (currentEXP != 0)
    {
        changeSize();
    }
}

0 references
public void setHealth(int exp)
{
    currentEXP = exp;
    if (nextLevelBoundary != 0)
    {
        changeSize();
    }
}

2 references
private void changeSize()
{
    transform.localScale = new Vector3(currentEXP / nextLevelBoundary, 1f, 1f); //sets the size
    if (currentEXP > 1)
    {
        transform.localScale = new Vector3(1f, 1f, 1f); //sets the size
    }
}
```

I then added setEXP() and setBoundary() to whenever their respective values get changed. The EXP bar worked perfectly, however whilst testing, I found that the passive healing power-up would heal beyond your max health, and whilst the game is paused.

Stage4Vid4

I changed the code to this:

```
if (pauseController.unpaused == 1)
{
    healTimer -= Time.deltaTime;
    if (healTimer <= 0)
    {
        health += 2 * passiveHealing; //heals every 4 seconds
        healTimer = 3;
    }
    if (health > maxHealth)
    {
        health = maxHealth; //ensures health never ursurps maxHealth
    }
    healthBar.setHealth(health);
}
```

This solved the issue.

Stage4Vid5

I then started coding the Start Screen, which will be properly designed in Stage 6.

GILBERT THE GODSLAYER

PLAY

I created the SceneChanger, which utilises a Singleton Pattern to ensure it is the only one to exist and made the button trigger startGameplay(), the method which puts you into the Gameplay scene. The button is what the player is most likely to click, which is why I gave it this purpose.

```
public class SceneChanger : MonoBehaviour
{
    public static SceneChanger instance;
    // Unity Message | 0 references
    void Awake()
    {
        if (instance == null) //checks if any instance has existed before
        {
            instance = this; //if not, this becomes the only instance
            DontDestroyOnLoad(this.gameObject); //makes sure this isn't destroyed when a new scene is loaded
        }
        else
        {
            Destroy(gameObject); //destroys this is an instance already exists
        }
    }

    // Update is called once per frame
    // 0 references
    public void startGameplay()
    {
        SceneManager.LoadScene("Gameplay");
    }
}
```

This worked perfectly.

Stage4Vid6

I then designed the Death Screen, and created the deathScreen() method for the sceneChanger script, which simply changes the scene to the Death Screen.

YOU DIED

get better

RETRY

```
public void deathScreen() {
    SceneManager.LoadScene("DeathScreen");
}
```

This will activate right after the player gets destroyed, which happens when they reach 0 health.

```
Destroy(this.gameObject); //destroys player
sceneChanger.deathScreen();
```

However, when I died in game, I got this error.

[15:12:12] Scene 'DeathScreen' couldn't be loaded because it has not been added to the active build profile or shared scene list or the AssetBundle has not been loaded. To add a scene to the active build profile or shared scene list use the menu File->Build Profiles

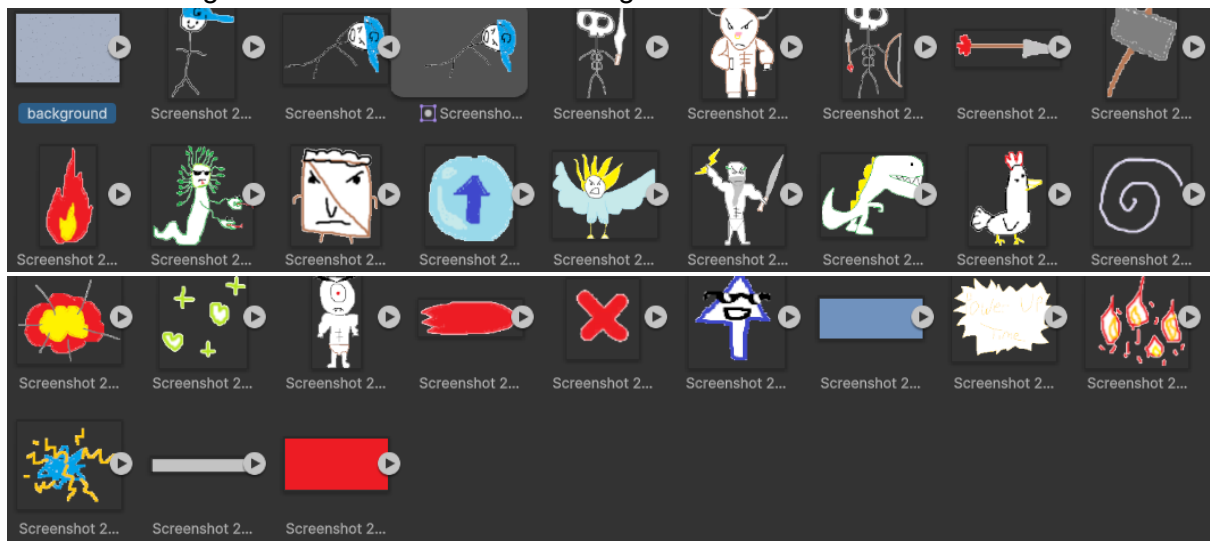
Inspecting this error message more, I found this:

Scene 'DeathScreen' couldn't be loaded because it has not been added to the active build profile or shared scene list or the AssetBundle has not been loaded. To add a scene to the active build profile or shared scene list use the menu File->Build Profiles

I followed the instructions – going to File, Build Profiles and clicking Add Build Profile - and it fixed it.

Stage4Vid7

Then I worked on creating more sprites and updating old ones to make the game look better. Originally, I wanted to go for a pixel-art art style, however, at this point, I don't think I have enough time to create quality pixel art sprites for everything, as I would rather prioritise gameplay features. Instead, I used a similar art-style as to what I have been using so far, but this time making sure to remove the white background.



I also wanted to work on some other visual improvements.

I added this code to flip the player depending on what direction they're moving. This just makes the game feel more immersive. To ensure the scale was correct, I had to check the regular base scale in the transform component and copy it into the vector,

```
if (h > 0f)
{
    transform.localScale = new Vector3(0.7688256f, 0.5f, 1);
}
else if (h < 0f) {
    transform.localScale = new Vector3(-0.7688256f, 0.5f,1); //flips the sprite if you are travelling left
}
```

This worked.

Stage4Vid8

I also wanted to make Hammers spin when you throw them, however I didn't know how to rotate objects, so I looked up the Unity documentation to find out how.

(Unity, n.d.)

Using this, I was able to implement this code in the Update() method for the Hammer. I put it in Update() so that the rotation would change every frame, making it look like a smooth animation.

```
transform.Rotate(0, 360 * Time.deltaTime * pauseController.unpaused, 0); //rotates the hammer by 360 every second
```

This code was functional, but just not how I intended, rotating it around the y-axis.

Stage4Vid9

I changed the code so that it rotated around the x-axis instead, which looked better. I also made the enemy projectile get oriented in the correct direction as it travels, making it more intuitive to dodge.

Stage4Vid10

```
private void directionCalc()
{ //method for calculating the direction
    directionVector = (player.transform.position - enemy.transform.position).normalized; //calculates the direction vector
    direction = Mathf.Atan(directionVector.y / directionVector.x); //calculates the angle for the direction the projectile should travel in
    this.transform.Rotate(0,0,180*direction/Mathf.PI); //rotates the angle
}
```

I changed the enemy movement code to include it rotating depending on what direction it is travelling in, once again making the game feel more immersive.

```
if (directionVector.x > 0)
{
    pos.x += speed * Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused * frozen;
    pos.y += speed * Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused * frozen; //calculates new location
    transform.localScale = new Vector3( scale.x, scale.y, scale.z );
}
else
{
    pos.x += speed * -Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused * frozen;
    pos.y += speed * -Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused * frozen;
    transform.localScale = new Vector3(-1 * scale.x, scale.y, scale.z);
}
```

Stage4Vid11

I then created a new effect which would appear whenever you heal. To get it to spawn, all I had to do was put this in playerStats whenever your health goes up.

```
GameObject heal = Instantiate(healEffect,transform); //instantiates a heal effect
```

When I tested this, however, it was a bit off centre, and the effect would come up even when you don't have the passive healing power-up.

I changed the heal effect instantiation so that it isn't part of the Player character, which made it better, and also put all passive healing code in an IF statement so that it doesn't trigger if you don't have any passive healing, which will save resources, and ensure the effect only spawns when passive healing is active.

Lastly, I also created a damage effect for whenever anything takes damage. This will be useful for telling the player that they are making progress and damaging the enemies.

I put this code in the damaged() methods for both the player and the enemies.

```
GameObject damageSprite = Instantiate(damageEffect);
damageSprite.transform.position = transform.position;
```

This worked for the player, however not for the enemies, instead crashing the game.

Stage4Vid13

However, this is simply because I forgot to assign them with the damageEffect game object. I changed this and it worked.

Stage4Vid14

Test Table:

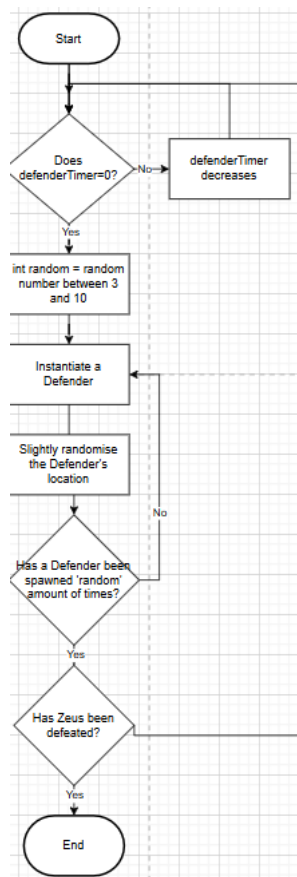
Test	Expected Result	Actual Result
Player's health damage decreases	The health bar scale decreases proportionally to the lost health	Expected result Stage4Vid3
Player's health damage increases	The health bar scale increases proportionally to the gained health	Expected result Stage4Vid5
Player's maxHealth increases	The health bar scale changes proportionally to the maxHealth gained so it is still accurate	Expected result Stage4Vid10
Player gains exp	The exp bar scale increases proportionally to the gained exp	Expected result Stage4Vid4
Player levels up	The health bar scale changes proportionally to the new exp value	Expected result Stage4Vid4
Player's health goes below 0	The scene is changed to the Death Screen	Expected result Stage4Vid7
Player clicks the 'RETRY' button on the Death Screen	The scene is changed to the Gameplay scene	Expected result Stage4Vid7
Player clicks the 'START' button	The scene is changed to the Gameplay scene	Expected result Stage4Vid6

Stage 5 – More Enemies and Power-Ups

Data Dictionary:

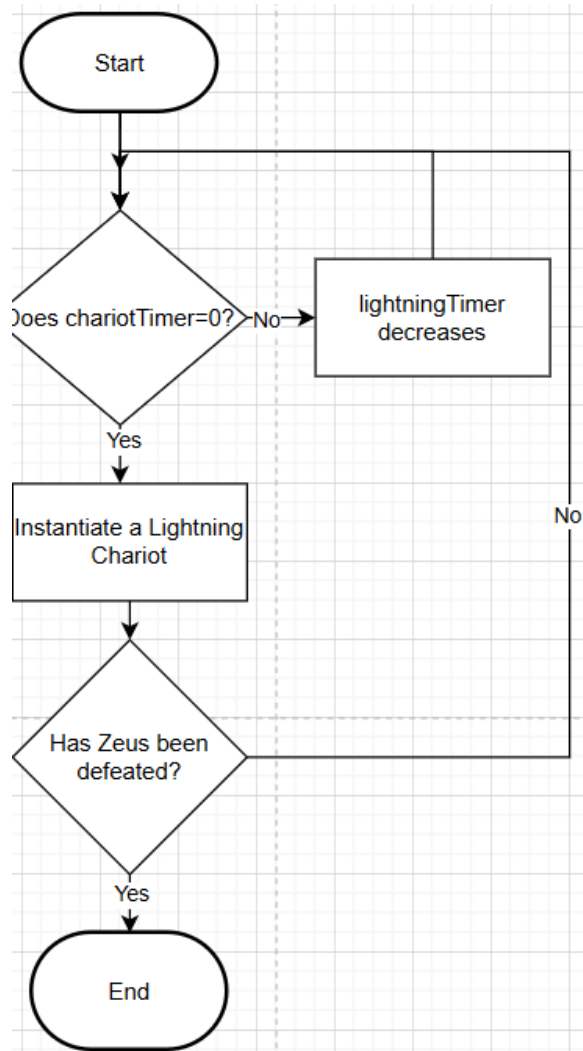
Class	Attribute	Data Type	Method	Description
Boss Enemy	lightningTimer	Float		Timer for the lightning ability
	laserTimer	Float		Timer for the lightning ability
	chariotTimer	Float		Timer for the lightning ability
	defenderTimer	Float		Timer for the lightning ability
			spawnDefenders()	Spawns a random number of Defenders
			spawnChariot()	Spawns a Chariot
			fireLasers()	Fires 8 laser projectiles at the player
			throwLightning()	Fires a lightning projectile at the player
Explosion			Collision()	Damages the enemies it collides with
Ally	Health	Integer		Decreases when damaged, destroys object when =0
	Speed	Float		The speed the ally moves at
	Damage	Integer		Damage dealt to enemy if they collide
			Movement()	Moves the ally towards the closest enemy
			Attack()	Damages the enemies it collides with

spawnDefenders():



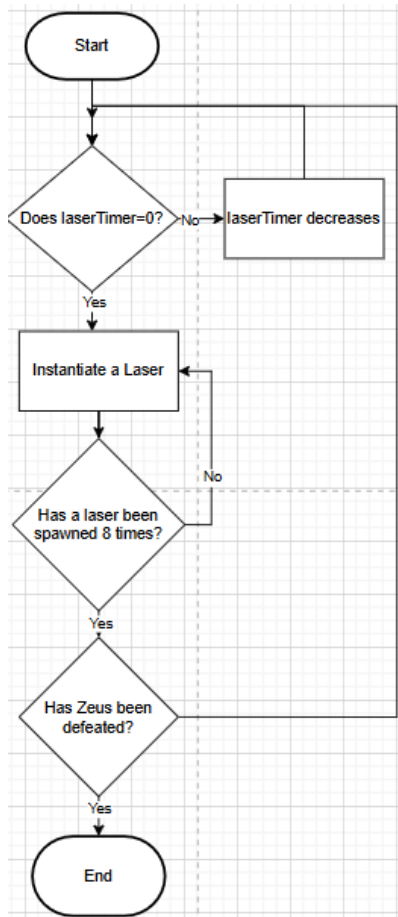
spawnDefenders() is a more complicated algorithm, so I chose a flowchart to represent it to really break it down into steps, making it far easier to code. The algorithm will first check if the defenderTimer attribute = 0, and if not it will decrease the timer and loop back to the start. Once the timer finally does = 0, it will then generate a random number and instantiate a Defender that amount of times. Finally, it will check if Zeus has been defeated and if not, loop back to the very start. It has to loop back to before the timer check because otherwise it'd constantly spawn Defenders, which may overwhelm the player and make the game too difficult.

spawnChariot():



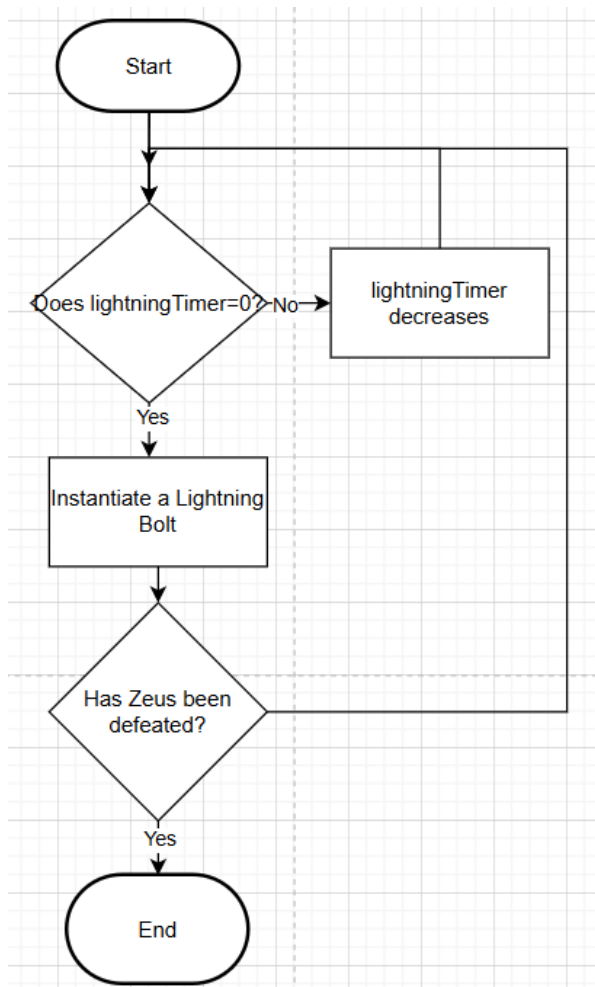
The algorithm for `spawnChariot()` is similar to `spawnDefenders()`, except it spawns one enemy this time. This will allow me to reuse and simplify the code from `spawnDefenders()`, which will speed up development.

fireLasers():



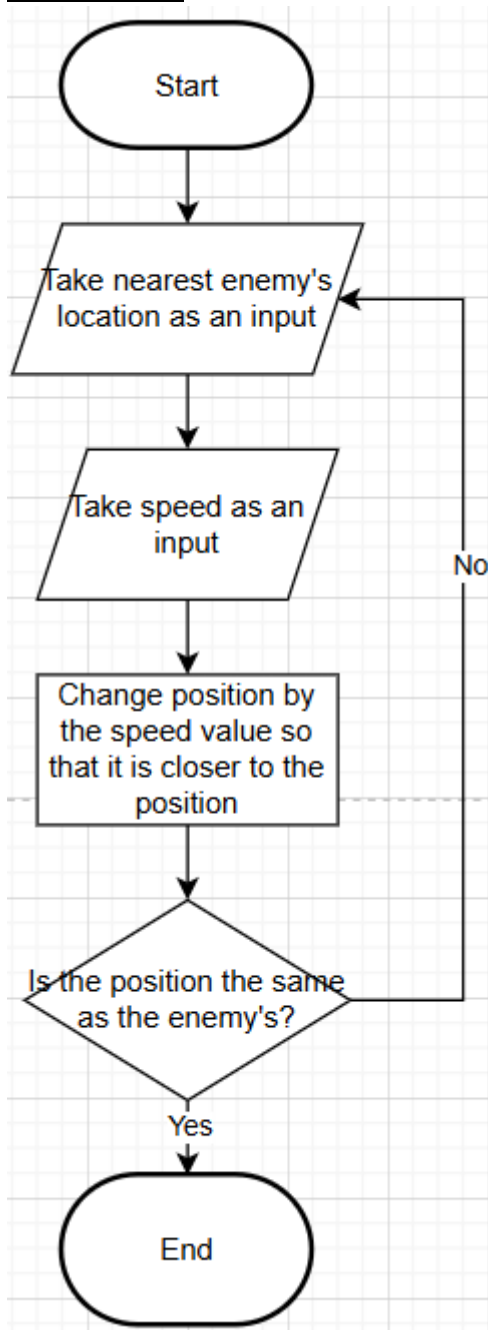
`fireLasers()` is another method that uses a lot of loops, so I used a flowchart to represent this in an easy-to-understand form. It will loop until Zeus dies, and will be stuck in another loop until `laserTimer = 0`. Once the timer does equal 0, it will then instantiate lasers, looping until 8 has spawned.

throwLightning():



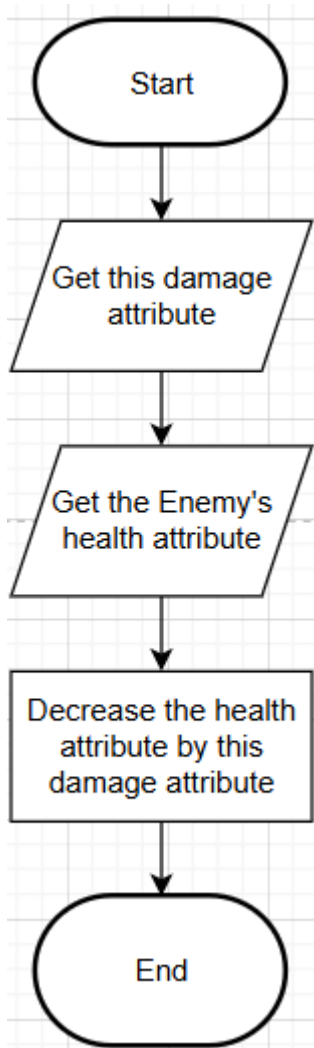
lightningTimer() is a simplified version of the fireLasers() algorithm, meaning I can reuse the code from that algorithm, as long as I the part where it instantiates 8 projectiles because the lightning is far more powerful, so firing 8 would make the game too difficult. This will save me a lot of time, allowing me to improve other aspects of the game further.

movement():



The movement() algorithm is always looping to keep the ally constantly moving. I took the enemy location and the speed stats as inputs, because these are important for me to control the location of the ally, and give it the appearance of movement. Using a flowchart to represent this algorithm makes it easier for me to see what needs to be looped and what doesn't, making this simpler to code.

attack():



The attack() algorithm should just be a sequence of steps that gets called every time there's a new collision. It will have to get the ally's attack stat, and the enemies health attribute. After that, it will just damage the enemy by decreasing its health by the damage stat. The flowchart perfectly represents this order.

Test Table:

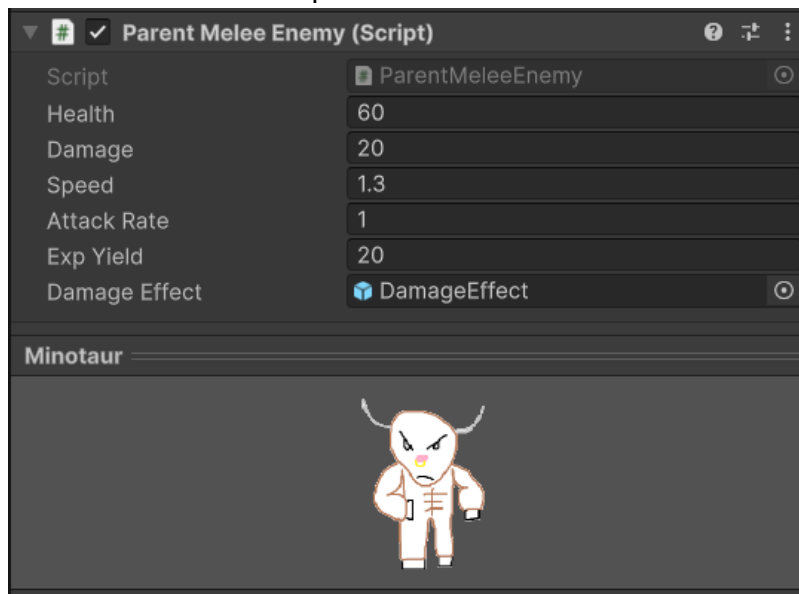
Test	Expected Result	Reason
Zeus' health <=0	Scene changes to Win Screen	This needs to work to convey to the player that they have beat the game.
Ally is created	The ally moves towards an enemy	This ensures the ally movement system is functional
Ally collides with an enemy	The enemy takes damage	This ensures the allies attack enemies, otherwise it is useless

Enemy collides with an ally	Ally's health attribute decreases	This makes sure allies aren't immortal, and end up flooding the map, creating lag
Enemy gets hit by a Hypno hammer	The enemy becomes an ally	This ensures the Hypno power-up is functional
Enemy gets hit by a T-Rex hammer	A T-Rex is instantiated	This ensures the T-Rex power-up is functional
Enemy gets hit by an Explosion hammer	An Explosion game object is instantiated	This ensures the Explosion power-up is functional
Enemy collides with Explosion	The enemy's health attribute decreased	This ensures the Explosion power-up is functional

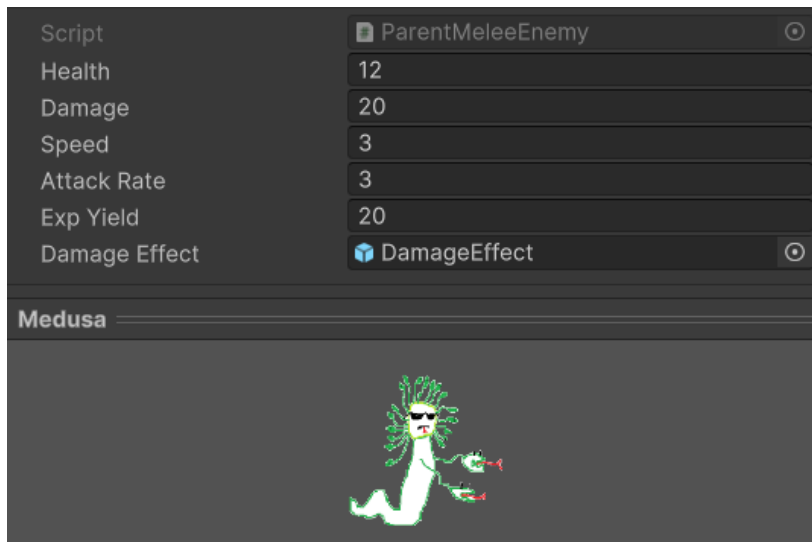
Stage 5 Development

I started this stage by designing new enemies.

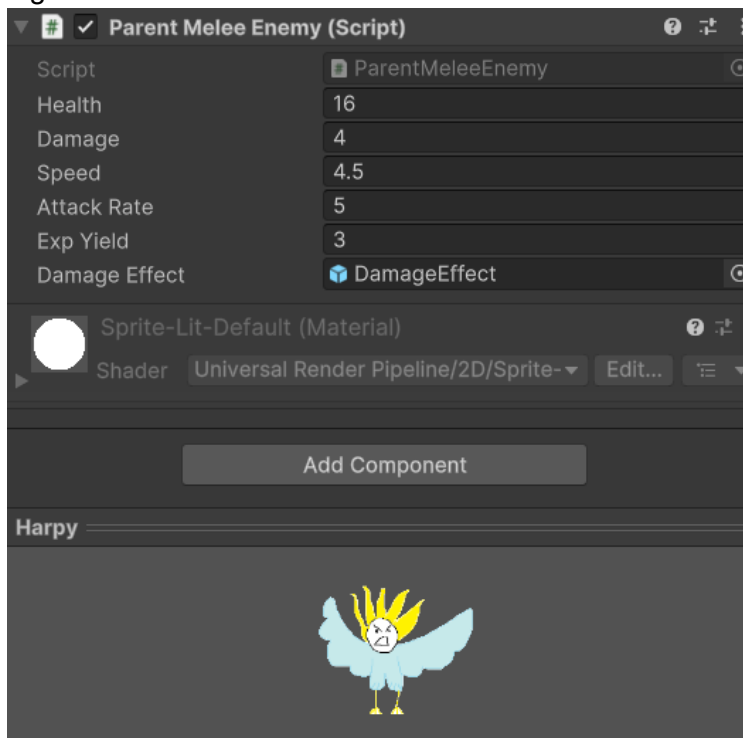
I began with the melee enemies. There wasn't any additional code I needed to make due to my previous efforts with creating the ParentMeleeEnemy script. Instead I just had to choose various stats to create a diverse set of enemies that challenge the player differently. The various enemies are explained below.



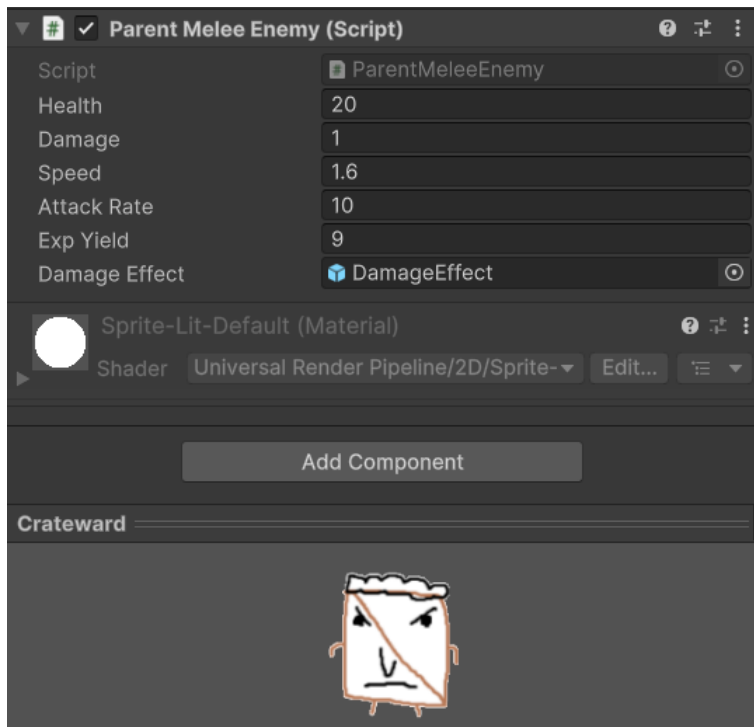
The first one I designed was the minotaur. He was designed to be a slow enemy that does a lot of damage and has a lot of health, forcing the player to run away from him until there is a clear opportunity to focus him down and whittle down his massive health bar.



After that I designed Medusa. She would be very fast and deal a lot of damage, but have low health. This stat spread should make the player panic when she spawns, and treat her like a big threat.

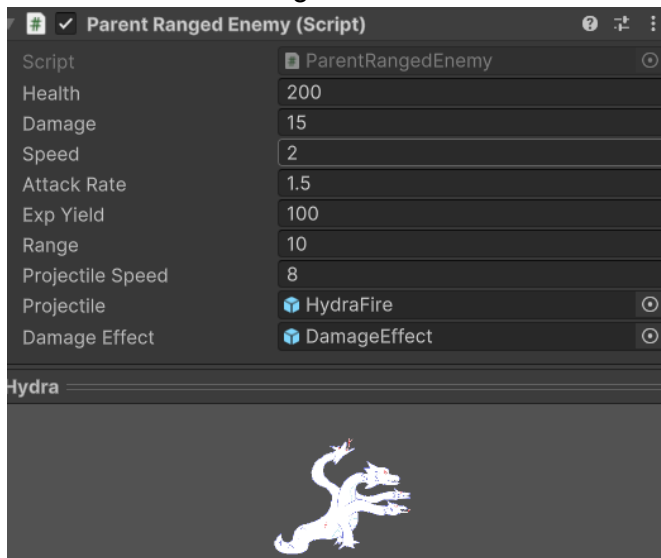


I then made the harpy. She would be very fast, but have relatively low damage. The harpy should mostly act as an annoying distraction to the player.

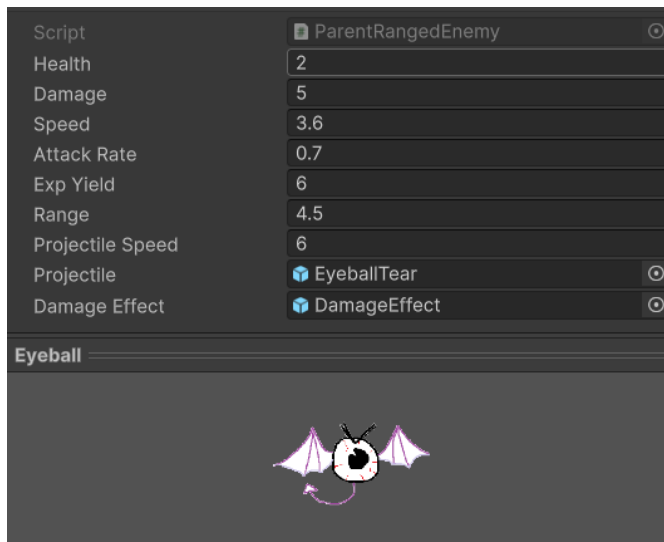


Lastly, I designed Crateward. This is an original design, which is something the stakeholders wanted, and is very weak, but drops a lot of exp, which should help the player level up in the early game.

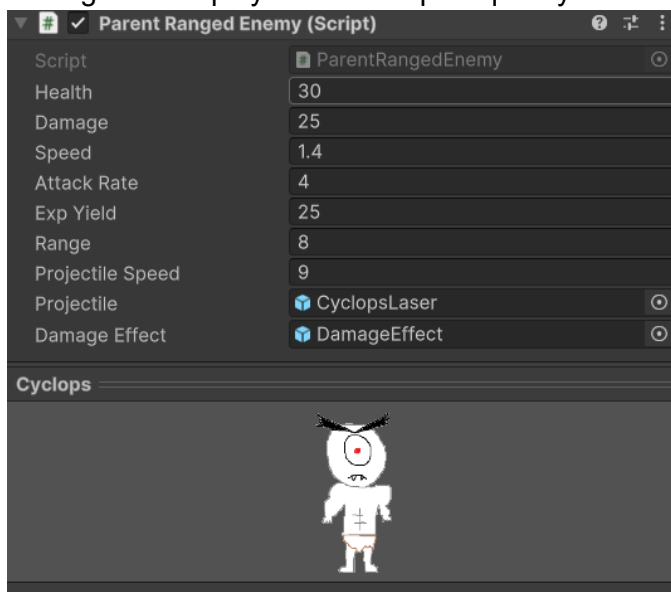
I then made the ranged enemies. Once again, there was no new code due to me designing the basics back in Stage 2.



First, I made the hydra. The hydra acts like a miniboss, doing lots of damage and having a lot of health and range. It should provide for a good challenge before the final boss spawns.



I then made the eyeball – another original design. This has extremely low health, and a low range, but is very fast and attacks very quickly. It should act as a quick threat that gets good damage on the player but dies quite quickly.



Lastly, I made the cyclops, which should do a ton of damage from a very long range, but be very slow. This means the player will have to try to get close to it whilst avoiding massively powerful attacks.

All in all, these enemies should force the player to play around them differently, creating a very diverse gameplay experience which I think will be quite enjoyable.

Whilst making these enemies, I noticed that the projectile the ranged enemies fire is sometimes oriented the wrong way, which I fixed with this line of code:

```
if (player.transform.position.x < transform.position.x) {
    transform.localScale = new Vector3(-1,1,1);
}
```

Stage5Vid1

I also noticed I forgot to add the code that rotates them depending on what way they're moving. I made sure to fix this.

```

if (directionVector.x > 0)
{
    pos.x += speed * Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused * frozen;
    pos.y += speed * Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused * frozen; //calculates new location
    transform.localScale = new Vector3(scale.x, scale.y, scale.z);
}
else
{
    pos.x += speed * -Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused * frozen;
    pos.y += speed * -Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused * frozen;
    transform.localScale = new Vector3(-scale.x, scale.y, scale.z); //flips it into the right direction
}

```

The enemies flipped correctly, but I noticed the scale would be wrong on projectiles that got flipped.

Stage5Vid2

I changed the flipping code to this, meaning no matter what the scale of the various projectiles are, they will just be reflected rather than changed to a new negative value, which would make it appear like the size changed as well as the rotation.

```

if (player.transform.position.x < transform.position.x) {
    Vector3 scale = transform.localScale;
    transform.localScale = new Vector3(-scale.x, scale.y, scale.z);
}

```

This fixed it, as seen in the following video.

Stage5Vid3

I then went to make spawners for everything. I noticed that in the spawner code it said that the time between spawns was '5/spawnRate', which I initially did to make it so that a higher spawn rate spawns more, which felt intuitive at the time. However, that makes calculations very inconvenient because I would need to input a very small number to make an infrequent spawn rate. I changed it to this:

```

yield return new WaitForSeconds(spawnRate); //makes it wait for the spawn rate before
if (timerTimer.getTime() < 300 - timerUntilSpawn 66 pauseController.unpaused=1)

```

I then started work on the Boss of my game – Zeus.

He would use the regular melee enemy script as a base, but also have an exclusive script. This is to differentiate him from every other enemy in the game, and make his fight feel far more interesting and unique.

I first created some attributes for the ZeusScript. It mostly needed timers for the various attacks he would do. I chose to use timers here instead of invoking because I can dynamically change the value of the timer to make the boss attack faster or slower depending on what's happening.

```

[SerializeField]
GameObject zeusLaser;

[SerializeField]
GameObject lightningBolt;

[SerializeField]
GameObject soldier;

[SerializeField]
GameObject lightningChariot;

Transform player;
PauseController pauseController;
Vector3 directionVector;
float direction;

float lightningTimer=3; //timer until he throws lightning
float laserTimer=8; //timer until he fires lasers
float chariotTimer=12; //timer until he spawns the chariot
float defenderTimer=3; ///timer until he spawns defenders

```

I then made the methods for the attacks that feature spawning. I want to use these because they should make the boss fight more interesting, as it will split the player's attention between the boss and the various enemies he spawns. I simply made these methods instantiate the enemy and set their position. For the soldiers, I made their position have a slight random variation of where they spawn, because otherwise all soldiers will be stacked on top of each other at the same location, which will make them too easy to fight, and won't convey to the player that there are multiple enemies.

```
private void spawnDefender() {
    GameObject spawn = Instantiate(soldier);
    soldier.transform.position = new Vector3(transform.position.x + Random.Range(-0.5f, 0.5f), transform.position.y + Random.Range(-0.5f, 0.5f), transform.position.z);
}

1 reference
private void spawnChariot() {
    GameObject chariot = Instantiate(lightningChariot);
    chariot.transform.position = transform.position;
}
```

I then made the attacks that spawn projectiles. These act very similarly to how regular ranged enemies fire projectiles, so I was able to reuse the code from there, which I did to save time.

```
private void throwLightning() {
    GameObject lightning = Instantiate(lightningBolt);
    EnemyProjectile projectileStats = lightning.GetComponent<EnemyProjectile>();
    Vector2 projectilePos = transform.position;
    lightning.transform.position = projectilePos;
    projectileStats.setEnemy(this.gameObject); //sets this as the projectile's enemy
    lightning = null; //resets Projectile and projectileStats
    projectileStats = null;
}

0 references
private void fireLasers() {
    GameObject laser = Instantiate(zeusLaser);
    EnemyProjectile projectileStats = laser.GetComponent<EnemyProjectile>();
    Vector2 projectilePos = transform.position;
    laser.transform.position = projectilePos;
    projectileStats.setEnemy(this.gameObject); //sets this as the projectile's enemy
    laser = null; //resets Projectile and projectileStats
    projectileStats = null;
}
```

I put these attacks on timers in the update method. To make the laser attack fire multiple lasers, I set up multiple invoke statements to fire a laser every 0.02 seconds. This means I don't need to create even more timers, which is handy because I am already utilising 4 separate ones, so creating 8 more will get very messy and confusing. I also used a FOR loop when spawning soldiers to spawn a random amount of soldiers – between 3 and 10. I used a for loop instead of a while loop because I will only need it to loop a set number of times, which is far easier to do using a FOR loop. I wanted to make it spawn a random quantity of soldiers to make the boss fight feel more varied, which is important seeing as this is the only boss I intend to implement.

```

void Update()
{
    lightningTimer -= Time.deltaTime*pauseController.unpaused;
    laserTimer -= Time.deltaTime * pauseController.unpaused;
    chariotTimer -= Time.deltaTime * pauseController.unpaused;
    defenderTimer -= Time.deltaTime * pauseController.unpaused;

    if (lightningTimer <= 0) {
        throwLightning();
        lightningTimer = 4;
    }
    if (laserTimer <= 0 && (player.position - transform.position).magnitude < 15) //checks if you are in range
    {
        Invoke("fireLasers", 0.1f);
        Invoke("fireLasers", 0.12f);
        Invoke("fireLasers", 0.14f);
        Invoke("fireLasers", 0.16f);
        Invoke("fireLasers", 0.18f);
        Invoke("fireLasers", 0.2f);
        Invoke("fireLasers", 0.22f);
        Invoke("fireLasers", 0.24f);
        laserTimer = 8.5f;
    }
    if (defenderTimer <= 0) {
        for (int i = 0; i < Random.Range(3, 10); i++) { //repeats a random number of times
            Invoke("spawnDefender", i/3);
        }
        defenderTimer = 12;
    }
    if (chariotTimer <= 0) {
        spawnChariot();
        chariotTimer = 15;
    }
}

```

Because Zeus doesn't have a parentRangedEnemy() component, I need to edit the enemyProjectile component to work with the zeusScript component.

I created a variable in enemyProjectile() to tell the zeusScript if it is dealing with a laser or lightning bolt – it'll equal 0 for a laser and 1 for lightning. If the projectile isn't a laser or lightning, the value will be null.

```

[SerializeField]
int laserOrLightning;

```

I then created attributes for both the laser and lightning's range, projectile speed and damage. I created getters for these, which take in an integer parameter. If this integer is 0, it'll return the laser stats, and if it is 1, it'll return the lightning stats. I did it this way so that I can get the correct stats from calling one method, instead of me having to use IF statements and create multiple methods, which will increase the file size and overcomplicate the code.

```

[SerializeField]
float laserSpeed;
[SerializeField]
float lightningSpeed;

float laserDistance = 14.5f;
float lightningDistance = 100f;

[SerializeField]
int laserDamage;
[SerializeField]
int lightningDamage;

```

```

1 reference
public float getProjectileSpeed(int what) {
    if (what == 0) { return laserSpeed; }
    else return lightningSpeed;
}

1 reference
public int getDamage(int what)
{
    if (what == 0) { return laserDamage; }
    else return lightningDamage;
}

0 references
public float getRange(int what) {
    if(what == 0) {return laserDistance; }
    else return lightningDistance;
}

```

Lastly I reworked setEnemy() in the enemyProjectile script to include a section to check if enemyStats is null. If it is, this means that the enemy that fired this projectile isn't a ranged enemy, the only other option being that it is Zeus. If it is Zeus that fired the projectile, it'll use the getters for each stat and pass in laserOrLightning, because the methods use that value to determine what to return. If it isn't Zeus, the code acts normally. I did it this way because it covers for all options in very little code – the script already tries to get enemyStats, and that can be used to determine if it is Zeus or not, and if it is Zeus, it just needs to use one parameter to find what stats it needs to return.

```

public void setEnemy(GameObject thisEnemy)
{
    pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
    enemy = thisEnemy;
    enemyStats = enemy.GetComponent<ParentRangedEnemy>();
    if (enemyStats == null)
    {
        ZeusScript zeusStats = enemy.GetComponent<ZeusScript>();
        damage = zeusStats.getDamage(laserOrLightning);
        speed = zeusStats.getProjectileSpeed(laserOrLightning);
        range = zeusStats.getRange(laserOrLightning);
    }
    else
    {
        speed = enemyStats.getProjectileSpeed(); //gets the projectile speed from the stats script
        damage = enemyStats.getDamage(); //gets the damage from the stats script
        range = enemyStats.getRange();
    }
    player = GameObject.Find("Player");
    ready = true;
    timer = 1.2f*(range / speed); //uses time=distance/speed to calc how long it should travel for, plus a little extra
    directionCalc();
}

```

When I tested it, everything worked perfectly.

Stage5Vid4

I then worked on getting the boss to spawn.

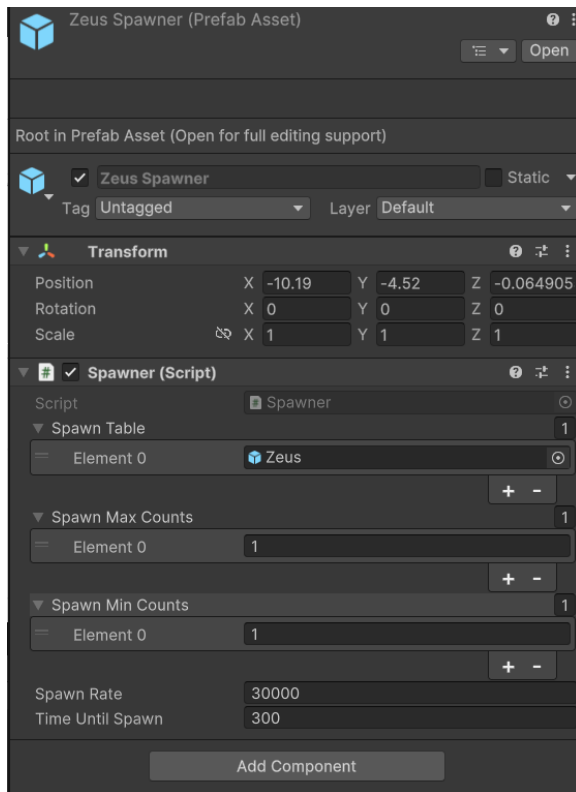
First, for aesthetic purposes, I made the timer delete itself after it has run out. I did this so it isn't counting down and displaying negative numbers after the boss spawns.

```

if (timer <= 0) {
    Destroy(gameObject);
}

```

I then created the Zeus spawner. I made it spawn after 300 seconds, because that's when the timer runs out, and I made the spawn rate extremely high, so there is no way the player will realistically see two.



When testing it, after the timer ran out, the boss didn't spawn. The timer was correctly deleted, though.

Stage5Vid5

This is because the IEnumerator in spawn only runs once every spawn-rate seconds, meaning in this case once every 30000 seconds.

To fix this issue, I changed the Spawner script so that the IEnumerator is called in Update() after the timer is below timeUntilSpawn, meaning that the IEnumerator only starts when it is time for the spawner to start spawning.

```
float spawnTime = 0;

Unity Message | 0 references
private void Start()
{
    pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
    player = GameObject.Find("Player");
    timer = GameObject.Find("Timer");
    timerTimer = timer.GetComponent<Timer>();
}

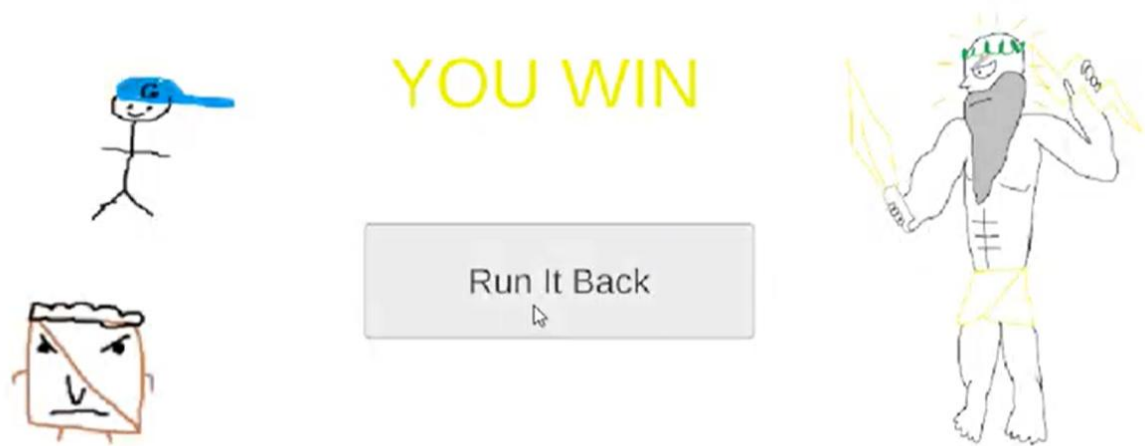
Unity Message | 0 references
private void Update()
{
    if (timerTimer.getTime() < 300 - timeUntilSpawn && started==0) {
        StartCoroutine(Spawning());
        started = 1; //ensures this if statement never runs again
    }
}

1 reference
IEnumerator Spawning() {
    while (player!=null)
    {
        yield return new WaitForSeconds(spawnTime); //makes it wait for the spawn rate before
        if (timerTimer.getTime() < 300-timeUntilSpawn && pauseController.unpaused==1)
        {
            spawnTime = spawnRate;
        }
    }
}
```

This worked (I set the `timeUntilSpawn` to 10 in the following video to make it easier to test).

Stage5Vid6

I then designed the Victory Screen. It is just a redesigned Start screen, in which I coloured the background white and added the images of some enemies to let the player remember the journey they have been on. The white and gold colour scheme is something I planned to add in my GUI design, to give this screen a rich, godly vibe, emphasising your victory over Zeus.



Stage5Vid7

I put this method in `SceneChanger` to change the scene to the Win Screen.

```
public void winScreen() {  
    SceneManager.LoadScene("WinScreen");  
}
```

In `parentMeleeEnemy` I put this for when a melee enemy dies. It checks if the enemy is Zeus by getting the `ZeusScript` component, then invokes the victory screen after a short wait. I added the delay so that the player can see that they defeated the boss before the scene suddenly changes.

```
if (GetComponent<ZeusScript>() != null) //if this is Zeus  
{  
    Invoke("victory", 2.5f); //set to the victory screen  
}
```

```
private void victory() {  
    SceneChanger sceneChanger = GameObject.FindWithTag("SceneChanger").GetComponent<SceneChanger>();  
    sceneChanger.winScreen();  
}
```

However, the scene didn't change once you beat the boss.

Stage5Vid8

This is because the script gets destroyed before it can invoke `victory()`. To fix this, it calls the `winScreen()` method immediately after invoking it, and `winScreen` then invokes a new method after a short delay, which then changes the scene. This fixes the issue of the `ZeusScript` getting destroyed before the `winScreen()` method gets invoked.


```

if (GetComponent<ZeusScript>() != null) //if this is Zeus
{
    SceneChanger sceneChanger = GameObject.FindWithTag("SceneChanger").GetComponent<SceneChanger>();
    sceneChanger.winScreen();
}

```

```

public void winScreen() {
    Invoke("invokedWin", 2.5f);
}

0 references
public void invokedWin() {
    SceneManager.LoadScene("WinScreen");
}

```

This worked.

Stage5Vid9

To add variety in the melee enemies, and provide a way to heal without relying on the passive healing power-up, I wanted Crateward's to drop orbs that heal. I chose Crateward because it is already a weak enemy that provides benefits – lots of exp – when you defeat it. I made a heal() method in playerStats for health recovery and then made a healOrb game object. The heal() method was designed to be very versatile so that I can call this method for any type of healing in future development, saving me from having to constantly repeat similar code. The healOrb game object just detects if it collides with the player, and then calls the heal() method, as well as destroying itself. This is so you can't repeatedly heal of one orb, as that disincentivises movement.

```

public void heal(int healAmount)
{
    health += healAmount;
    if (health > maxHealth)
    {
        health = maxHealth;
    }
    GameObject heal = Instantiate(healEffect); //instantiates a heal effect
    Vector3 healPos = transform.position;
    heal.transform.position = healPos;
}

```

```

public class HealOrb : MonoBehaviour
{
    Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            PlayerStats playerStats = collision.gameObject.GetComponent<PlayerStats>();
            playerStats.heal(1);
            Destroy(gameObject);
        }
    }
}

```

This worked.

Stage5Vid10

I then worked on getting Crateward to spawn the orbs on death adding a new attribute which holds the healOrb game object. If it isn't null when the enemy is defeated, it will instantiate

this game object before it gets destroyed. This means I always have the option to let other melee enemies drop heal orbs in the future, without disrupting any enemies that shouldn't.

```
[SerializeField]
private GameObject healOrb;

if (healOrb != null)
{
    GameObject healOrbs = Instantiate(healOrb);
    healOrbs.transform.position = transform.position;
}
```

This worked.

Stage5Vid11

When testing, though, the screen would get flooded by these healing orbs due to them never despawning, so I wanted to make them get destroyed after 8 seconds. This was as easy as invoking a method that destroys the gameObject after 8 seconds.

```
private void Start()
{
    Invoke("destroyMethod", 8);
}

0 references
private void destroyMethod() {
    Destroy(gameObject);
}
```

This worked.

Stage5Vid12

I then worked on getting the remaining power-ups functional.

For the PUExplosion script, I made the following code, taking some code from PUElectricity due to the similarities between the power-ups. It would get the explosion game object from PowerUpController, and instantiate it when it collides with an enemy. It also colours the hammer black because that tells the player that this hammer is explosive.

```
public class PUExplosion : MonoBehaviour
{
    GameObject explosion;

    PauseController pauseController;
    SpriteRenderer spriteRenderer;
    // Unity Message | 0 references
    void Start()
    {
        explosion = GameObject.FindWithTag("PowerUpController").GetComponent<PowerUpController>().getExplosion();
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.color = Color.black; //colours the hammer black
        pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
    }

    // Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy") && pauseController.unpaused == 1)
        {
            GameObject explode = Instantiate(explosion); //creates the explosion game object
            explode.transform.position = transform.position;
        }
    }
}
```

```

1 reference
public GameObject getExplosion()
{
    return explosion;
}

```

I then created the explosion object. It would simply destroy itself after 0.3 seconds, but if it collides with an enemy in that time, it does a lot of damage to them. I made sure to make the script destroy itself because otherwise you would permanently see explosion sprites laying around the map.

```

public class ExplosionScript : MonoBehaviour
{
    ParentMeleeEnemy melee;
    ParentRangedEnemy ranged;

    // Unity Message | 0 references
    private void Start()
    {
        Invoke("destroyThis", 0.3f);
    }

    // Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy"))
        {
            melee = collision.gameObject.GetComponent<ParentMeleeEnemy>();
            ranged = collision.gameObject.GetComponent<ParentRangedEnemy>();
            if (melee != null)
            {
                melee.damaged(14); //deals 14 damage to a melee enemy
            }
            else if (ranged != null)
            {
                ranged.damaged(14); //deals 14 damage to a ranged enemy
            }
        }
    }

    0 references
    private void destroyThis()
    {
        Destroy(gameObject);
    }
}

```

I also added the following code to playerShoot, which allows hammers to become explosive.

```

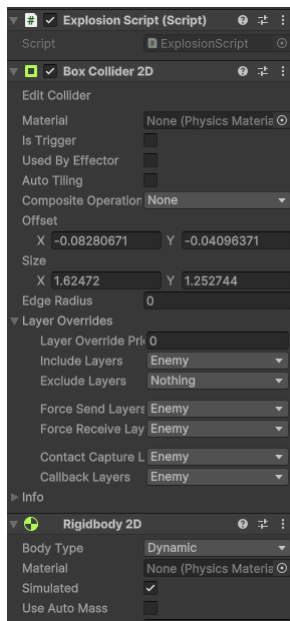
case 2:
    hammer.AddComponent<PUExplosion>();
    break;

```

This code spawned the explosion correctly, but never destroyed the explosion, and the explosion didn't deal any damage.

Stage5Vid13

This, however, was just due to me not adding the explosion component to it, nor a rigid body and collider. I changed that and it worked.



Stage5Vid14

The remaining 2 power-ups rely on creating allies, so I had to code the ally script. I first had to add getters to both enemy scripts for all attributes that the ally would use. This is to ensure the ally's stats are consistent with the enemy they are based on.

```

public float getSpeed() {
    return speed;
}

0 references
public int getDamage() {
    return damage;
}

0 references
public float getRate()
{
    return attackRate;
}

0 references
public int getHealth() {
    return health;
}

```

I started off by making these attributes and assigning them in the Start() method. I chose these because these are all the attributes a melee enemy uses, and I want the ally to work just like a melee enemy.

```

public class AllyScript : MonoBehaviour
{
    GameObject target;

    [SerializeField]
    private int health; //how much damage the ally can take
    [SerializeField]
    private int damage; //how much ally the enemy deals to the player
    [SerializeField]
    private float speed; //how fast the ally will move
    [SerializeField]
    private float attackRate; //how fast the ally will attack

    ParentMeleeEnemy melee;
    ParentRangedEnemy ranged;
    PauseController pauseController;

    private Vector3 directionVector;
    private float direction;
    private Vector3 scale;
    Ⓜ Unity Message | 0 references
    void Start()
    {
        target = GameObject.FindWithTag("Enemy");
        melee = gameObject.GetComponent<ParentMeleeEnemy>();
        ranged = gameObject.GetComponent<ParentRangedEnemy>();
        scale = transform.localScale;
        pauseController = GameObject.Find("PauseController").GetComponent<PauseController>(); //Gets the pause controller

        if (melee != null) { //sets all the stats if this enemy is melee
            health = melee.getHealth();
            damage = melee.getDamage();
            speed = melee.getSpeed();
            attackRate = melee.getRate();
        }
        else if (ranged != null) //sets all the stats if the enemy is ranged
        {
            health = ranged.getHealth();
            damage = ranged.getDamage();
            speed = ranged.getSpeed();
            attackRate = ranged.getRate();
        }
    }
}

```

I then reused and altered these methods from ParentMeleeEnemy to make the ally move. I first added code in Update() which ensures there is always an enemy that the ally is targeting, so that it is always doing something and being useful to the player. To make the movement code work for the ally, I replaced all instance of it mentioning the player with the target, so that it would constantly move towards its next target.

```

void Update()
{
    if (target == null) {
        target = GameObject.FindWithTag("Enemy");
        print(target);
    }
    else
    {
        if (transform.position != target.transform.position) //makes sure the enemy isn't at the same location as the player
        {
            directionCalc();
            movement();
        }
    }
}

1 reference
private void movement()
{
    Vector2 pos = transform.position;

    if (directionVector.x > 0)
    {
        pos.x += speed * Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused;
        pos.y += speed * Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused; //calculates new location
        transform.localScale = new Vector3(scale.x, scale.y, scale.z);
    }
    else
    {
        pos.x += speed * -Mathf.Cos(direction) * Time.deltaTime * pauseController.unpaused;
        pos.y += speed * -Mathf.Sin(direction) * Time.deltaTime * pauseController.unpaused;
        transform.localScale = new Vector3(-1 * scale.x, scale.y, scale.z);
    }

    transform.position = pos; //sets new location
}

1 reference
private void directionCalc()
{
    directionVector = (target.transform.position - this.transform.position).normalized; //calculates the direction vector from the enemy to the
    direction = Mathf.Atan(directionVector.y / directionVector.x); //calculate the angle it should travel at
}

```

This worked fine.

Stage5Vid15

I then added the code to make them attack. Once again, I was able to reuse code from ParentMeleeEnemy due to the similar forms of attacking. I did this because it saves me time. I made sure to change it so that instead of colliding and damaging the player, it collides with and damages anything tagged with "Enemy". I made sure to get the ParentMeleeEnemy and ParentRangedEnemy components so that it can damage all enemies no matter the type.

```

private void OnCollisionStay2D(Collision2D collision) //every frame that there are collisions
{
    if (collision.gameObject.CompareTag("Enemy"))
    {
        //if it collides with the player
        GameObject enemy = collision.gameObject;
        attack(enemy); //passes in the enemy game object to the attack method
    }
}

1 reference
private void attack(GameObject enemy)
{
    melee = enemy.GetComponent<ParentMeleeEnemy>();
    ranged = enemy.GetComponent<ParentRangedEnemy>();
    if (attackTimer <= 0)
    {
        if (melee != null)
        {
            melee.damaged(damage); //damages melee enemy
            attackTimer += 1;
        }
        if (ranged != null) {
            ranged.damaged(damage); //damages ranged enemy
            attackTimer += 1;
        }
    }
}

```

This also worked.

Stage5Vid16

Finally, I needed the ally to be able to take damage.

I put this method in AllyScript, which I could copy straight from ParentMeleeEnemy. I did this to save time, both with testing and with development.

```
public void damaged(int damaged)
{
    health -= damaged;
    GameObject damageSprite = Instantiate(damageEffect);
    damageSprite.transform.position = transform.position;
    if (health < 0)
    {
        Destroy(gameObject);
    }
}
```

I did have to change OnCollisionStay2D in ParentMeleeEnemy so that it can also damage the ally. I did this by simply checking that it wasn't the player it collided with, and calling that objects damaged() script, before resetting the attack timer. The other option would be to rework the attack() method to accompany for allies as well as the player, however I decided against that because of how specifically tailored that method is to damaging the player, meaning it would take far more code than the solution I went with.

```
private void OnCollisionStay2D(Collision2D collision) //every frame that there are collisions
{
    if (collision.gameObject.CompareTag("Player"))
    { //if it collides with the player
        GameObject plyr = collision.gameObject;
        attack(plyr); //passes in the player game object to the attack method
    }
    else if (collision.gameObject.GetComponent<AllyScript>() != null)
    {
        if (attackTimer <= 0)
        {
            collision.gameObject.GetComponent<AllyScript>().damaged(damage); //damages player
            attackTimer += 1;
        }
    }
}
```

I did the exact same thing for EnemyProjectile, so those can damage the allies as well.

```
else if (collision.gameObject.GetComponent<AllyScript>() != null)
{
    collision.gameObject.GetComponent<AllyScript>().damaged(damage); //damages player
    Destroy(gameObject);
}
```

This worked.

Stage5Vid17

I then developed the code for the T-Rex power up script. I chose to colour the Hammer green, because this tells the player their Hammer will spawn a T-Rex. It will then instantiate a T-Rex at the location of the collision. I chose to make it instantiate at the collision point to add some risk to using it, because if you miss your shot, you miss out on getting a T-Rex, forcing the player to be careful with their shots.

```

using UnityEngine;

// Unity Script | 1 reference
public class PUTrex : MonoBehaviour
{
    private GameObject trex;
    PauseController pauseController;
    PowerUpController powerUpController;

    // Unity Message | 0 references
    private void Start()
    {
        powerUpController = GameObject.FindWithTag("PowerUpController").GetComponent<PowerUpController>(); //gets the trex
        gameObject.GetComponent<SpriteRenderer>().color = Color.green; //makes hammer colour green
        pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
        trex = powerUpController.getTrex();
    }

    // Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy") && pauseController.unpaused == 1)
        {
            print(1);
            GameObject dino = Instantiate(trex);
            dino.transform.position = transform.position; //makes the dino's position equal to the hammer
        }
    }
}

```

I also expanded the switch case statement to accompany for this power-up.

```

case 4:
    hammer.AddComponent<PUTrex>();
    break;

```

I tested the power-up, and it worked perfectly.

Stage5Vid18

Finally, I wanted to work on the hypnosis power-up.

First, I needed to alter a hammer with this power-up to do 0 damage, so it doesn't immediately kill weaker enemies instead of hypnotising them, as this may frustrate the player. I did this by putting a setter for damage in the Hammer script that sets allows you to change its damage even after it was shot by the player.

```

public void setDamage(int damage2){
    damage = damage2;
}

```

I then made the PUHypno script. For this power-up I coloured the Hammer purple to convey a mysterious, magical feel. I set the Hammer damage to 0, so that it doesn't immediately kill weaker enemies that you might want to hypnotise. When the Hammer collides with an enemy, it will add the AllyScript component to that enemy. This is the easiest way to turn it into an ally, because otherwise I'd have to rewrite the entire enemy script to factor in the potential of it turning into an ally.

```

public class PUHypno : MonoBehaviour
{
    PauseController pauseController;

    // Unity Message | 0 references
    void Start()
    {
        SpriteRenderer spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.color = new Color(200, 0, 200); //makes the hammer purple
        pauseController = GameObject.Find("PauseController").GetComponent<PauseController>();
        Hammer hammer = GetComponent<Hammer>();
        hammer.setDamage(0); //makes the hammer do no damage
    }

    // Unity Message | 0 references
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy") && pauseController.unpaused == 1)
        {
            collision.gameObject.AddComponent<AllyScript>(); //converts the enemy into an ally
        }
    }
}

```


I altered the AllyScript so that, if the ally was previously an enemy, it makes them purple, and destroys their previous enemy script. I have to destroy the enemy script so that it purely acts as an ally and not an enemy, and I made it purple to tell the player that enemy is hypnotised, otherwise they may still try to attack it and get confused at why it isn't dying. I also made it so that it would get all the stats from the enemy it once was so that it acts like that enemy, making it seem to the player that it is just that enemy that has changed team.

```
if (melee != null) { //sets all the stats if this enemy is melee
    health = melee.getHealth();
    damage = melee.getDamage();
    speed = melee.getSpeed();
    attackRate = melee.getRate();
    spriteRenderer.color = new Color(200, 0, 200);
    Destroy(melee); //destroys the enemy script, so it functionally is no longer an enemy
}
else if (ranged != null) //sets all the stats if the enemy is ranged
{
    health = ranged.getHealth();
    damage = ranged.getDamage();
    speed = ranged.getSpeed();
    attackRate = ranged.getRate();
    Destroy(ranged);
}
```

Lastly, I made it so that PlayerShoot would add the PUHypno component to the hammer when it needs to.

```
case 3:
    hammer.AddComponent<PUHypno>();
    break;
```

This successfully added the script to the enemy, however they wouldn't damage any enemies, and it would get very laggy if lots of allies were stacked on top of each other. This is because the enemies would be incorrectly tagged, and on the wrong layer, meaning they were all colliding with each other non-stop.

Stage5Vid19

I wasn't sure how to change the tag or the layer, though, so I had to look up how to change the layer of a GameObject in the Unity documentation. With the knowledge I gained from this I could figure out how to change the tag.

(Unity, n.d.)

I put this at the end of Start() in AllyScript to set it to the correct layer, and remove the enemy tag. This should correct the issues with incorrect collisions.

```
gameObject.layer = 7; //sets the object to layer 7 - the player layer
gameObject.tag = null; //removes the enemy tag
```

This, however, crashed the game and returned this error message.

```
[11:17:42] Tag: tag name is null or empty.
UnityEngine.GameObject:set_tag (string)
```

Stage5Vid20

To fix this I changed the tag line to this

```
gameObject.tag = "Untagged"; //removes the enemy tag
```

This stopped the crashing but didn't change that the hypnotised enemies weren't dealing damage.

This is because, due to how their collisions were set up, even after hypnotism and their layer changing, the layers they interact with don't change.

I ready up on the Unity Documentation for Rigidbody2D to figure out how to fix this.

(Unity, n.d.)

I created the following code which simply changes all the layers the game object interacts with. I knew that the layers that the T-Rex interacts with would work perfectly, so I copied those layers.

```
rigidBody.includeLayers = LayerMask.GetMask("Enemy","EnemyProjectile");  
rigidBody.excludeLayers = LayerMask.GetMask("Background", "Default", "Effects", "Hammer", "PowerUpGem", "Water", "UI");  
capsuleCollider.contactCaptureLayers = LayerMask.GetMask("Enemy", "EnemyProjectile");  
capsuleCollider.callbackLayers = LayerMask.GetMask("Enemy", "EnemyProjectile");  
capsuleCollider.forceSendLayers = LayerMask.GetMask("Enemy", "EnemyProjectile");  
capsuleCollider.forceReceiveLayers = LayerMask.GetMask("Enemy", "EnemyProjectile");
```

I also added this in case the hypnotised enemy would target an enemy that then gets hypnotised. This would cause an issue because the first ally would constantly chase an ally which you can't kill to get it to retarget.

```
else if (target.gameObject.CompareTag("Enemy") == true)  
{  
    target = GameObject.FindWithTag("Enemy");  
}
```

These both worked.

Stage5Vid21

I also wanted to make it so that some enemies can't be hypnotized. The Hydra has too much health to be fair to hypnotise as it would take way to long to be defeated, and if you hypnotise Zeus you wouldn't be able to defeat him, meaning you could never win the game. I added a new attribute called 'hypnotisable' in the enemy scripts and made a getter for it. I then altered the code for when a hypnotising Hammer collides with an enemy to get this value, and if it equals 1, it will hypnotise the enemy. This is a simple way to exclude certain enemies from being able to be hypnotised as to balance the game.

```
[SerializeField]  
private int hypnotisable;
```

```
public int getHypno()  
{  
    return hypnotisable;  
}
```

```
if (collision.gameObject.GetComponent<ParentMeleeEnemy>().getHypno() == 1 || collision.gameObject.GetComponent<ParentRangedEnemy>().getHypno() == 1)  
{  
    collision.gameObject.AddComponent<AllyScript>(); //converts the enemy into an ally  
}
```

This worked.

Stage5Vid22

Finally, whilst playtesting, I realised the speed power-up wasn't working in a lot of circumstances. If you dash before ever collecting the speed power-up it wouldn't change your speed. This is because it only ever increases your speed when dashing = 0, which I did to allow dashing to have a constant speed. However, dashing is a float, so it would very rarely equal exactly 0. The fix to this was simple, though, as all I had to do was make it <= 0, and that would fix it fully.

```
if (dashing <= 0f) {  
    speed = playerStats.getSpeed();  
}
```

Stage3Vid23

Test Table:

Test	Expected Result	Actual Result
Zeus' health <=0	Scene changes to Win Screen	Expected Result Stage5Vid9
Ally is created	The ally moves towards an enemy	Expected Result Stage5Vid15
Ally collides with an enemy	The enemy takes damage	Expected Result Stage5Vid16
Enemy collides with an ally	Ally's health attribute decreases	Expected Result Stage5Vid17
Enemy gets hit by a Hypno hammer	The enemy becomes an ally	Expected Result Stage5Vid19
Enemy gets hit by a T-Rex hammer	A T-Rex is instantiated	Expected Result Stage5Vid18
Enemy gets hit by an Explosion hammer	An Explosion game object is instantiated	Expected Result Stage5Vid14
Enemy collides with Explosion	The enemy's health attribute decreased	Expected Result Stage5Vid14

End Stage Testing

Beta Testing:

My first End Stage test was a full run-through of my game, in which I tried to experience every aspect of the game as intended to ensure they all work under normal circumstances.

FinalVideo

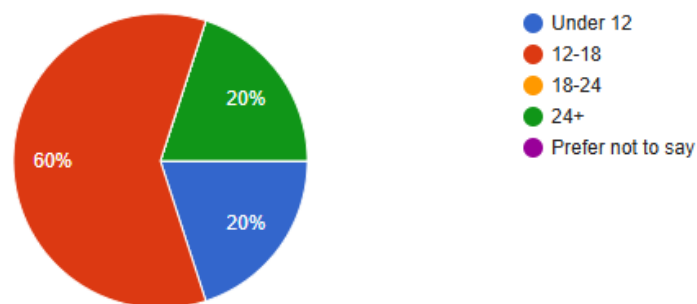
In this video, I was able to utilise every power-up, defeat every enemy, swap between every screen and beat the game in the above video without the game crashing, or experiencing any detrimental bugs.

The next stage of my beta testing consisted of providing a copy of my game to a group of my stakeholders and getting them to fill out a survey based on their experiences with the game, as beta testers. The survey will consist of them rating various aspects of the game, such as difficulty or graphics, to allow me to find where my game succeeds and where it fails. I can also compare this set of answers to my original survey in the Analysis section to see how well my game fits what the stakeholders originally wanted.

How old are you?

 [Copy chart](#)

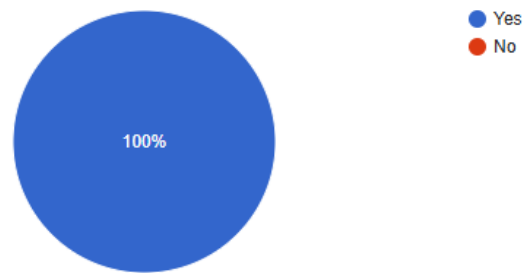
5 responses



My first question was to figure out the age ranges of who played my game. I wanted to get a range of demographics as beta testers, but still focusing on my target demographic. 60% of testers was within my target age-range, showing the results of this survey largely come from my target demographic, but there are also 40% of responders who are from other age-ranges, giving me a wide range of opinions.

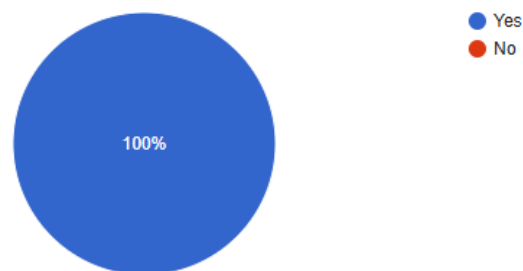
Have you played 'Gilbert the Godslayer'?

5 responses



Have you beaten 'Gilbert the Godslayer'?

5 responses

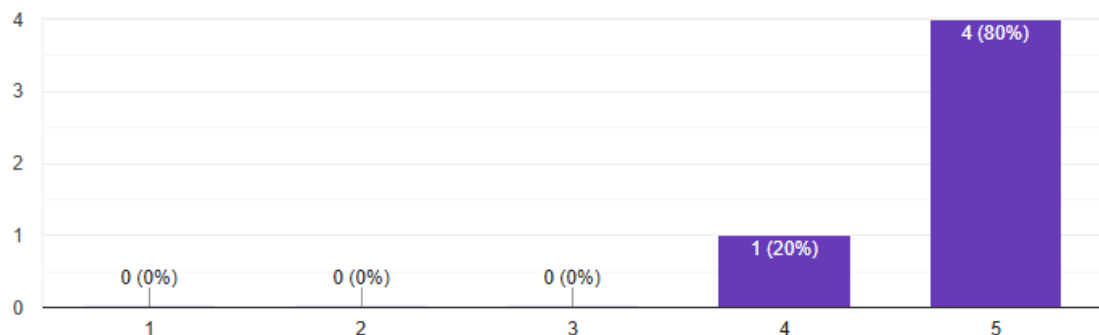


The next two questions were to ensure all the beta testers I surveyed have fully experienced 'Gilbert the Godslayer', validating their responses for the following questions. This also shows they were all able to get it to work, meaning it can run on a variety of devices with a varying levels of strength, as well as beat the game, meaning it is not too difficult or inaccessible to the average player.

How easily could you navigate the menus?

 Copy chart

5 responses

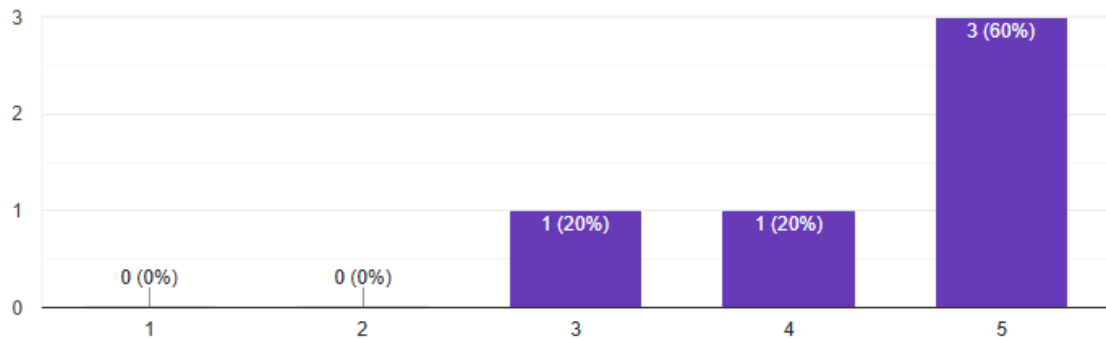


I then wanted to ask the beta testers about the usability of my game. I first asked them how easily they could navigate the menus, which is important so that they can quickly get into the game without confusion. 80% of responders gave it 5/5, with one other giving it 4/5, meaning I successfully designed easy-to-use menus which are usable for all demographics. This shows the plans from my GUI design were effective.

How quickly did you figure out how to play the game?

[Copy chart](#)

5 responses

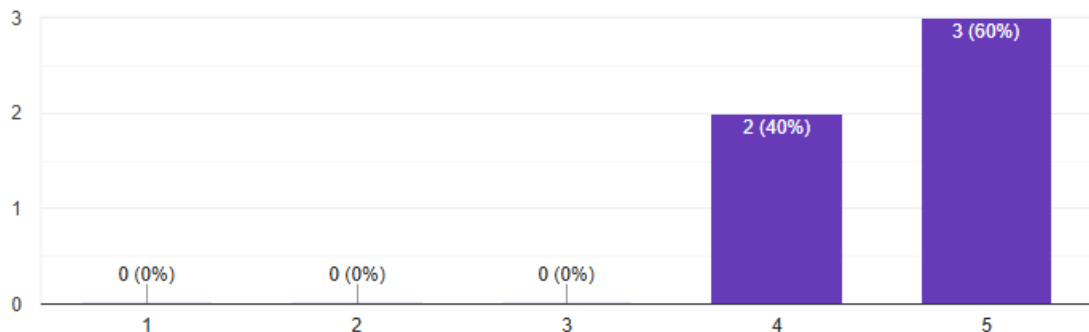


I then asked how quickly the players could figure out how to play the game. The average rating was 4.4/5, with the lowest being 3/5, meaning that I was able to convey the aim of the game very effectively to the players.

How well did the GUI elements in the gameplay screen (health bar, exp bar and the timer) display information to you?

[Copy chart](#)

5 responses

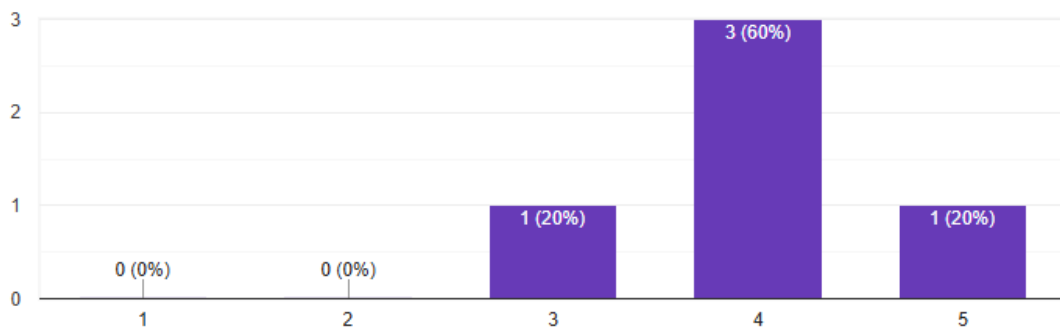


I wanted to check how well the UI elements of gameplay conveyed the information they were intended to. The average rating was 4.6, showing all the players quickly figured out what the UI elements were meant to represent. This is another example of my GUI designs also being effective in practice.

How well do enemies stand out from the background?

[Copy chart](#)

5 responses

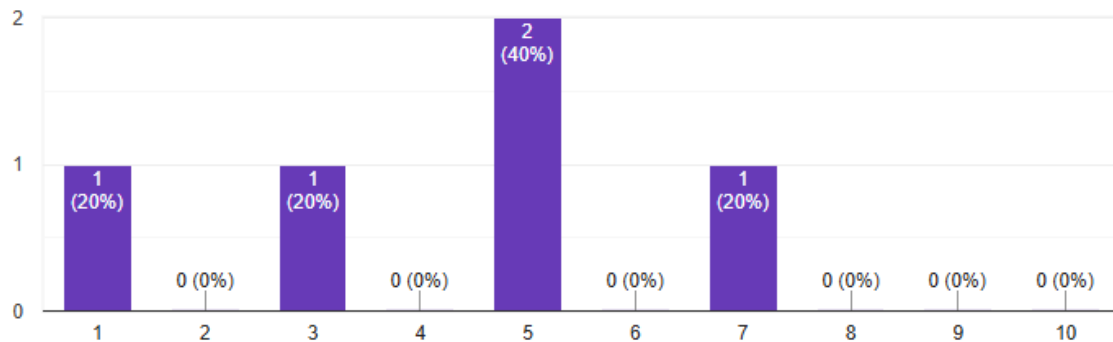


I also asked if the enemies stood out well from the background. This is important to allow the players to see what was going on. The average rating was 4, showing the enemies were distinct from the background, and everybody was much able to differentiate them.

Rate the difficulty

[Copy chart](#)

5 responses

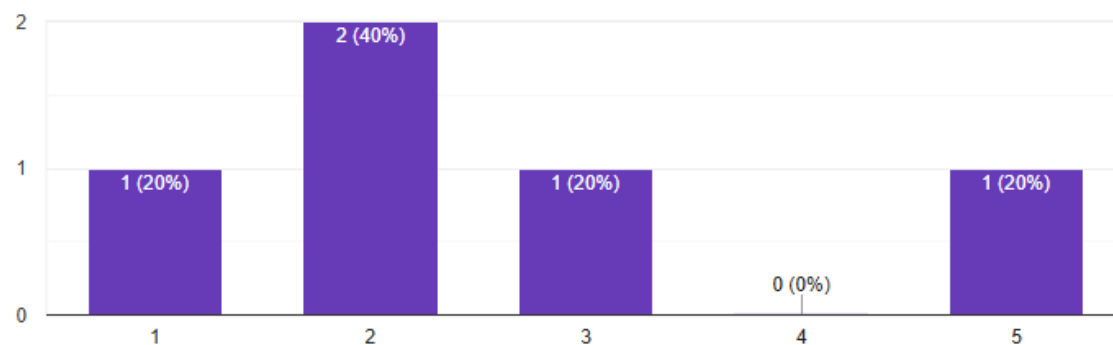


I then asked the beta testers to rate the difficulty of the game out of 10. The most common answer to this question was 5/10, with the average rating being a 4.2/10. This means the game was, on average, not too difficult, leaning towards being easy. One player struggled more than the others, but still didn't find it extremely difficult, and one found it very easy, meaning there was a range of experiences, however comparing the average difficulty of 4.2/10 to the stakeholder's desired difficulty of 8/10 from the original survey shows that I should have made it harder.

Rate the graphics

[Copy chart](#)

5 responses

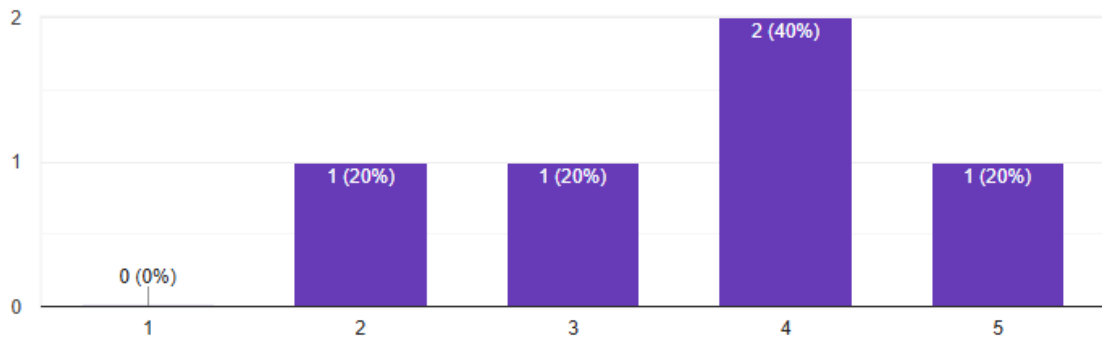


I then asked them to rate the graphics of the game. The average score for this question was 2.6/5, a very middling score. This is understandable as, in the original survey, it seemed the players wanted a pixel-art art-style, which is something I was intending to implement, however I prioritized other gameplay aspects over the purely visual art-style. This decision seemed to not be liked too much by most players, however one seemed to love the more cartoony art-style, rating it 5/5, showing I haven't fully put off my stakeholders from the graphics of the game.

Rate the enemy diversity

 [Copy chart](#)

5 responses

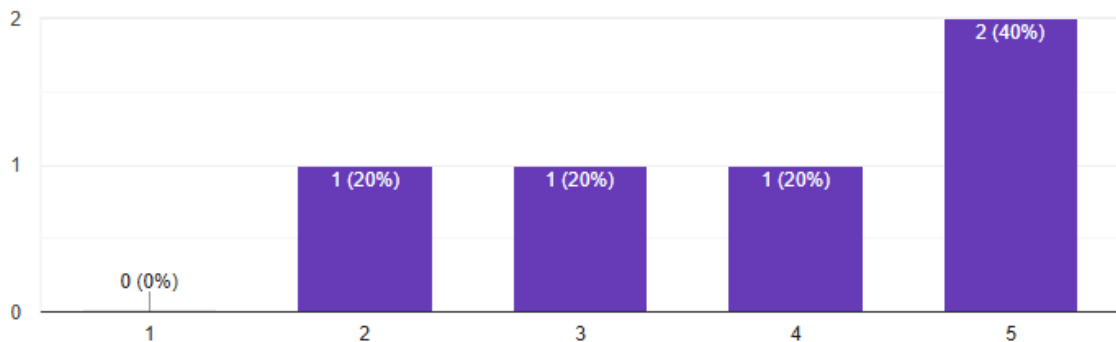


I then asked to rate the enemy diversity. From my original survey, this seemed to be quite important to the stakeholders, so it was something I tried to prioritize, ending with a total of 12 enemy types. The most common score was 4/5, with the average being 3.6/5, which overall is a good score, showing I succeeded in this aspect of the game. There were a couple ratings, however, that were on the lower end, though, meaning I could push this further in the future.

Rate the enemy designs

 [Copy chart](#)

5 responses



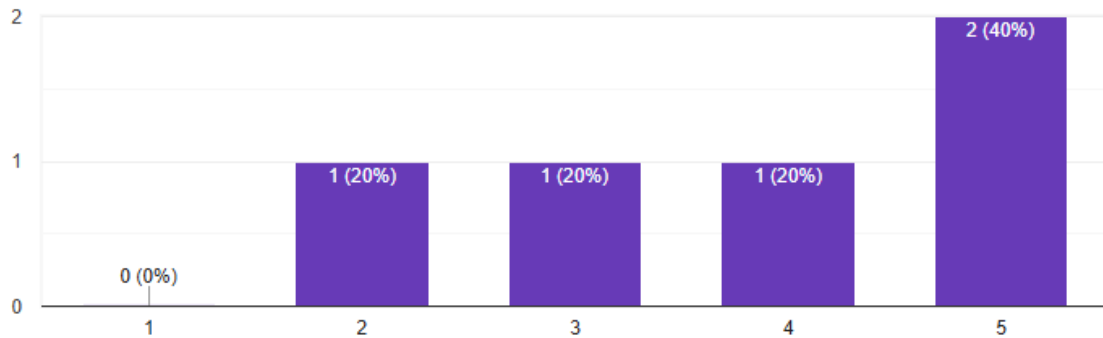
Following my previous question, I asked them to rate the designs of the enemies. The most common rating for this question was 5/5, and the average was 3.8/5, showing I successfully added endearing enemy designs which the player's enjoyed. This makes sense, as I followed their advice on how I should design enemies – mostly following the Greek aesthetic with some original designs.

One user noted: "My favourite aspect about this game was the creativity in the enemies, I adored the little crates that would follow you (they were cute), and I found the lighting chariot projectile to be intimidating when fired, it really did provide the necessary atmosphere for a final boss."

Rate the power-up diversity

 [Copy chart](#)

5 responses

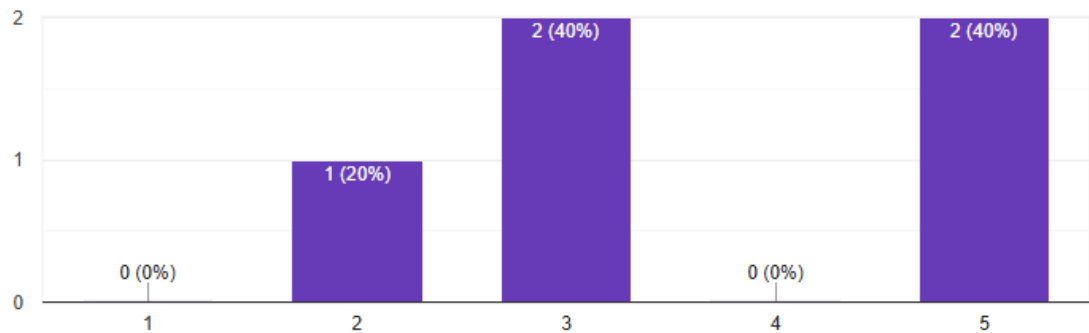


I then asked them to rate the diversity of the power-ups. The average rating was a 3.8/5, with the most common being 5/5. This meant I succeeded in creating a diverse range of power-ups, which hopefully created a wide range of gameplay experiences.

Rate the power-up quality

 [Copy chart](#)

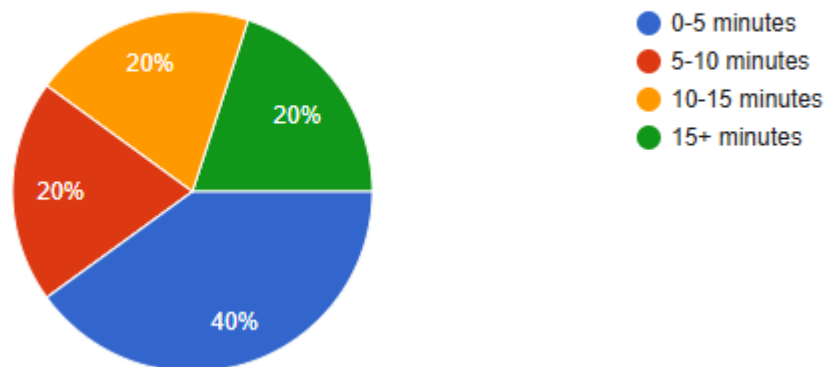
5 responses



I then wanted to see what the beta testers thought of the quality of the power-ups. This question got mixed responses, with an equal split between people loving them and liking them, with only one person rating it negatively. The average rating was 3.6/5, showing people on average enjoyed the power-ups, however the range of responses meant that I should have made more universally liked power-ups.

How long did it take you to beat?

5 responses

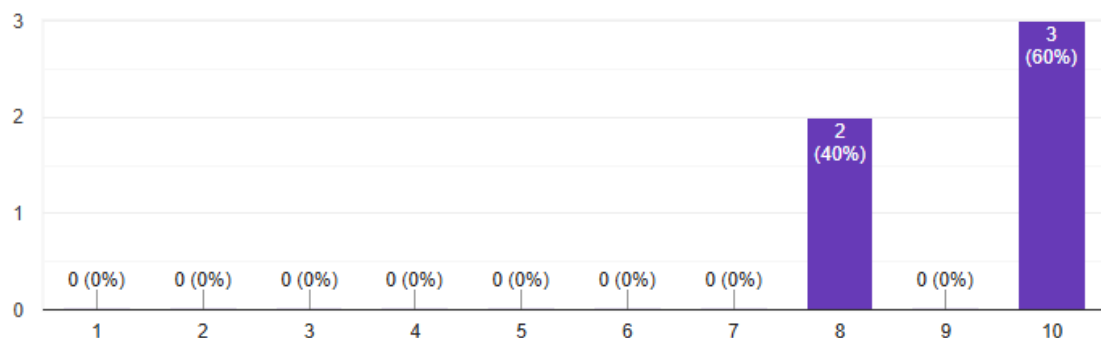


I then asked the beta-testers how long it took for them to beat. The most common time was 0-5 minutes, meaning they beat it in 1 run, reflecting the previously mentioned low difficulty, however there did seem to be a range in times, with one player taking over 15 minutes. From my original survey, it could be seen that the stakeholders wanted a run to last predominantly between 5 and 10 minutes, which only 1 person said they landed in.

How enjoyable did you find the game?

[Copy chart](#)

5 responses



I then asked how enjoyable people found the game. The average rating was 9.2/10, showing the beta testers really enjoyed the game. Despite some aspects not being to their liking, this shows that the foundation of the game is solid enough to create an incredibly enjoyable experience overall. This also shows I succeeded in creating a game which appeals to the audience I was going for.

Did you find any bugs or issues?

3 responses

No

very rarely a sudden drop in frames.

Visual bug when gilbert was walking, there was a jittery affect(sometimes).

yeah.

I then asked the beta-testers if they experienced any bugs or issues. One said no, one said yes (without explaining, meaning it most likely wasn't a game-breaking bug), and one said there were occasional lag spikes, and the movement could be a bit jittery at times. This shows that I didn't leave any massive bugs in my game, and it never seemed to fully break for anyone.

Any comments?

4 responses

Finally, I asked for general comments on the game. I got 4 responses:

"It would be cool if you could hold down the click instead of having to spam click the whole time"

"it would be cool if you held down to fire. and you could how a power up to up your fire rate. it would be cool if dashing into enemies knocked them back a little, and with fire step set them alight as well. would also be nice if the hammers did slight knock back."

"I thought that it was an enjoyable game, It had a surprising amount of content with a consistent theme and art style particularly with the characters, I think that the game was able to replicate a top down rogue like shooter (granted I don't have much experience playing this genre). The movement wasn't janky and I thought that the minimalist movement system complimented the gameplay style well, and the controls felt responsive to what I had intended to do. It was refreshing to see an game using all original assets as I thought it gave a unique visual identity for the developer. The power up screen was charming, and I think that the frequency at which the player gains powerups scaled nicely and complimented the increased spawn rates of the enemies, The power up screen was appealing to look at and it was intuitive. The powerups also had a decent impact on gameplay, I could definitely feel the effects powerups and I think that the weapon affects were clearly shown, and intuitive to what they do. I also think that the spawn rate of the enemies was overall balanced, I never felt overwhelmed with the amount of enemies as there was always a clear path out. I found that the enemy sprites were obvious at representing what they were supposed to be however with a certain charm to it. I had found the dragons design to be most interesting specifically, during one of my playthroughs I had noticed a scar on the dragons back, this had sparked my mind and plunged me into the world of the game, it got me asking questions about this character, I wanted to know more about the world, details like these moved this seemingly superficial game into a more 3 dimensional experience with a living breathing world. The two pickups in this game were concise to what they were

supposed to be and they felt natural when playing the game as I never found them to spawn too much, too little or in an inconvenient place. I thought that the execution of projectiles was done well, I found them easy to identify amidst the chaos, I appreciate that they aren't all the same either, and I thought that they never felt unfair when they had hit me as it doesn't move at an unreadable pace. In regards to the duration of the game, I thought that the runtime was appropriate and left room for replay ability as it didn't feel like a chore to get through. The final boss was also done quite well, I enjoyed the timer counting down his spawn, it was very suspenseful and gave a dramatic atmosphere to the entire game. In terms of the actual boss, his design (like the other enemies) was obvious and charming, I enjoyed fighting him in particular as he had a diverse move set (unlike other enemies), and the choice to spawn him alongside normal enemies was smart as it provided the correct amount of challenge to keep me engaged. His health bar wasn't too bloated, and his damage output only seemed fair for a final challenge. My favourite aspect about this game was the creativity in the enemies, I adored the little crates that would follow you (they were cute), and I found the lighting chariot projectile to be intimidating when fired, it really did provide the necessary atmosphere for a final boss. Another thing that I appreciated was the inclusion of otherwise non-orthodox powerups like the T rex (as opposed to fire or electric), I found that this promoted more than one playstyle and it provided (in my opinion) comedic affect as it was so baffling and absurd. Here is what I'd want to improve: -The lack of a menu and pause feature annoyed me as I was essentially committing to a run each time I press play. -It also would have been nice to have controller support as I am unfamiliar with mouse and keyboard controls. -I found it to be tiring when attacking, and by the end my hand was burning. -In regards the specific speed powerup, I thought that the change in speed could be a little bit too jarring when selecting it. -Although I loved the artstyle the background left a lot to be desired, the lack of terrain diversity and rather boring environment was unpleasant to look at and in my opinion confusing as what is was supposed to be. The change in size of the background details was an odd choice and didn't really make sense to me. - The void at the end of the map was also strange, i found it to be out of place and unfitting to the already displeasing map. -If there was a menu, I would also like a feature allowing me to check the powerups that I had selected so that I could focus my builds better. -I also thought that the enemy movement could have been more varied, granted there are exceptions, but the standard slow or fast walk got a bit repetitive. -I believe that it would be cool to have a scoring system so I can compare to previous runs and see my improvements, either that or just have a timer when the boss spawns to see how quickly you can defeat it. - Although unnecessary I thought it would be fun to have a cosmetic system, even just changing the colour of gilberts hat would be nice. - Despite their being implications of lore, I would have preferred there to be at least intro text to describe why gilbert is attempting to slay a God. -The lack of sound was also disappointing, I understand it is not a requirement but it would have added an extra level of polish. -It would also have been nice to be able to rebind my keys, as i didn't enjoy the space bar being dash."

"game of the year 2025"

These comments are a mix of advice and compliments, but overall positive. I can use the advice to decide on what I may change about my game in the future, but these comments also tell me that I have done a lot of things correctly. Also, once again, no major bugs have been mentioned.

Robustness Testing:

I tested for robustness in multiple ways. First, my stakeholders played the game on a variety of devices with a wide range in power, all of which could handle the game. Only one player who had a lower-end computer mentioned frame drops, however they also said it was quite rare (see “Did you find any bugs or issues” in the survey I sent to beta testers).

Bellow I attached examples of what the game looks like on my computer, and then what it looks like on one of the beta-tester’s. A different screen size just increases how much you can see without changing anything’s functions. This means the game will work on any screen size.



Another tester used an autoclicker whilst playing, meaning, due to there being no limit on the amount of Hammers you can fire in one second, around 20 projectiles were being fired per second. The game handled this fine, firing Hammers like normal without any lag during gameplay. The only issue this caused (outside of making the game very easy) was a lag spike when exiting the power-up menu, due to the game having to register so many clicks in a short amount of time. I recreated this in the following video (however, due to using more powerful hardware than the tester, I didn't experience the lag spike).

AutoclickerTest

I also tested having an extreme amount of enemies loaded without defeating any. This is not only a difficult situation to get in, meaning it is very unlikely to appear for any players, but it caused no issues. The following video shows what happens after not defeating enemies for around 100 seconds, meaning hundreds have loaded in and clumped together. You can see there is no lag, and the game functions fine.

ManyEnemyTest

Evaluation

M = Met **A = Almost Fully Met** **P = Partially Met** **N = Not Met**

For this part of my evaluation, I'm comparing [the user requirements I created on page 16](#) of the Analysis stage to my final product, to see how well I completed each feature. This both helps me see and compile what I have done, but also allows me to see how successful I have been at creating the game I envisioned. The description of these features are located in the [User Requirement table on page 16](#). I have used **green** to highlight any point where you may want to reference a separate page to help the readability of this table.

Feature	Met	Priority Level	Explanation and Justification
Start Menu	A	Desired	I had mostly completed the Start Menu. It had the title and Start Button completed perfectly, however was missing the high-score display I wanted to add. This is because I hadn't implemented a high-score system, meaning there was nothing I could display that wouldn't just confuse the player, so I decided to just leave it off. Looking at the GUI design I created for this in the Design stage on page 27 , I have recreated the core functionality of what I intended, containing all the features I wanted on the screen that I fully developed, leaving incomplete or unattempted features off so that the screen doesn't contain broken features that may confuse the player. It also has a different colour scheme and font to that which I described in the GUI design because I wanted to focus my attention on getting it functional before I went back and made it look good later, if I had time. I prioritised the functionality over the design because giving the player the ability to start the game is more important for the enjoyment of the game than having a nice-looking start screen when they load in, otherwise they can't play. You can see the final design of this screen on page 111 .
High-Score System	N	Nice-to-Have	I hadn't created any of the high-score system. Using the results from my original stakeholder survey on page 15, I determined that the high-score system would be a good feature to add, just further down the line. This is because the stakeholders seemed to enjoy the idea of this system, but over half of them didn't think it would massively improve the game, which is why I pushed it back to a later stage that I wasn't able to get do, due to me wanting to perfect the core features of the game, as that is what the users will spend the most time interacting with.
Start Button	M	Essential	I have created a button on the start screen which changes the scene from StartScreen to Gameplay. This was exactly what I said I wanted in the User Requirement table on page 16 , meaning it was fully a success. It is in the same location as I wanted it to be on the GUI designs (page 27) , because I determined that would be the most visible place to put it whilst leaving room for other items on the start

			screen. You can see the button functioning perfectly on Stage4Vid6 .
Movement System	M	Essential	Using the WASD keys, the player can move in the 4 cardinal directions. They can also hold down multiple at once, moving diagonally, to give them more freedom of movement for precise dodging and to help the game feel more responsive, due to them not needing to stop in one direction to move in another. This can be seen functioning in Stage1Vid2 . To ensure moving diagonally doesn't move them too quickly (due to the laws of Pythagoras), I made it so that, if they moved diagonally, they would slow down, otherwise the player would always be incentivised to move diagonally, which may feel weird to the average person.
Shooting System	M	Essential	This is a very important feature as this is what I decided to be the main way to attack enemies. When you click the screen during gameplay, a Hammer is instantiated and flies towards the location you clicked, before despawning after a short distance, just as described in the User Requirements on page 17 . See Stage1Vid12 for a video of this functioning. For aesthetic purposes and to increase the skill level needed to master the game, I made the Hammer's spin, both providing a cool animation, and giving the Hammer's a slightly shifting hitbox, making them slightly harder to hit. This animation can be seen in Stage4Vid10 .
Dash	M	Important	The dash is a movement feature that I added to allow the player to easily dodge enemy projectiles, and run away from fast enemies, making the game not feel brutally difficult, as the stakeholders wanted around an 8/10 difficulty (as seen from the survey on page 13). The dash functions exactly as intended, launching you forward, and giving you temporary invincibility. The user also can't move themselves during the dash as a way to balance it, as giving them full control may make it too powerful. Stage2Vid10 shows off the movement. I also added a timer which determines if you can dash or not, so that the player can't get permanent invincibility whenever they want, forcing them to be tactical with the dash. Lastly, I gave the player a different sprite whilst dashing, giving them visual feedback that lets them know they can't be damaged. Stage2Vid10 shows off the movement best, Stage2Vid12 shows the effect of adding the timer, and Stage2Vid13 shows the sprite change.
Enemies Spawn	M	Essential	Getting enemies to spawn randomly across the map is a crucial part of the gameplay loop. There needs to be enemies to provide a challenge, otherwise there would be no point in playing my game. I was successfully able to get enemies to spawn in a random location along the border of the player's vision. The spawner prefab has customisable enemy pools, a changeable spawn rate, and a delay until it starts spawning, allowing me to use it to create lots of different spawners that provide different gameplay challenges. I also created a variety of different spawners that spawn different enemies to add some variety to the

			gameplay. Stage2Vid34 shows the spawning working correctly. Stage2Vid35 also shows how the spawner works if you want it to have a delay. In FinalVideo , you can see the effects of all the different spawners, spawning different quantities at different rates and times around you, and how it makes for engaging gameplay.
Enemies Attack	M	Essential	To make enemies feel threatening, I needed to have them attack. As explained in the User Requirements section on page 18 , I also wanted there to be ranged and melee enemies that attack differently which gives variety to gameplay. Melee enemies will chase you down and deal damage if they touch you. In Stage2Vid12 , you can see the melee enemy chasing you, and in Stage2Vid17 , you can see it dealing damage to the player, eventually defeating them. I also wanted to make enemies to contrast the melee ones, and make the player approach the game differently. I wanted these to fire projectiles at the player if they were in range, and these projectiles would damage the player if they hit. This should force the player to try and dodge the projectiles to avoid taking damage. In Stage2Vid28 , you can see the fully functional ranged enemy, with it moving into range, and shooting at the player on a timer until it has been defeated. These two different attack patterns provided the variety of gameplay that I wanted, and make for a good challenge, which is why I counted this section as 'Met'.
Enemies can be defeated	M	Essential	To ensure the game is possible, I needed the enemies to be beatable. Therefore, I made it so that, if they collide with a Hammer, they take damage. If they take enough damage, they die, and the player gains exp, giving them an incentive to find enemies and engage in combat. You can see an enemy getting successfully defeated in Stage2Vid3 , and you can see the player gaining exp from defeating one in Stage2Vid18 .
10 Enemy Types	M	Essential	I wanted my game to have a minimum of 10 enemy types – 5 melee, 4 ranged, and a Boss enemy. I met this baseline. All enemies have completely different stats, forcing the player to play around each of them differently, which would make the gameplay far more fun and engaging. The first two were created in Stage 2, and you can see them in the videos I mentioned previously in the 'Enemies Attack' section above this . The 4 basic melee enemies are described in Stage 5 on page 122-123 , and the 3 other ranged ones are described on pages 124-125 . I will explain the boss in more detail in a later stage. Overall, I met the number of enemies I intended for, and gave them enough variety in stats to call this feature a huge success.
More Enemies	P	Important	In my survey for the stakeholders that I sent out in the Analysis stage on page 13 , I asked if they'd prefer me to add lots of enemies or lots of power-ups. The majority seemed to prefer having lots of enemies, so that is something I wanted to prioritise in my final stage. Whilst I didn't reach this stage, I did add 2 more enemies outside

			of my original 10, which I did to appease the stakeholders. These were the Lightning Chariot and the Defenders, which are exclusive to the Zeus boss fight. I wanted to implement these guys to add more of a challenge to that fight, making it a worthy final boss. These two enemies are showcased in the Zeus boss fight I did in Stage5Vid4. I only counted this as partially met though, as I would have liked to add even more enemies into the game, to increase variety even further, and also the extra enemies I added are exclusive to one boss fight, which means they won't be seen throughout the game.
Gaining a Power-Up After Enough EXP is Collected	M	Important	To make the player want to engage in combat and fight the enemies, I decided they should be rewarded with exp, which would eventually culminate in getting a power-up. This is how I decided to let the player get Character type power-ups (power-ups that affect the player as apposed to the Hammers), as it should feel like the player character is getting stronger. The amount of exp required to level up increases each time to give the game a sense of difficulty scaling, as well as ensuring the player isn't too weak at the start of the game. The level-up system is explained in Stage 2, between pages 63-64, and you can see it functioning on Stage2Vid21. I then altered it to give you a power-up in Stage 3, due to that being the stage based around power-ups, and you can see it working properly when I demonstrated it in Stage3Vid13. Due to completing everything I wanted from this criteria, I think it has been fully met.
Power-Up Gems	M	Essential	The other method for getting power-ups consists of collecting randomly spawning power-up gems. This is how the player would get hammer-type power-ups, as I think separating Hammer and Character types would make the power-up system more immediately understandable, and using this method for Hammer-type specifically prevents the player from immediately getting the very powerful power-ups too early, which would make the game too easy. I was able to get these to spawn randomly, anywhere on the map, as you can see in Stage3Vid2. This is because it should force the player to explore the map to find power-ups, which will make the game more interesting and challenging, instead of staying in one location. I did change one thing about how these work, however I'm still counting this stage as being met, due to it being to improve this feature instead of me simply not doing it. In the original description of this user requirement on page 19 in the Analysis section, I said you would get the power-up from the gem by throwing a Hammer at it. However, in the final product, I changed it so that you collect it by touching the gem. I did this because I think it both makes it increases the risk of collecting a power-up, as you need to reach the exact location it spawned on, even if it is swarmed by enemies, but also makes it so that if you want to hit an enemy that is behind the power-up gem, your shots don't get blocked by it, which some users may find frustrating. Once you touch it, it allows you to collect a new

			power-up, and it disappears. You can watch this in Stage3Vid9 .
10 Power-Ups	M	Essential	I decided I needed a minimum of 10 different power-ups to provide the game with enough build variety to make it replayable. Any less and I believe the game would get too repetitive, and no-one would want to play it more than a few times. I decided on having 5 hammer-type power-ups and 5 character-types, because if I split it up in a different way, the pool of power-ups the player could get would be too small and make every opportunity to get a power-up feel the same, removing any excitement from collecting power-ups. I will now explain the power-ups I chose to develop. I first designed a damage-up power-up. This is a staple power-up for most games I researched in the Analysis section, such as The Binding of Isaac on page 7, so I thought it should be a good addition for my game. It is a power-up designed to make the player feel more powerful, which is done by letting them take out weak enemies with one projectile, and allow them to take out stronger enemies quicker. If you watch Stage3Vid11, you can see it working. I made a health boosting one, which the player can use to either get more survivability if they aren't too good at dodging, or to heal a little if they are on the brink of death and feel like they need help. I think it is a good addition to give struggling players more options to survive. You can see it working in Stage3Vid12. I also made a speed-boosting power-up. This is to give the player an easier time dodging projectiles and get across the map faster, letting them find more Hammer-type power-ups, which are the more visually interesting power-ups for the player. This one is best seen in Stage5Vid23, as I didn't realise it wasn't properly functional until right at the very end. Next I made the passive healing power-up, which will heal you by 2 health every 3 seconds. This is to make it so that taking damage isn't too punishing and permanent, as that would make the game feel unfair and may make the player want to give up at the start of the game if they believe they've made it unwinnable. You can see this one working in Stage3Vid14. For the last character-type power-up I wanted to make something that feels more unique and flashy, because the other ones aren't particularly visible, which may make some players lose interest in these power-ups. I made the fire step power-up, which leaves a trail of fire that damages enemies that touch it, to fix this, as it is a visually interesting, cool power-up. It also encourages the player to take risks and dash through enemies to damage them, which will make the game feel more intense, which is something a lot of the stake holders wanted in the survey I did on page 12. You can see this power-up functioning on Stage3Vid18. I then coded the hammer-type power-ups. Something cool I decided each power-up would do is that they change the colour of the hammer so that the player can tell if their hammers are affecting the battlefield, and which ones should cause an effect. The first hammer-type

			power-up I developed was Fire. The Fire Hammer was designed to give the player another damage buff, and is also the most commonly triggering effect, due to it only affecting one enemy per hammer. It would just do damage over time, to increase the rate at which the player can defeat larger enemies, so they don't get overwhelmed. You can watch this one function in Stage3Vid21. After that I made the Electricity power-up. This would create a ball of electricity that would damage and stun any enemy that collides with it, as well as spawning another ball on the enemy that gets hit. This is because the player will need ways to deal with big groups of enemies as to not get overrun, because that would be a frustrating way to lose the game. Stage3Vid24 demonstrates this working. I then developed the Explosion power-up. This one will cause an explosion that deals big damage to anything it hits. This is a combination of doing a lot of damage, and helping with crowds, which is a unique combination between power-ups, giving the player a good reason to choose this one. I also thought it would make the game seem more exciting, with explosions going off everywhere, which would appeal to the target audience of 12-18 year olds. You can see this in Stage5Vid14. The next one I developed was the T-Rex power-up, which is a power-up that spawns a t-rex which helps you in combat. This was a power-up that I knew I wanted, even in the introduction on page 1 of the Analysis section, as it best describes the nothing-off-limits approach I wanted to take with my power-ups. This power-up was also mentioned in the comments of one beta-tester as something they enjoyed about the game – “Another thing that I appreciated was the inclusion of otherwise non-orthodox powerups like the T rex” – a quote that shows I was right about my initial beliefs that people would enjoy the t-rex power-up. You can see it functioning in Stage5Vid18. The last power-up I made was the Hypno power-up, which converts an enemy into an ally that works for you and fights enemies. I came up with this because it adds another layer of complexity to combat – instead of just shooting any enemy you see you might want to try and hypnotise a powerful enemy, but if you attack it too much before you hypnotise it, it will be at low health. I also made it so that some enemies – Zeus, his spawns and the Hydra – can't be hypnotised, otherwise the game would be either too easy, or unwinnable as Zeus can never die. You can see the power-up working in Stage5Vid21, and the unhypnotizable enemies in Stage5Vid22. Overall, I created 10 diverse power-ups which all function as intended, meaning I can call this feature met.
More Power-Ups	N	Desired	In the final stage, if I had time, I would have liked to add more than 10 power-ups, to make the game more diverse and fun to play. However, this wasn't necessary to add due to the minimum of 10 that I set being enough to make the game feel varied. I wanted to spend my time perfecting

			these base 10 power-ups, which meant I didn't have time to add any more.
Screen for Selecting Power-Ups	A	Desired	When you run into a power-up gem or level up, you are brought to the power-up selection screen. Here you click a button with the title of the power-up you want, and it gets applied to your character. I also created the pause controller, which contains a variable that, if set to zero, stops all movement and timers, creating the effect of the game being paused. This is so that you aren't able to be attacked whilst on a different screen, which would lead to you dying to something you can't defend against or see, which would be extremely unfair to the player, but it also ensures the player can't abuse the new screen to fully heal with the passive healing power-up, as that would make the game too easy. Looking at the GUI design I made for this screen on page 31 and comparing it to the final design I made in Stage 3 on page 87, you can see I recreated it very well. The only thing I am missing is a description of the power-up (which is why I counted this stage as almost fully met) which I excluded because the names of the power-ups give enough of a sense of what they do without me having to spend a long time creating a lengthy description for all power-ups. It would also be difficult to fit a whole block of text on the buttons whilst keeping this screen looking clean. You can see the functionality of this screen on Stage3Vid10.
HUD	P	Desired	In the gameplay screen, I wanted a HUD to provide useful information to the player on aspects of the game like their current exp, health, the time until the boss spawns, and when they can't dash. You can see the original design of the HUD in the GUI design I made for it on page 29. This design also shows the player's score, and an indicator pointing to a power-up gem. If you contrast it to the gameplay in FinalVideo, you can see it looks quite different, hence why I said it was only partially met. I did create the health bar, which accurately shows the player's current health. It is in the same location as the GUI design, but contains a bar that wasn't there originally, as well as text. This is so that, if the player wants to know their approximate health, they don't have to read to know it, and can instead just look at the size of the bar to know if they are in danger or not. You can see the bar functioning in Stage4Vid3. The exp bar was designed to show the player how close they are to levelling up. It doesn't have text, because exact numbers won't help the player, and will just confuse the player instead of giving them useful information. You can see it working in Stage4Vid4. I also created the timer, which displays how long it is until the boss spawns. I made this because it tells the player how long they have to prepare for the boss fight. It works perfectly, as seen in Stage2Vid29, however is just displayed as text, instead of being in a clock-like format that I originally wanted, as it looks cooler, and is more interesting for the player. I didn't make it look like this purely as a time-saving measure, as having it display as

			text makes it serve the exact same function, even if it looks different. The HUD was missing some aspects I originally wanted – the score counter, the dash timer, and the pointer for power-up gems. The score counter wasn't included due to me not developing the score system, which I explained previously, and the other features didn't provide enough value to the gameplay for it to be worth me prioritising them over other, more essential features. The time between dashes is short, so the timer would never be up for long, and may act as a distraction that blocks your view, and I made enough power-up gems spawn that you will find them naturally just by exploring without needing a pointer.
Boss Spawns after Timer	M	Essential	After 300 seconds in the Gameplay scene, the boss, Zeus will spawn. I did this by making the Boss spawner game object, which is the exact same as regular spawners, with the only thing it can spawn being Zeus, and its timeUntilSpawn attribute being 300. This meant that I didn't have to create and test any extra code for specifically this special spawner, because it has already been developed and tested, saving me time and allowing me to improve his boss fight. If you skip to around 8:50 in FinalVideo, you can watch him spawn, as well as see his boss fight. To make his boss fight interesting, I gave him an extra script – ZeusScript – which adds a variety of other attacks to differentiate him from regular enemies. This includes 2 different types of projectile attacks: a lightning bolt throw, which has infinite range, meaning he can attack the player from anywhere on the map, discouraging running away from him and forcing the player into combat, as well as making sense with Zeus' mythology, strengthening the Greek mythology theme the stakeholders enjoyed, as seen in the survey on page 16. He also has a high damage burst of lasers which can quickly kill the player if they aren't good at dodging, making his battle harder and ensuring the player needs to be skilled to beat him, otherwise he may feel anticlimactic and not be a worthy final boss. He can also spawn two types of enemies, one being the Lightning Chariot, and the other being a hoard of Defenders. These are designed to force the player to make quick decisions on whether to focus down Zeus, letting the minions put pressure on them, or spend time defeating the minions, and ignoring Zeus. This is another way I made the boss fight harder. Because I got the boss to spawn once the timer hits 0, as well as successfully implementing all the attacks I wanted to give him in the user requirements for this section, I would call this criteria met.
Victory Screen	A	Important	After you defeat Zeus, you are sent to the Victory Screen. On it, there is text saying 'YOU WIN' and a button which allows you to play again. You can see the final design on page 134, and if you compare it to the GUI design on page 31, you can see it is pretty similar. I kept the gold and white colour scheme, and the general layout of text in the middle, and buttons at the bottom, however I also added

			images of the player, Zeus, and a Crateward. This differentiates this screen from the other ones, making it seem special, which shows that it is an achievement to have beat the game. It also allows the player to reconcile on the journey they have had with my game. The major difference between the initial design and the final product is the lack of the Main Menu button, which would take you back to the Main Menu. I didn't include it in my final product, as it would mean I'm spending time developing something without a proper use, as there is no reason to go back to the main menu in my game, because it is missing the features that make it unique (such as volume controls). It is also missing the score display, due to that not being implemented yet. Because of these missing features, I can only call this almost met.
Loss Screen	A	Important	If the player's health goes below 0, they die, and are taken to the Loss Screen. You can see the final design on page 112, and if you compare it to the GUI design on page 31, you can see I did a decent job recreating it. Similarly to the Win Screen, I can only count this as almost met, due to it missing the exact same features for the exact same reasons. If you read the section above this, you will see the explanation.
Pause Menu	N	Nice-to-Have	I have not implemented any pause menu. I originally wanted to add one to give the player a way to halt the game if they needed to take a short break whilst playing without having to die and restart, which would become frustrating, however I didn't end up completing it. The game is short as it is – only 5 minutes per run – so the player will lose a max of 5 minutes of their time if they just leave half way through the run, and there were more consequential features that would be used more often that I would like to add instead, which I ended up spending my time on, because I would prefer to add to the gameplay instead of giving the option to stop the gameplay.
Pie-Chart Displaying Probabilities for Power-Up Activation	N	Nice-to-Have	This would have been a feature which allows the player to keep track of what power-ups they have, and how likely they are to trigger. I believe this information would have been useful for the player, especially if more power-ups are added, however, as it currently stands, when there are only 5 hammer-type power-ups in the game, it isn't very hard to keep track of, meaning I would have spent a lot of time on a feature that isn't providing overly useful to the player.
Pixel-Art Art Style	N	Important	When I asked my stakeholders what art-style they wanted in the stakeholder survey on page 9, 77.8% of them wanted it to be in pixel-art, so this is something I would have liked to be done. However, this is purely a visual feature, and would require a lot of time to create sprites for every enemy, projectile and effect. Instead, I made the decision to code power-ups and gameplay features, whilst using sprites I made quickly in an online drawing program. I then used the extra time to code additions to the

			gameplay, which I believe would benefit the fun of the game more than making it look good would.
Animations	P	Nice-to-Have	Originally, I wanted to add animations to the player and enemies because it would make the game feel more alive and visually engaging to the player. I ended up animating a few things, but not enough for me to call this stage a success. Firstly, if the player dashes, their sprite changes to a dash sprite, which acts as a short, simple animation. It also allows the player to see when they are invincible, which helps the player know what is going on. You can see this in Stage2Vid13 . Hammers also spin as they fly which gives the Hammers a slightly shifting hitbox, making them slightly harder to hit. This animation can be seen in Stage4Vid10 . These were the only animations I created, though, because I wanted to spend my time making the game more fun to play as opposed to fun to look at.
Victory Cutscene	N	Nice-to-Have	Once you defeated Zeus, I wanted a small cutscene to play, adding to the story of the game and to make your victory more rewarding. This was very low priority, though, due to it not affecting the gameplay at all, and most likely taking up a lot of time which I could spend improving many aspects of the game. This was one of the lowest priority features in my game, so I was never going to add it before something like pixel-art, which would drastically improve the look of the game.
Dungeon-like Setting	P	Important	When I surveyed the stakeholders on what the atmosphere of the game should be on page 12 , most people said they wanted it to be dark and gritty. I thought setting the game in a dungeon would work for that vibe. In the user requirement section , I said I wanted to add piles of bones and pillars around the map to give it that vibe, but this is something I didn't do. This is because adding obstacles would get in the way of enemies movement with their current system, and to get around this I would have to implement a path finding system, which would take a lot of effort for me to add what is mostly just an aesthetic feature to the map. Instead, I opted for making the background of the game look like the cracked rock floor of a dungeon to allude to this setting.
Sound	N	Nice-to-Have	I initially wanted to add sound effects when you get hit, hit an enemy etc, to add life to the game and make it feel more interesting to play. However, this feature wouldn't impact gameplay at all meaning it wasn't high on my list of priorities. Instead of creating these sound effects, I wanted to spend my time adding power-ups and enemies that actually change how the game is played.
Volume Controls	N	Nice-to-Have	If I added sound, I would need to add volume controls to allow the player to change the volume of the sound effects. This would allow my game to be more accommodating to players with sensitive hearing, as well as hearing loss. However, as previously explained, my game has no sound so there is no point in adding this.
Colour Blind Mode	N	Nice-to-Have	Another accessibility feature I wanted to add was a colour blind mode that changes the colour of the screen to be

			more accommodating to those with colour blindness. Adding this feature would make my game more accessible to a group of people who wouldn't otherwise easily play my game. However, this feature would only benefit a small group of people, whereas other gameplay features would benefit everyone who plays, so I decided to prioritise other features over this one, and never got around to completing this stage.
--	--	--	---

Maintenance:

I designed the game in Unity, a game development program. I chose this due to being beginner-friendly, and being able to easily allow your game to work on a multitude of platforms. It allows you to create a build of your game for Windows, Mac and Linux, meaning I can easily create a version that works for any of these operating systems. Currently, though, the build I released to the beta testers is only functional on Windows systems, which I chose due to it being the most popular, meaning as many people can test my game as possible without me having to distribute 3 different builds which may confuse the beta testers and lead them into downloading the wrong one which won't function on their device. However, in the future, I can easily create the different builds for each OS so anyone can play no matter what device they are using.

I also ensured the code would be maintainable by using sensible variable. Doing this throughout the code will make it far easier to understand what my code does, as you can look at the variable names to understand what stat may be used, or what aspect of the game is getting affected. Every variable is named based on the stat it relates to or what it does, and uses camel case, which makes it simple for me to see what is a variable, which will make debugging far easier.

I also added comments so that, if anything complex needs to be explained, you can read the comments I put throughout the code for a full explanation. This makes it easier for me to develop, as I will more easily understand what I did in the past, so I don't have to spend a long time figuring out how exactly a piece of code works, and it allows me to have other people work on my code to update the game, because they can understand what it does without me having to explain everything to them.

I also used indentation to make my code more readable. This allows me to see what parts of code are in an IF statement or a loop, which will allow me to traverse my code more easily. It also breaks up different sections of code, so I can more easily find a specific section in a script without having to meticulously look through the entire thing.

The build of the game is stored in two places. I have it downloaded on my laptop, meaning I can access it without internet from anywhere, as long as I bring my laptop. I also stored a backup on a Google Drive, which means, if something happens to my laptop, I can still access the file from any device that has Internet access. Using both means there should never be a situation where I lose all files relating to my game.

Further Development and Limitations:

If I continued development on my game, there are many things I would like to add. Firstly, I would like to take the advice from **two comments beta testers gave me on page 148**, and make it so that you can hold down the mouse button to fire, and implement a timer between shots. This is to balance the game, because otherwise it becomes way too easy if you download an autoclicker, as seen in **the stress test I did on 152**. It also makes it less physically straining to play for long periods, as spam clicking the mouse button for 30 minutes straight tends to cause hand pain. This would be very easy to implement, as I can reuse the code that stops you from dashing multiple times in the playerMovement script.

After that, I would like to finish the rest of the user requirements that I didn't complete. One limitation of my current game is the lack of a high-score system would add far more depth to the game, because it gives the player another goal to strive for – getting the high score. This would increase replayability, and promote risk taking for the sake of bettering their score, leading to more fun and intense gameplay. I didn't add it in yet due to me wanting to focus on perfecting the core gameplay loop, and ensuring the main goal of the game – collecting power-ups to get strong enough to beat Zeus - fully works before adding in the secondary goal of getting a high score. If the features that lead to defeating the boss are unfinished or not functional, the game won't be good enough to make the player want to get a high-score, meaning there was no point in adding it in the first place. To add in the high-score system, I would need to create a 'score' attribute, which will increase when defeating enemies. This wouldn't be difficult, as I can reuse a lot of the code that increases the player's exp when defeating an enemy. To save the high-score, I would probably need to create a separate text file which will store the player's current high-score, and would get overwritten when the player beats it. This is because there needs to be a way to store information so that the player's high-score gets saved even after they close the game. You can then read from the file whenever you need to get the high-score.

Another limitation is the simplistic, unprofessional graphics. This would have been fixed with visual features such as animations or the pixel-art art style, would improve the look of the game massively. The **beta-testers rated the graphics of the game a 2.6/5, as seen on page 146**, which is a rather low score, telling me there is a lot of room for improvement. Most of the stakeholders in the original **stakeholder survey on page 13** wanted pixel-art, so implementing that is sure to improve the graphics in a way they'll enjoy. Adding animations should add character to the game, and make it feel more alive, as well as being more visually interesting due to all the moving parts. To implement the art-style, I would have to design some sprites for everything using a pixel-art website which is specifically made to make pixel-art sprites, because the features added to the website to help with pixel-art design will speed up creation. If I added more animations, it wouldn't be by making new sprites and looping them, as that would take an extremely long time. Instead, I would add code into the enemy script that changes the scale of the enemy to make it stretch and squish, which would create the appearance of movement without me having to create lots of new sprites.

A final limitation of the game that I would want to fix is that this game is solely for mouse and keyboard, meaning someone who doesn't have access to these devices, or is simply less adept with a mouse and keyboard, can't play. Adding controller support was also mentioned by a beta-tester in their [comment on page 150](#), showing it is something people want. To implement this feature, I will just need to make it so that the code that takes in the player's inputs will also react to certain controller inputs. For example, I would make the left joystick correlate to the player movement code, the right joystick to aim the crosshair, and make a button shoot and dash.

Once I have fixed the limitations of my game, I would want to continue adding enemies and power-ups. Adding more unique enemies is something that the stakeholders clearly wanted, as seen in the [survey on page 13](#), and should create more unique combat situations that may create memorable experiences for the player. The more enemies there are, the more there is to see, too, so the player will be encouraged to keep on playing to make sure they see all the enemy designs. Adding more power-ups will also add replay value, and overall make the game more interesting. This is because the player can play multiple runs, each one focusing on a different set of power-ups, giving them a unique experience every time. Adding these features would be quite simple – I designed the `parentMeleeEnemy` and `parentRangedEnemy` scripts to act as a customisable base-line that can be quickly changed to create new enemies with ease, and the system I made to add power-ups is also easy to add to, meaning all I have to do is develop the code for the script.

Usability

I added many features to make the game more usable to the player:

- I made the buttons very large, with text on it to tell you their function ([see pages 112 and 113 for the Start Screen and Death Screen examples](#)). This should make their use extremely clear to the player, allowing them to play the game without confusion on how to begin. The large size should also make them very hard to miss, to ensure people with poorer vision can still find it.
- When the game switches to the Power-Up screen before you can choose a power-up, an effect pops up that reads 'Power Up Time' to convey to the player that they need to select a power-up. This is to ensure that they don't accidentally click on a power-up without reading it because the screen switched right as they were about to click. It also makes sure the player isn't confused at what this screen is that just appeared, as it tells them it is to do with power-ups.
- The HUD features in Gameplay were designed to provide the player with information on what is going on, without blocking the screen. I think I was successful in this. Looking at [FinalVideo](#), you can see that the HUD doesn't take up much space, due to it being put in the corner, which is important because it lets the player see as many enemies as possible, allowing them to plan out the best course of action. It also conveys information well. It is noticeable that, when you take damage, the health bar decreases, and it is similarly noticeable that, when the exp bar fills up, you gain a level and get a new power-up. This means that the player will be able to associate these events with the bars to understand their purpose without needing to be told.

However, there are a few issues:

- The main issue with usability is that the gameplay is never explained. The player is expected to figure out that they need to survive, and then defeat the boss, despite it never being told to them at any point. If they don't figure this out, they may run away from Zeus and never engage with him in combat, leading to this game lasting nearly forever without the player experiencing victory, which may make the player feel unsatisfied with the game or think it is bugged. This would be easily fixed, however, with a quick tutorial when they play the game for the first time, which would tell them their goal, giving them clear instruction on what to do.
- Another issue is that the controls are never explained. The player currently has to figure out that the WASD keys, space bar, and mouse are all crucial for gameplay, and may feel stuck if they never interact with any of these buttons. Whilst the mouse and WASD keys are an industry standard that frequent gamer may instinctively try, less experienced players may not know to try to use these, which is an issue. I can fix this by putting a control scheme on the main menu, so they immediately know what keys they'll need once it loads. I can put it on the Gameplay screen, too, so they can find the exact key they need in the heat of battle with ease.

Overall, I do think I successfully made the menus very usable, however the players may be a bit confused at what to do in terms of gameplay. However, when I surveyed the beta-testers on the usability of my project on pages 144-145, they seemed to think it was very usable. The average rating for 'How easily could you navigate the menus?' was 4.8/5, showing they were incredibly easy to understand. The average rating for 'How quickly did you figure out how to play the game?' was 4.4/5, which means that, despite the lack of instructions, the beta-testers could still figure it out easily enough. Lastly, 'How well did the GUI elements in the gameplay screen (health bar, exp bar and the timer) display information to you?' got an average answer of 4.6/5, showing the GUI elements were successful at telling the players what they need to know. The results of this survey were very positive in terms of usability, which makes me confident in saying that I was able to successfully make my game usable.

Conclusion:

In conclusion, I am happy with the game I created and feel like this project has been successful. I was extremely ambitious with my original goals, and whilst I wasn't able to complete all of them, I was able to get the features I deemed essential working exactly as I intended. The gameplay was fun and exciting, and the menus were functional. The beta testers enjoyed the game a lot, with an average rating of 9.2/10 when I asked them how enjoyable the game was, as well as having left a lot of positive comments that provided good feedback, as well as complimenting the game. I think I correctly catered the game towards my stakeholders, as well as ensuring anyone outside of my target audience can still enjoy it, and I successfully made it so that it can run on a wide range of devices with different strengths. Whilst there are features I still want to add, I am very satisfied with the game in its current form.

Citations:

(n.d.). Retrieved from

<https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Collider.OnCollisionStay.html>

Destroy(this) not working properly - Questions & Answers. (2013, June 22). Retrieved October 4, 2025, from Unity Discussions: <https://discussions.unity.com/t/destroy-this-not-working-properly/74052>

freeCodeCamp. (2025). Learn Unity - Beginner's Game Development Tutorial. Retrieved September 21, 2025, from YouTube:

<https://www.youtube.com/watch?v=gB1F9G0JXOo&t=3178s>

Modding by Kaupenjoe. (n.d.). *Player FOLLOWING THE MOUSE in Unity.* Retrieved from YouTube: https://www.youtube.com/watch?v=2DHy_l4Ffe0&t=131s

Raspberry Pi Foundation. (n.d.). *Update the text in a TextMeshPro GameObject.* Retrieved from Raspberry Pi Foundation: <https://projects.raspberrypi.org/en/projects/unity-update-textmeshpro>

Scripting API: Mathf.Sqrt. (n.d.). Retrieved September 21, 2025, from Unity - Manual: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Mathf.Sqrt.html>

Uni#. (n.d.).

Unity - Scripting API: Input.mousePosition. (n.d.). Retrieved September 26, 2025, from Unity - Manual: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Input-mousePosition.html>

Unity. (n.d.). *create an object inside a canvas.* Retrieved from Unity Discussions: <https://discussions.unity.com/t/create-an-object-inside-a-canvas/734705>

Unity. (n.d.). *GameObject.layer.* Retrieved from Unity Documentation: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/GameObject-layer.html>

Unity. (n.d.). *Rigidbody2D.* Retrieved from Unity Documentation: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Rigidbody2D.html>

Unity. (n.d.). *SpriteRenderer.* Retrieved from Unity Documentation: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/SpriteRenderer.html>

Unity Technologies. (n.d.). *Collider.OnCollisionStay(Collision).* Retrieved from Unity Documentation: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Collider.OnCollisionStay.html>

Unity. (n.d.). *Transform.Rotate.* Retrieved from Unity Documentation: <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Transform.Rotate.html#:~:text=Use%20Transform.,world%20axes%20or%20local%20axes.>

zombiegorilla. (n.d.). *[Unity 4.3] How to change the opacity of a 2D sprite ?* Retrieved from Unity Discussions: <https://discussions.unity.com/t/unity-4-3-how-to-change-the-opacity-of-a-2d-sprite/526034>